

The judge is the bottleneck

I ran every test and evaluation, read what DeepSeek and others have published on judging and long stories, and ran one new experiment. The short version: Versatile writes fine, but **the part that grades the writing is nearly blind**. Every improvement you try is graded by that part. Until it can see, you can't tell whether anything else helped.

Health: green

3,234 unit tests pass, 0 failing. The flaky group-chat test is fixed. Typecheck and policy clean.

Benchmark fixed

It reported success while failing all 28 cases, and it let a model grade itself. Both are fixed; honest scores dropped from 8.7 to 8.0 and exposed a 1/10 answer (section 2b).

New result

Reading the judge's *probabilities* instead of its single number: separation went from 0.77 to 0.91 (1.0 = perfect).

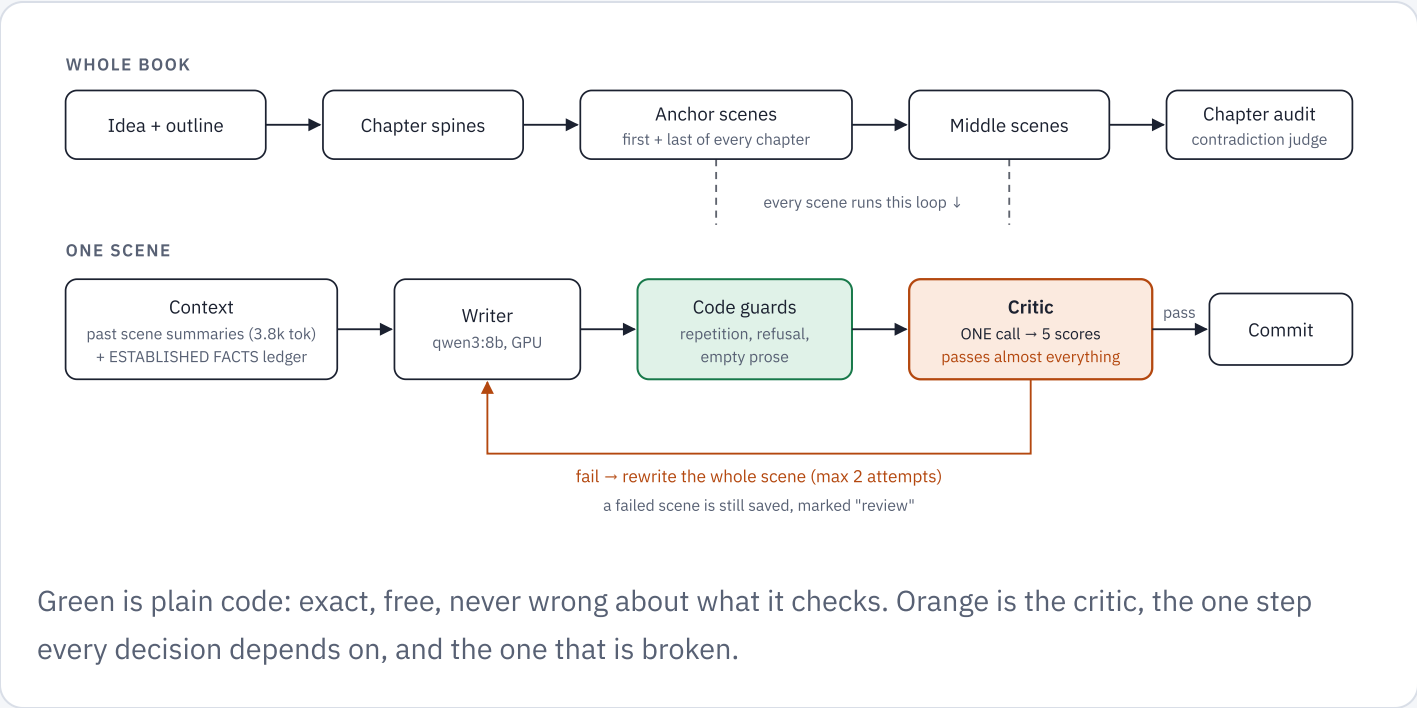
Real-scene test: gate misses subtle flaws

On 78 real scenes it caught none of the problems independent reviewers agree on (section 5i). On *planted* flaws it caught 133 of 145 (5c–5e). Against masterpieces it fails 11 of 12 (5j): it rewards LLM style, not quality. The fixes are in section 7.

1. How Versatile writes a book today

Before fixing anything, here is the machine. The book level runs top to bottom: outline, then spines (one plan per chapter), then scenes. Scenes are **not** written in reading order. The first and last scene of every chapter (the "anchors") are written first, then the middles fill the gaps.

Every scene goes through the same small loop, shown in the bottom row.



2. What I ran today

CHECK	RESULT	WHAT IT MEANS
Unit tests (vitest)	3,255 / 3,257	Day one: 1 flaky failure (useGroupChat.test.js timing out under load), since fixed. 26 Sep: 3,255 passed, 0 failed, 2 skipped.
Typecheck, policy	clean	No type errors, no policy violations.
eval:rag , eval:scene	too easy	Semantic retrieval scores a perfect 1.000 on single and cross-topic queries. A test that is always perfect can't show whether a change

CHECK	RESULT	WHAT IT MEANS
		made things better or worse. Noise queries sit at 67%. Metadata-only retrieval is weak (MRR 0.325), which is why the blend relies on embeddings.
pipeline:quality	6 / 6 pass	Scores 6.3–7.8 on the fixed regression cases. No regressions.
pipeline:benchmark	fixed · 28 / 28	First run: 0 of 28, because phi4-mini:3.8b wasn't installed, and the script still reported success. Model installed and two bugs fixed. It now grades honestly and exits with an error when it should. Section 2b.
pipeline:drift , pipeline:active	sample data only	They ran on the bundled 16-record sample, not real run history, so "no drift" says nothing about your runs.
Live: gate sensitivity (earlier runs, re-read)	fixed, re-measured 26 Sep	<i>Day one:</i> the old critic passed 40 of 40 deliberately broken scenes, and the focused one often blamed the wrong dimension (a scene turned into summary failed on "voice"). Today (5n): the focused critic is the default. A summary never touches voice (0 of 60). Only 3 of 145 planted defects fail a scene for the wrong reason.
Live: new probability probe	0.77 → 0.91	Section 5. Same model, same prompts; only the way the answer is read changed.

2b. The benchmark that graded itself

You asked me to install the missing model and retry. The retry "passed" with high scores, but those scores were wrong in an instructive way. The benchmark had **two** bugs. Installing the model only hid the first one.

BEFORE



AFTER

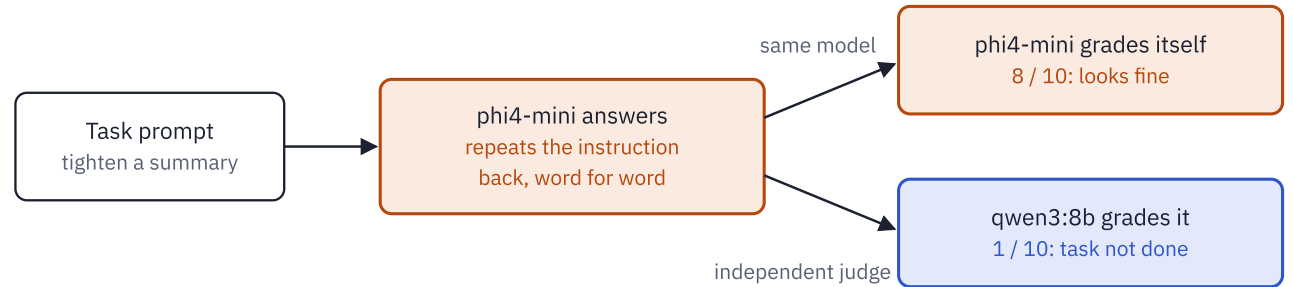


the answering side is unchanged: only who grades, and what "success" means, moved

The grading fallback read the same setting as the model under test, so without cloud keys a model always graded itself. The exit code ignored errors completely.

RUN	QUALITY-REGRESSION (6)	TASK SUITE (22)	EXIT
Before: model missing	0 done	0 done	0 = "success"
Model installed, phi4 grades itself	avg 8.1 · range 7.3–10	avg 8.7 · range 7.3–9.7	0
Fixed, qwen3:8b grades	avg 6.7 · range 5.5–8	avg 8.0 · range 1–9.8	0, and correct
Fixed, model missing (negative test)	0 done	0 done	1 + reason printed

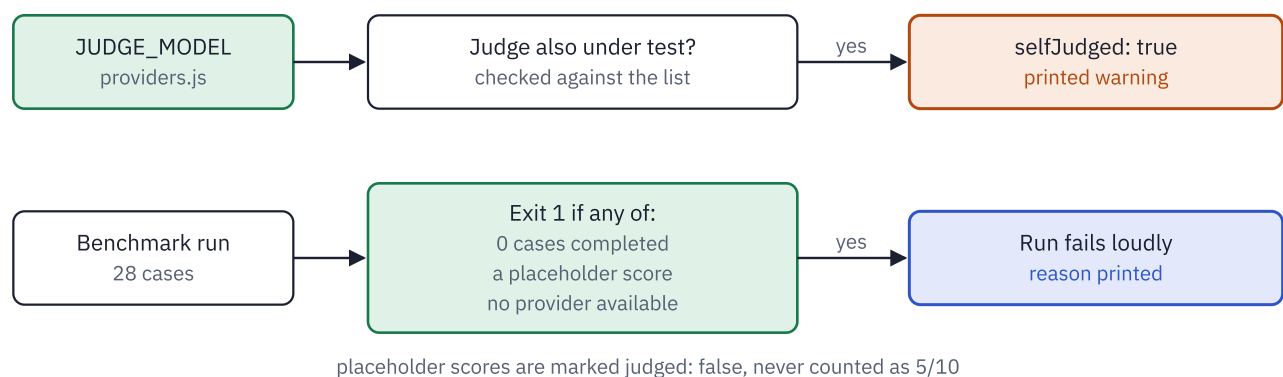
The 1/10 is the proof. On `polish-business-summary` , phi4-mini didn't summarise anything. It repeated the instruction back ("Tighten this executive summary to half the length..."). Grading itself, it had given that kind of answer 8/10. The independent judge wrote *"No revision provided, task not completed"*. Same lesson as the scene critic: **a judge that shares the writer's blind spots can't see the writer's mistakes.**



Same empty answer, two graders. Grading itself, phi4-mini missed the mistake it had just made. An independent model called it "task not completed".

What changed in the code

- `providers.js` : a new `JUDGE_MODEL` setting (defaults to `qwen3:8b` locally). Every judge gets a `selfJudged` flag, computed by checking it against the list of models under test.
- `judge.js` : the judge runs with thinking off (`think: false`). Placeholder scores from a failed or missing judge are now marked `judged: false` instead of passing as real 5/10s.
- `evaluate.js` : prints who grades and warns on self-grading. It exits 1 when a model completes 0 cases, when any score is a placeholder, or when no provider is available.
- `benchmarkJudge.test.js` : 5 new tests. The first draft assumed a Groq judge is never a contestant, and the flag proved it wrong: every remote default judge (llama-3.3-70b, gpt-4o-mini, Claude Haiku, gemini-flash) is also benchmarked by its own provider. The README now says to pick a `JUDGE_MODEL` outside the contest.



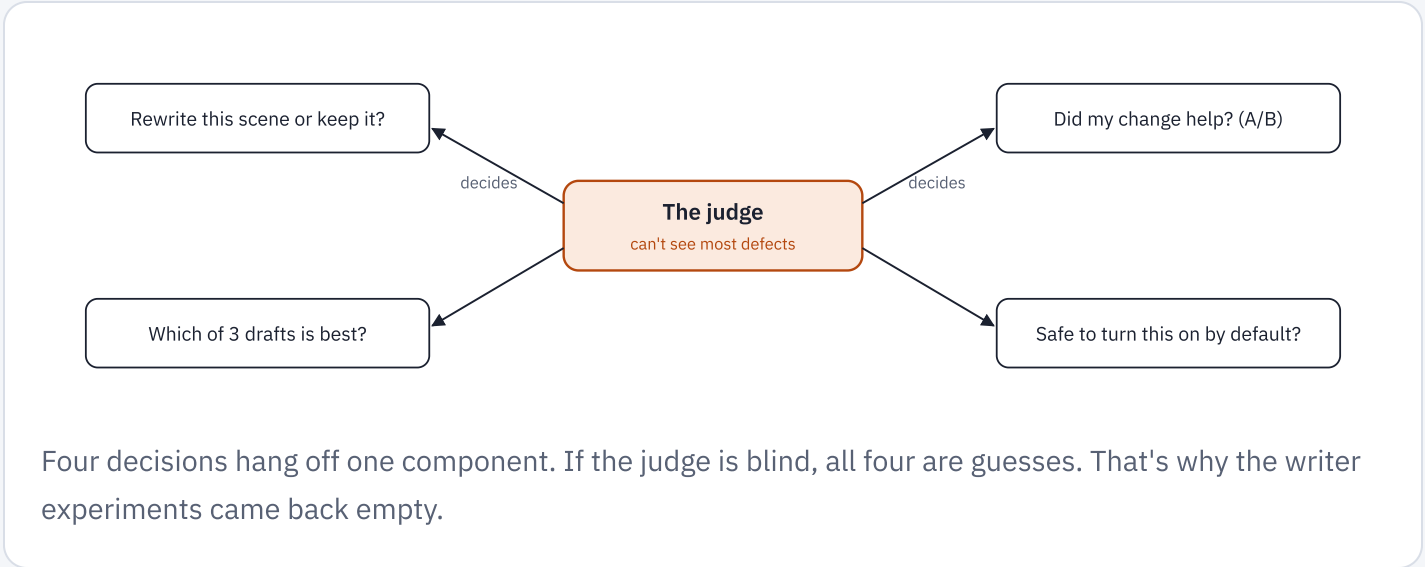
Two guards around the benchmark. One flags a judge that is also a contestant. The other turns a run with nothing real in it into a failure, so CI can no longer read "0 of 28" as success.

3. Where you stand: three things work, none proven better

Over the last week you built and measured three real mechanisms. All three work mechanically. None has been shown to make the book better.

CHANGE	DOES IT WORK?	DID THE BOOK GET BETTER?
Critic reads the whole scene (it used to miss the last quarter)	yes	can't tell the critic still gives 8/10 to everything
Writer gets 3.5× more memory of earlier scenes	yes, and it costs no extra time	not significant p = 0.125
Chapters linked by an ESTABLISHED FACTS list	yes	trended worse +0.67 contradictions/scene, p = 0.25

The pattern: every "did it help?" question goes through a judge. When the judge can't see defects, every answer comes back "no difference". So the next move isn't a fourth writer feature. It's fixing the ruler.



4. What the research says, in plain words

I looked for work that solved *our* problem: a weak judge, small local models, long stories that must not contradict themselves. Each entry says what they did and what we take from

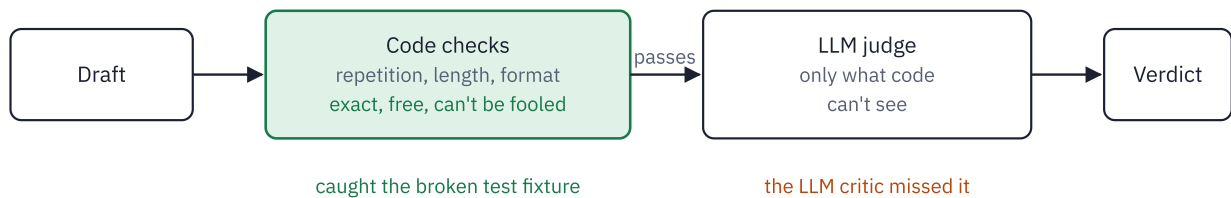
it. The DeepSeek work comes first because you asked. Its most useful ideas are about **judging and checking**, not model architecture.

DeepSeek

DeepSeek-R1 2025 · arXiv 2501.12948

They taught a model to reason using rewards that **code** can check (is the answer right, is the format valid) instead of a model's opinion. Code rewards can't be flattered or fooled.

For us: move every check you can out of the LLM and into code. Your repetition guard already proves this works: it caught my broken test fixture instantly while the LLM critic missed it.

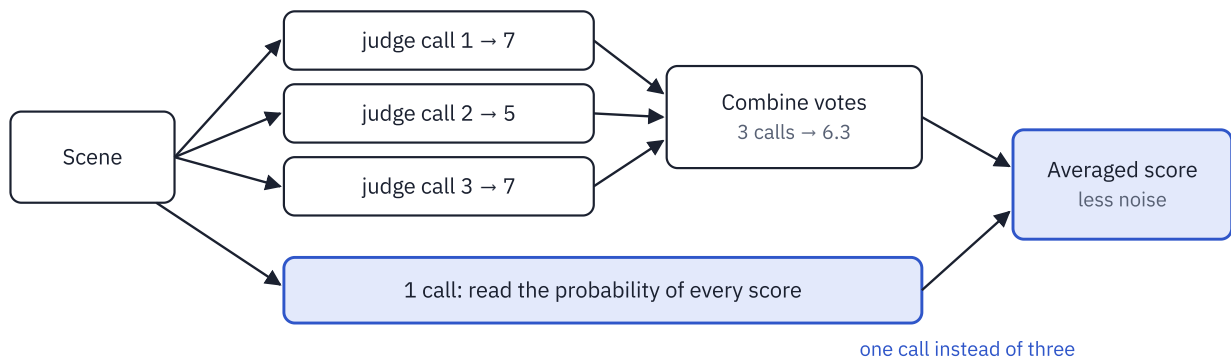


DeepSeek-R1's rule applied here: whatever code can check is checked first, exactly and for free. The model is kept for what code can't see. The repetition guard already caught a broken fixture that the LLM critic missed.

DeepSeek-GRM (SPCT) 2025 · arXiv 2504.02495

Their judge first writes the rules that matter for *this* answer, then critiques, then scores. They ask it several times and combine the votes. More votes, better judgments.

For us: voting is the idea. Section 5 shows a cheaper way to get the same effect: read the model's probabilities, which is the average of infinitely many votes in one call.

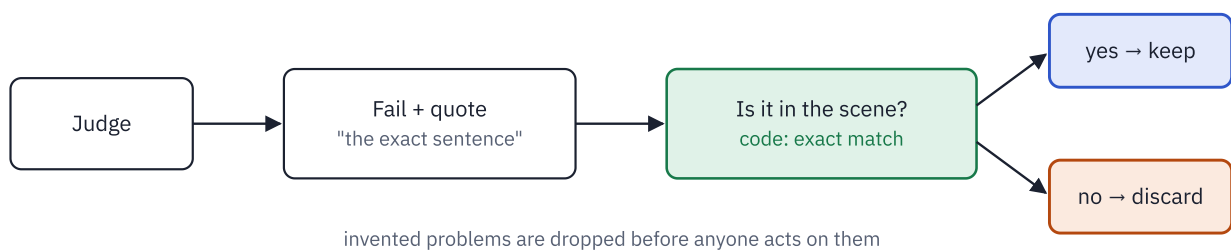


DeepSeek-GRM asks the judge several times and combines the votes. The probabilities from one call hold the same information, which is the cheaper route tested in section 5.

DeepSeekMath-V2 2025 · arXiv 2511.22570

The checker must point at the exact problem, and a second checker verifies that the problem really exists. This cut down "hallucinated issues", where the checker complains about something that isn't there.

For us: every failure must quote the bad sentence, and code checks that the quote is really in the scene. This targets the 2-in-8 clean scenes the focused gate wrongly fails.



DeepSeekMath-V2's "prove it" step. A complaint only counts if its quote is really in the scene, and plain code does the checking.

DeepSeek-V3.2 / V4 2025–2026 · arXiv 2512.02556, 2606.19348

Mostly about the model itself: sparse attention, a compressed memory cache, 1M-token context. Built into the model, so we can't copy it on an 8 GB GPU.

For us: the lesson "reading is cheap, so read a lot" is already in your wider continuity budget, and your own probe confirmed it costs nothing.

Inside DeepSeek's model

- sparse attention
- compressed memory cache
- 1M-token context
- can't be copied to an 8 GB GPU

What we can take: read more

before

1x

now

3.5x (3.8k tokens)

writer's memory of earlier scenes · no extra run time

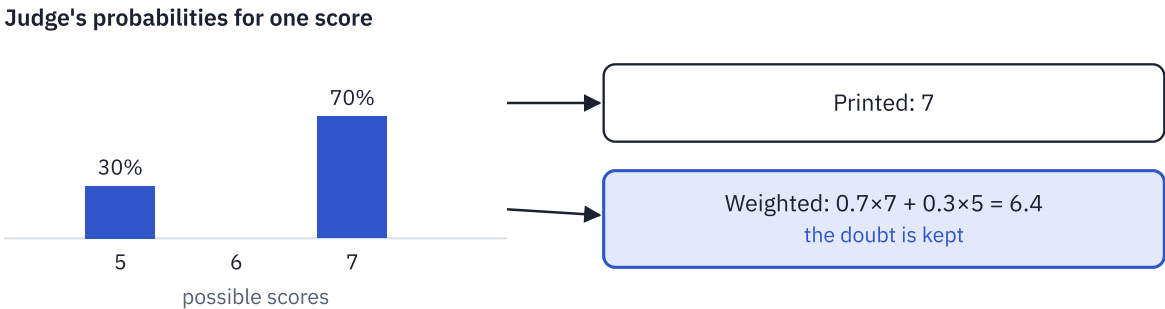
DeepSeek's long-context tricks live inside their model, so they can't be copied here. The idea behind them, that reading is cheap so read a lot, is already in the writer's 3.5× wider memory, which the probe showed costs no extra time.

Making a judge reliable

G-Eval 2023 · arXiv 2303.16634

Don't take the one number the judge prints. Take the probability it gave to *every* number and compute the average. "70% sure it's a 7, 30% sure it's a 5" becomes 6.4 instead of just "7".

For us: tested today. Ollama 0.34 returns these probabilities. Biggest single win, smallest change.

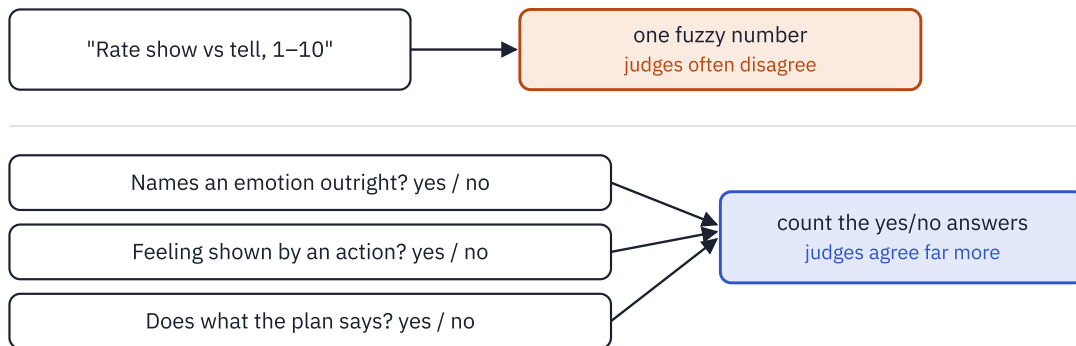


G-Eval in one picture. The printed number throws the doubt away. Weighting every possible score by its probability keeps it, so a 70/30 split between 7 and 5 becomes 6.4.

CheckEval · TICK 2024 · arXiv 2403.18771, 2410.03608

Replace "rate this 1–10" with small yes/no questions ("does any sentence name an emotion directly?"). Judges agree with each other and with humans much more on yes/no than on 1–10.

For us: turn each dimension into 4–6 yes/no checks, some generated from the scene's plan.

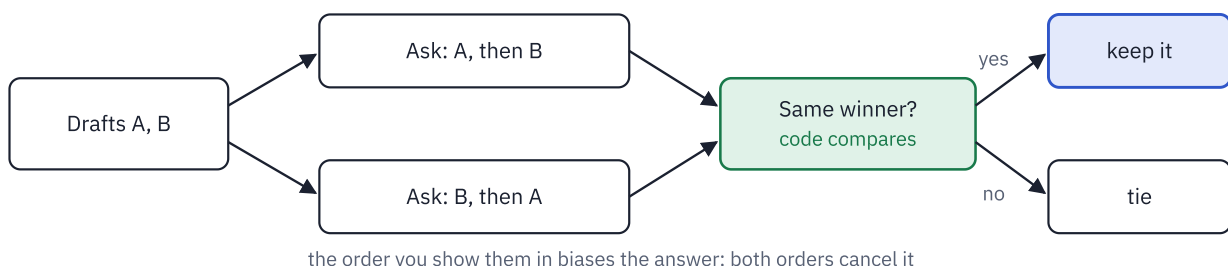


CheckEval and TICK: swap one 1–10 rating for a few yes/no questions, some written from the scene's own plan. Judges, and humans, agree far more on yes/no than on a score.

Pairwise vs pointwise 2024–25 · arXiv 2403.16950, 2504.14716

"Which of these two is better?" is more sensitive than scoring one alone, but easier to fool with irrelevant differences, and the order you show them in biases the answer.

For us: use pairwise only to *pick* between two drafts, asking in both orders. Keep pass/fail as single-scene checks.



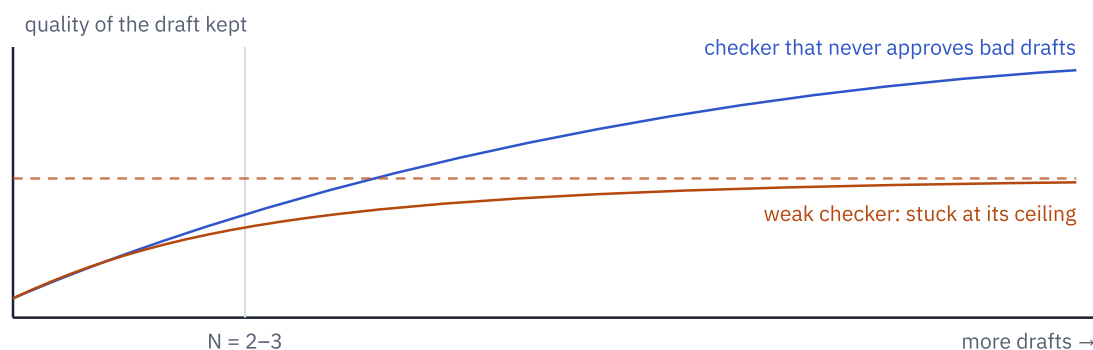
Pairwise judging is more sensitive than scoring one draft alone, but the position of each draft biases it. Asking in both orders and trusting only agreement removes that bias. Used only to pick

between drafts.

Inference Scaling fLaws 2024 · arXiv 2411.17501

If your checker sometimes approves bad answers, generating more drafts stops helping at a hard ceiling. You can't out-sample a weak checker.

For us: this is why "write 5 drafts, keep the best" must wait until the judge is fixed. Keep N at 2–3.



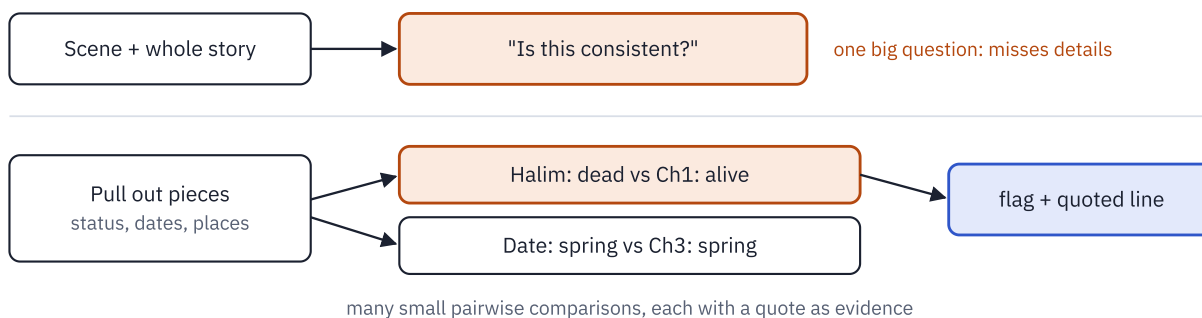
Schematic of the Inference Scaling fLaws result. If the checker sometimes approves bad drafts, writing more drafts stops helping at a ceiling set by the checker. That's why N stays at 2–3 until the judge is fixed.

Long stories that stay consistent

ConStory-Bench · ConWriter 2026 · arXiv 2603.05890, 2608.05169

They find contradictions by pulling out specific pieces (a character's status, a date) and comparing them **in pairs**, with a quoted line as evidence. Most errors show up 40–60% of the way through a story, in facts and timelines. ConWriter tracks story state and checks that each scene makes the *changes* it was supposed to make.

For us: stop asking one big question ("is this scene consistent?"). Ask many tiny ones.

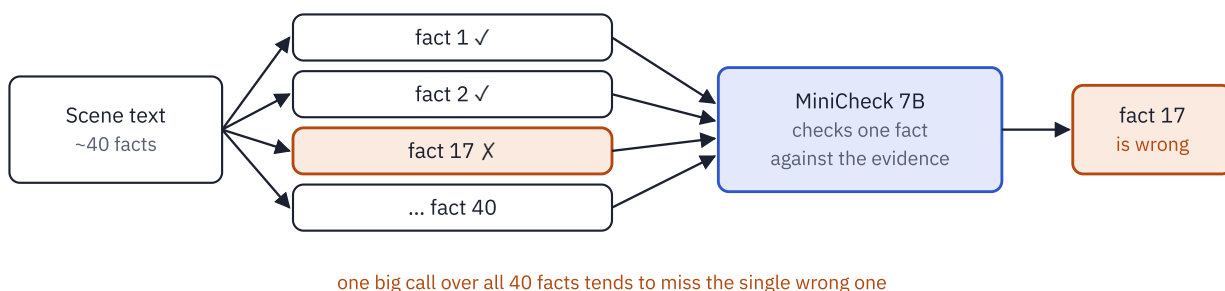


ConStory-Bench's method. Instead of one big question, pull out small facts and compare them in pairs, each backed by a quoted line.

FactScore · SAFE · MiniCheck 2023–24 · arXiv 2305.14251, 2403.18802, 2404.10774

Split text into single-fact sentences, look up the evidence for each one, check each one alone. MiniCheck is a small checker built for exactly this, and a 7B version runs on Ollama.

For us: the new contradiction checker. It may also explain why the fact ledger "didn't help": one big judge call can't see a single wrong fact among 40.

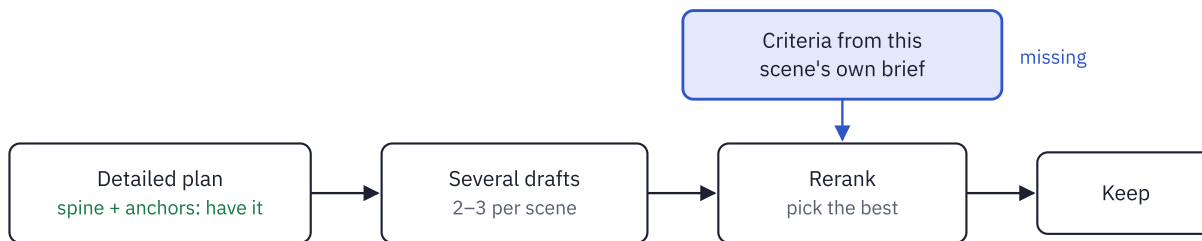


FactScore and MiniCheck: split text into single facts and check each one alone. One wrong fact among 40 is easy to spot on its own and easy to miss in one big question.

Re3 · DOC · Agents' Room · WritingBench 2022–25 · arXiv 2210.06774, 2212.10077, 2410.02603, 2503.05244

Plan in detail, write several candidates, rerank them, and generate the grading criteria from the specific request instead of a fixed list.

For us: your spine + anchor design is already in this family. The missing piece is criteria generated from each scene's own brief.

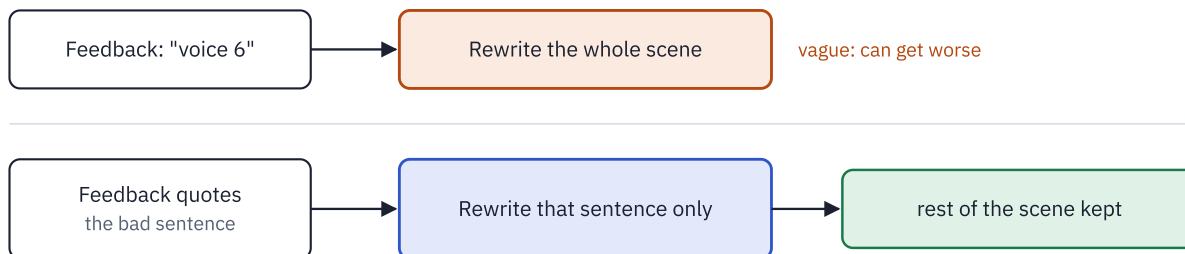


Re3, DOC, Agents' Room and WritingBench share this shape. Versatile already plans in detail. The missing piece is grading criteria written from each scene's own brief instead of a fixed list.

Self-Refine vs "LLMs can't self-correct yet" 2023 · arXiv 2303.17651, 2310.01798

Asking a model to "improve this" only helps when the feedback is specific and trustworthy. Vague feedback often makes the output worse.

For us: rewrite only the quoted bad sentences, not the whole scene. That's more precise and much cheaper.

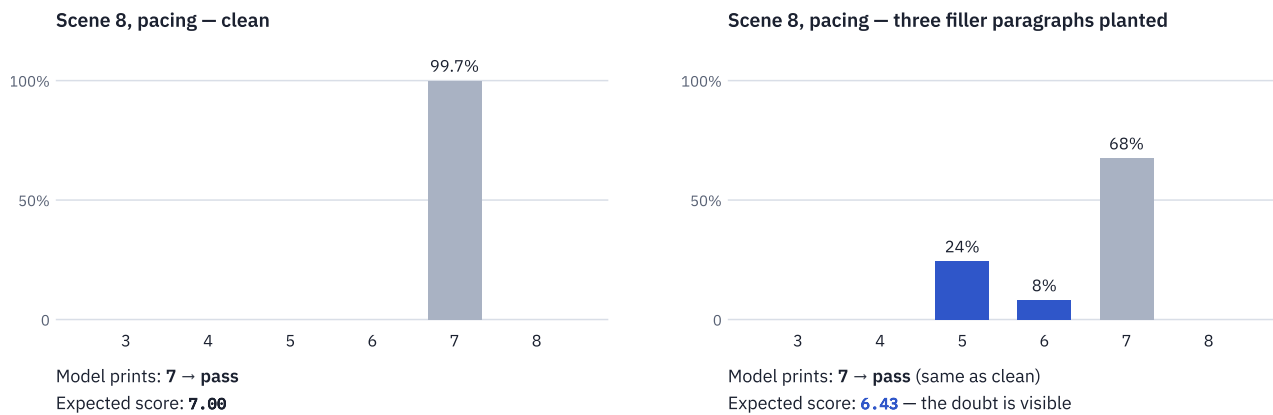


Self-Refine helps only when the feedback is specific and trustworthy. So rewrite only the quoted bad sentences: more precise, and much cheaper than redoing the scene.

Most of these citations were checked on arXiv in this session. Six were recalled from memory: G-Eval, FActScore, SAFE, Re3, DOC and Self-Refine. They are well known, but confirm the arXiv numbers before you quote them anywhere.

5. Today's experiment: read the probabilities, not the number

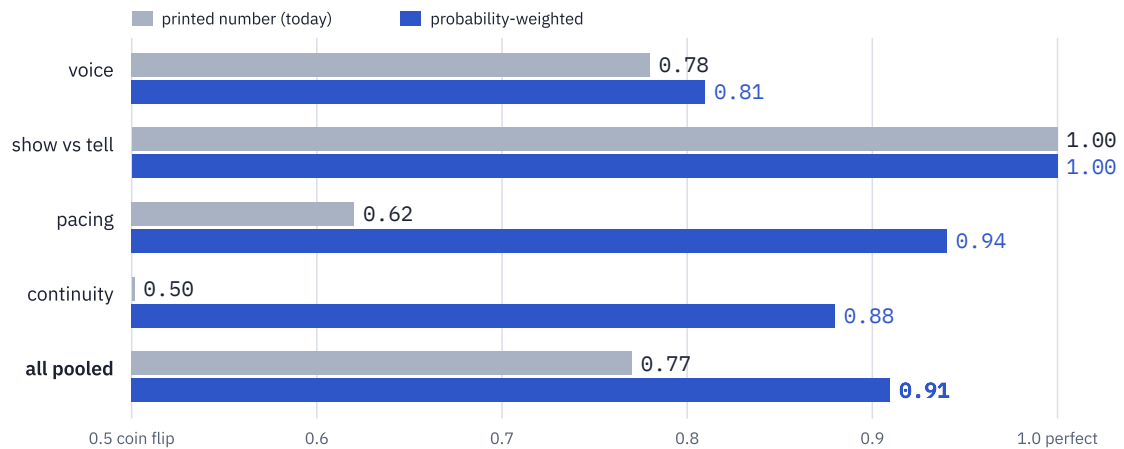
This was the cheapest idea to test, so I tested it. The setup is the same as your gate-sensitivity probe: 4 real scenes, each in 5 versions (1 clean, 4 with a planted defect), and each version judged on all 5 dimensions. That's 100 calls to qwen3:8b in 6.3 minutes, using the same focused prompt the production gate uses. The only difference: Ollama was asked for `logprobs`, so every answer comes with the probability of each possible score.



Same judge, same question. The printed number hides the doubt, and the probabilities show it. The judge had noticed the filler: it put 32% on "below 7". It just didn't print that. The expected score is $\sum \text{probability} \times \text{score}$.

How to read the next chart: AUC in one sentence

Pick one broken scene and one clean scene at random. **AUC is how often the judge gives the broken one the lower score.** 0.5 means a coin flip, and 1.0 means it's never wrong.

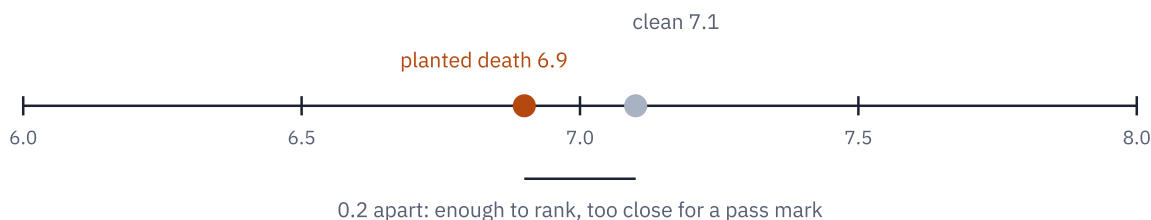


Each dimension compares 4 broken scenes against the same 4 scenes clean, on that dimension's own score. Pacing and continuity improve the most, because the printed number was stuck on 7 for almost every scene, so it couldn't rank anything.

What this does NOT prove yet

- **Small sample.** 4 scenes per defect. Read it as "worth building", not "proven". The same scene was only judged once per arm, so there are no error bars.
- **Continuity's gain is ranking, not a usable gate.** The planted "X has been dead two years" moved the expected score only from about 7.1 to 6.9. That's enough to rank correctly, far too small to set a pass mark on. Continuity still needs the claim-by-claim checker.
- **One global threshold doesn't work.** With a single pass mark for every dimension, clean prose still fails 1–2 of 4. Scene 14 gets voice 5 even when clean, which may be a real weakness in that scene. Each dimension needs its own pass mark, learned from labelled examples (step 2 below).
- **The judge often blames the wrong dimension.** A scene turned into summary sometimes fails on "voice" instead of "show vs tell". The gate still fails it, but the rewrite instructions point at the wrong problem. Quoted evidence (step 3) fixes that. *(Fixed since: see 5b and 5n. Today, 3 of 145 planted defects fail on the wrong dimension, and none of the summaries touch voice.)*

Continuity, expected score



Planting "X has been dead two years" moved continuity's expected score by about 0.2. That's enough to rank the broken scene below the clean one, but no pass mark could sit safely in a gap that small.

The probe and its analysis are saved in the repo (untracked) as `tools/geval-probe.py` and `tools/geval-analyze.py`. The raw results are in `reports/live/geval-probe/summary.json`.

5b. Why the gate blamed the wrong thing, and the fix

The focused gate caught broken scenes but often named the wrong problem: a scene turned into summary failed on "voice". I ran four small experiments, each testing one explanation.

IDEA TESTED	WHAT HAPPENED	VERDICT
Is it just the baselines?	Show-tell dropped the most in 4 of 4 summary scenes. It just wasn't the <i>lowest</i> score, because voice already sits low on some scenes.	part of it
Make each failure quote its evidence (DeepSeekMath-V2)	Every quote was real, and for broken scenes it was the exact sentence I planted . But the judge quoted that same sentence for voice, pacing, emotion... whatever it was asked about.	the judge sees the flaw
Ask "which aspect does this passage fail?"	Works for one-sentence flaws (summary → show-tell, "dead two years" → continuity, both about 100% sure). Fails for flaws that are a pattern across the scene (flat voice, filler), and calls clean passages "show-tell".	not shipped
Show each judge only its own evidence	Voice judged from the dialogue alone: clean 8.7–9.0, flattened 3.0–3.6 on 2 of 3 scenes, and other defects barely move it. Pacing as a label per	shipped

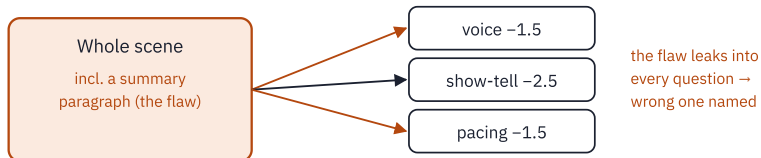
IDEA TESTED

WHAT HAPPENED

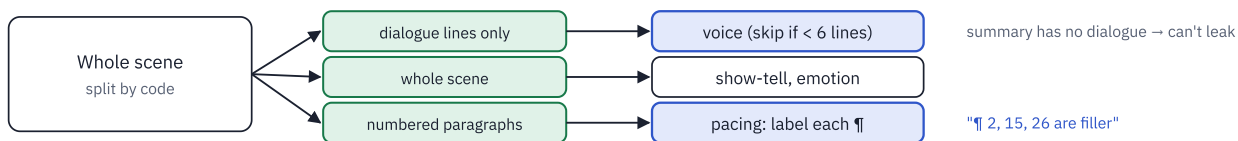
VERDICT

paragraph: **10 of 12** planted filler paragraphs found by number.

BEFORE — every judge reads everything



AFTER — each judge reads only its evidence

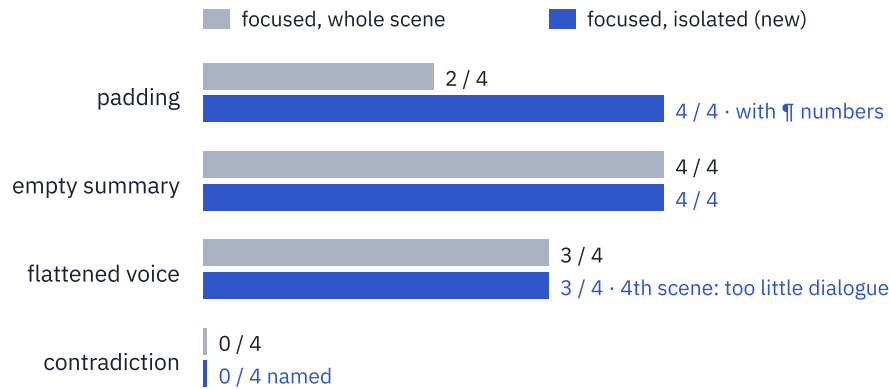


Green boxes are plain code splitting the scene. The judge sees less, so there's less to leak. Pacing now answers *which* paragraphs, so a rewrite can cut exactly those instead of redoing the whole scene.

The acceptance test: production critic, same 16 broken scenes

	COMBINED (DEFAULT)	FOCUSED, WHOLE SCENE	FOCUSED, ISOLATED (NEW)
Broken scenes caught	0 / 16	11 / 16	15 / 16
Right problem named	0 / 16	9 / 16	11 / 16
Other dimensions dragged under the pass mark	0	13	9
Clean scenes failed	0 / 4	1 / 4	2 / 4

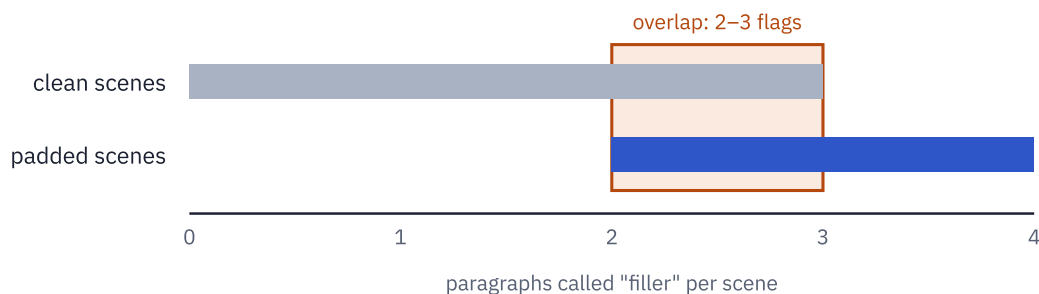
By defect: **padding went from 2/4 to 4/4**, with paragraph numbers. Summary stayed at 4/4. Voice stayed at 3/4; the fourth scene has only 2 lines of dialogue and is now correctly not judged. Contradiction was named **0/4** both times. Its "catches" are pacing false alarms, not real detections, and continuity still needs the claim-by-claim checker from step 4.



Right problem named, per planted defect (4 scenes each). Isolation fixed padding and points at the exact paragraphs. Contradiction was never named, which is why continuity got its own checker.

What's still broken: pacing false alarms

The same pacing prompt flagged 0 paragraphs of clean scene 8 in my probe and 3 in production. The only difference was how the request reached Ollama (`/api/chat` vs `/api/generate`). On borderline paragraphs the labels flip on tiny formatting changes. Clean and padded scenes overlap in filler counts (clean up to 3, padded 2–4). I did **not** retune the threshold on these 4 scenes: that would be grading my own homework, the same mistake as the benchmark. Next: count only paragraphs the judge is confident are filler (probabilities again), and pick thresholds on the 30-scene set, then test on different scenes.

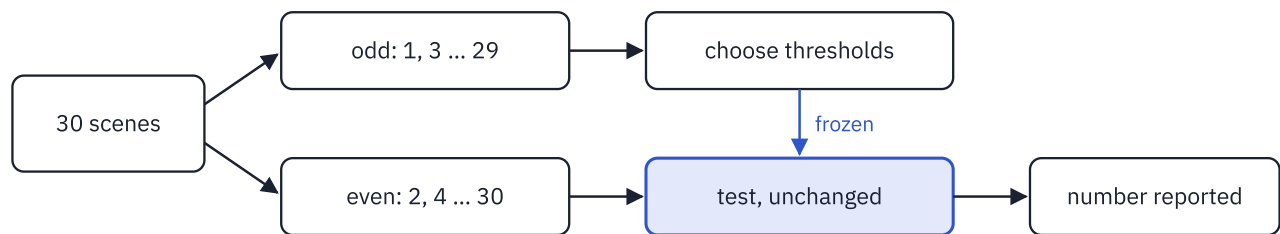


Clean scenes got 0–3 paragraphs called filler, padded ones 2–4, and borderline labels flipped between two ways of calling Ollama (0 flags versus 3 on the same clean scene). A pass mark picked on these 4 scenes would be fitted to noise.

The code is in `src/composables/criticIsolation.ts` (still behind the focused-gate flag, which stays off by default), with tests in `criticIsolation.test.js`. The comparison script is `tools/gate-attribution.py`, the probes are in `tools/judge-probes/`, and the full write-up is §19 of `GENERATION-PIPELINE-ANALYSIS.md`.

5c. Closing it out on all 30 scenes

Four scenes were enough to see the gate move, but too few to tune it and then trust it. So everything below uses **all 30 scenes**, with a rule: any threshold is chosen on the odd-numbered scenes and then tested, unchanged, on the even-numbered ones. That keeps me from grading my own homework.

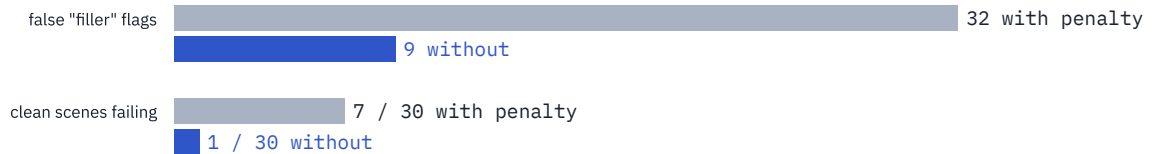


Tuning and testing never share scenes. A threshold picked on the odd half is frozen, and only its score on the even half is reported, so the number isn't graded on its own homework.

Discovery 1: a setting meant for prose was sabotaging the judge

The app sends every Ollama call a `repeat_penalty` of 1.15, which nudges the model away from repeating itself. That's good for prose. But the pacing judge's answer is a list like "ADVANCES, ADVANCES, ADVANCES...", the same word over and over. The penalty kept pushing the model away from "ADVANCES", so paragraphs near the end of the list got called "FILLER". My probe didn't send the penalty, which is why it disagreed with production.

Same 30 clean scenes, same prompt; only the penalty changes



Padded scenes caught: 30 / 30 either way. Nothing is lost by switching it off.

A setting chosen for writing quietly broke a component that isn't writing. The fix is one line: judge calls now use `repeatPenalty: 1` (neutral). On the held-out half: 0 of 15 clean scenes fail pacing, 15 of 15 padded scenes caught.

Discovery 2: "summary gets blamed on pacing" was my test, not the judge

My summary defect replaced paragraphs with empty sentences like "The moment passed, having accomplished roughly what it needed to accomplish." That's telling, *and* it's filler, so a pacing judge is right to complain. I built a fairer version: each paragraph becomes one plain sentence that keeps its plot facts. On that, pacing barely moved (-0.07) while show-tell dropped (-1.83). The pacing judge was fine all along.

Empty summary (my first fixture)



Faithful summary (fairer fixture: plot kept)



My first summary defect was also filler, so pacing was right to complain. With a fairer version that keeps the plot, pacing barely moves and show-tell takes the hit. The pacing judge was fine all along.

Discovery 3: ties were decided by list order

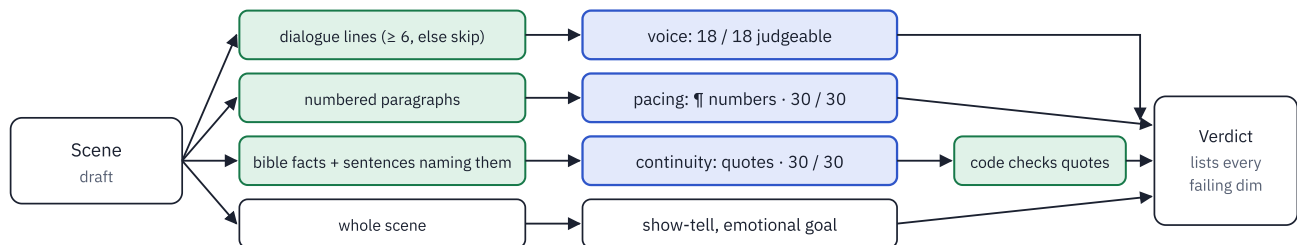
The verdict named only one "weakest" dimension. When show-tell and emotional goal tied at 5, whichever came first in the list won, and show-tell lost all 11 ties. The verdict now lists **every** failing dimension, and the rewrite step gets all of them.



When two dimensions tied, a fixed list order picked the "weakest", so show-tell was never named. A tie is two failures: the verdict now lists both, and the rewrite gets both.

Discovery 4: continuity works when the judge sees only facts and the sentences that could break them

The continuity judge now sees the story bible's facts plus only the sentences that mention someone in it. It must return each contradiction as the exact sentence *and* the exact fact it breaks, and code checks that both strings really exist. That's DeepSeekMath-V2's "prove it" step, checked by code instead of a second model. It found the planted contradiction in **30 of 30** scenes (8 of 30 before), and falsely accused 0 clean scenes. It also found a **real** error nobody had seen: committed scene 14 says "since the day Halim died", but the bible says Halim is alive.



every judge call: repeat penalty off · blue = isolated input · grey = still reads everything

The gate as it stands. Three of the five judges now see only their own evidence. The two grey ones still read the whole scene, and that's where the remaining spillover lives.

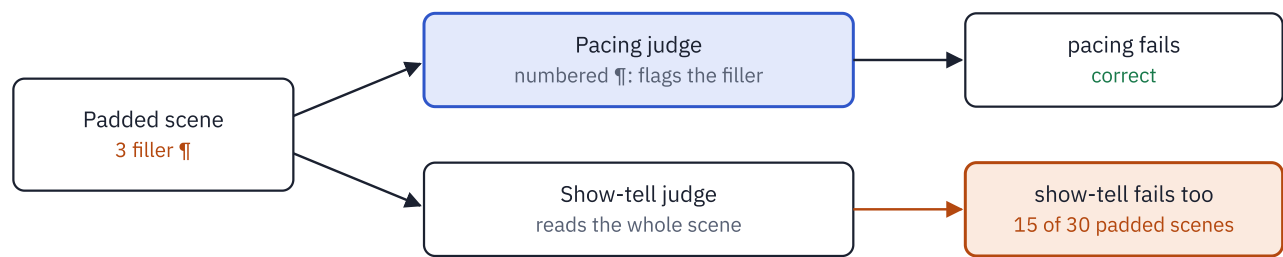
The final acceptance test: 30 scenes × 6 versions, real production critic

	COMBINED (DEFAULT)	FOCUSED, START OF TODAY	FINAL	HELD-OUT HALF ONLY
Broken scenes caught	0 / 16	11 / 16	133 / 145	64 / 72
Right problem named	0 / 16	9 / 16	129 / 145	63 / 72
Clean scenes failed	0 / 4	1 / 4	2 / 30	1 / 15

DEFECT	RIGHT PROBLEM NAMED	NOTES
Contradiction ("X dead two years")	30 / 30	was 8 / 30; the quote points at the exact sentence
Filler paragraphs	30 / 30	names the paragraph numbers
Paragraphs replaced by empty summary	30 / 30	
Flattened voice	18 / 18 judgeable	7 more: 6 with under 6 dialogue lines (skipped on purpose), 1 caught by the repetition guard
Faithful telling (plot kept)	21 / 30	the mildest defect; some of my cached summaries still have imagery

What's left, honestly

- **2 of 30 clean scenes fail pacing.** Scene 4's flagged paragraphs are slow, atmospheric material; a human editor might flag them too. Scene 5 had a line of plot-moving dialogue called filler, which is a real false flag.
- **Show-tell and emotional goal still co-fail with other defects,** because those two judges still read the whole scene. For example, show-tell also fails on 15 of 30 padded scenes. Some of this is defensible: filler is flat telling, and stating feelings outright does weaken them. Show-tell is the next thing to isolate. The per-paragraph approach failed earlier, but that test ran before I found the repeat-penalty problem, so it deserves one retry.
- **The focused gate is still off by default.** It now works, but turning it on means more rewrites and longer runs. That needs its own measurement on a real book run.



One filler paragraph, two failures. Show-tell and emotional goal still read everything, so a flaw that belongs to pacing also drags them down. Section 5d fixes this by blaming each flaw once.

5d. The last three items: blame once, confirm first, measure the cost

Show-tell by paragraph: tried again, still no signal

With the repeat penalty off, the per-paragraph "dramatised or reported?" labels still call **61%** of clean paragraphs "reported", exactly the same rate as the paragraphs I planted as telling. The labels carry no signal, so that approach is dead for good. But the same test called **88 of 90** filler paragraphs "reported". That was the real clue: filler *is* flat telling. When show-tell failed on padded scenes, it was one flawed paragraph being counted twice, not a leaky judge.

Share of paragraphs labelled "reported"

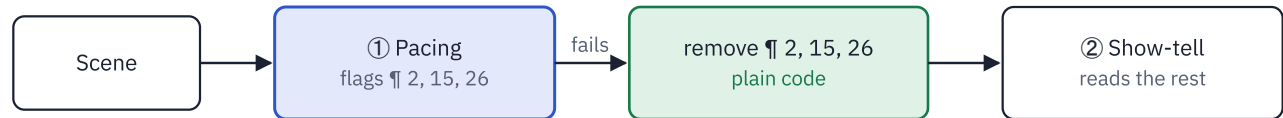


same rate for clean and telling: no signal. Filler is flat telling.

Per-paragraph show-tell labels can't tell clean from telling (61% each). But nearly every filler paragraph counts as "reported": filler is telling, so one flaw was failing two judges.

Fix 1: blame each flaw once

Pacing now runs first. If it fails the scene, the paragraphs it named are removed before the show-tell judge reads the scene. Padded scenes where show-tell also failed went from **15 of 30 to 0 of 30**.

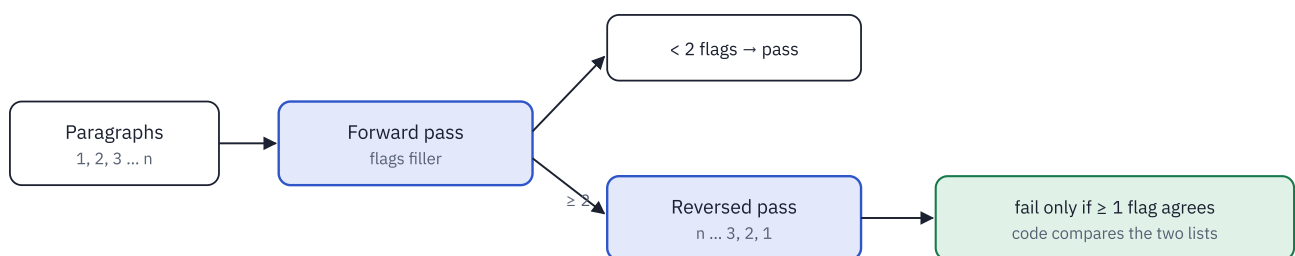


padded scenes where show-tell also failed: 15 of 30 → 0 of 30

Blame each flaw once. Pacing goes first. If it fails, the paragraphs it named are hidden from show-tell, so the same filler can't be counted twice.

Fix 2: a failure must be reproduced (pacing)

The same judge, asked about the same paragraph, sometimes flips on a borderline one. When the pacing judge would fail a scene, it's asked a second time with the paragraphs in **reverse order**. The scene only fails if the second pass agrees on at least one flagged paragraph. This is DeepSeek-GRM's voting idea at its cheapest: two votes, and the second is only cast when the first says "fail".



Clean scenes failing pacing: 1/15 → 0/15 on the half the rule was picked on, and **1/15 → 0/15** on the held-out half. Padded scenes: 15/15 both.

Clean scenes almost never reach the second pass, so they pay nothing. Only a scene about to fail gets the extra check.

Voice: the "false alarms" were right

A clean scene failed voice in one run and passed in another, so I tested the same confirm-first rule for voice: 3 normal passes plus 1 reversed. It changed nothing. The clean scenes

that fail voice fail **every single time**. So I read their dialogue:

Scene 17, voice 5: "Does it hurt?" / "It does," / "But not enough to stop us." / "This will help," / "I know." Any character could say any line.

Scene 2, voice 9: "Trust is a luxury among traders. We deal in certainty, not faith." That's a person.

The judge was right. My "clean" scenes were never guaranteed good: they're model-written prose that I simply hadn't broken. **"Not injected" doesn't mean "good"**. A judge that fails some of them is doing its job, and those are exactly the scenes the old critic, which passed everything, let through.

30 "clean" scenes = not broken by me, not guaranteed good

Scene 2: voice 9
"We deal in certainty, not faith."
a real person talking

passes

Scene 17: voice 5
"It does," / "I know."
any character could say it

4 runs (3 + 1 reversed):

5

5

5

5

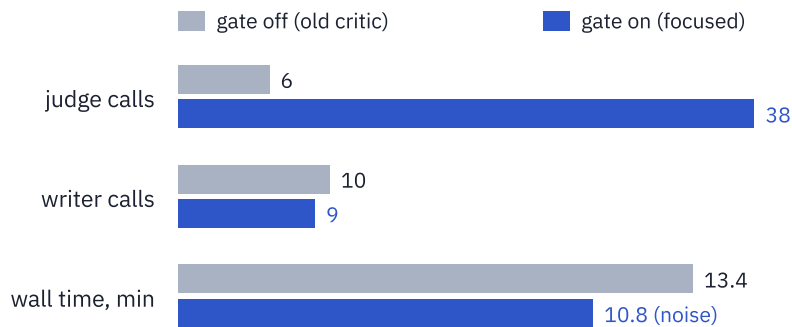
Scene 17 fails voice on every run, including a reversed one, so it isn't noise. Reading its dialogue shows why: the lines are interchangeable. "Not injected" never meant "good".

Fix 3: what does turning the gate on cost?

I ran two identical small books (2 chapters × 3 scenes): one with the old critic and one with the new focused gate.

	GATE OFF (OLD CRITIC)	GATE ON (FOCUSED)
Wall time	13.4 min	10.8 min
Model calls	41 (6 judge)	66 (38 judge)
Writer calls	10	9
Scenes the gate couldn't clear (saved as "review")	0 of 6	1 of 6

The gate adds about 32 judge calls of a few seconds each, and **no measurable time**. The writer is what takes time. The gate-on run finishing 2.6 minutes sooner is noise: the two runs wrote different books. One run per arm tells us the cost, not whether gated books read better. That needs more books.

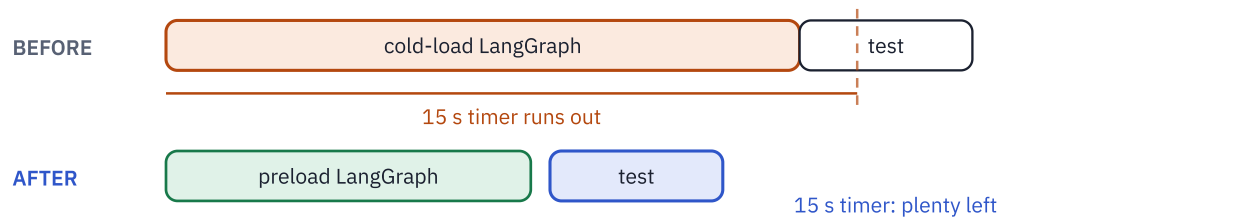


Judge calls went up six-fold, but each takes a few seconds. The writer's long calls set the wall time. The gate-on book finishing sooner is noise: the two runs wrote different books.

Final numbers (30 scenes)

	START OF DAY (DEFAULT CRITIC)	NOW
Broken scenes caught	0 / 16	133 / 145
Clean scenes failing pacing	–	0 / 30
Clean scenes failing anything	0 / 4 (it passed everything)	1 / 30: scene 10's voice, which reading the dialogue suggests is a real weakness
Padded scenes where show-tell also failed	–	0 / 30 (was 15)

Also fixed on the way: the flaky `useGroupChat` test. Its first group test was the first code to load LangGraph, and that cold load alone could exceed the 15-second limit under full-suite load. The module now loads before the tests start, and the suite passed 3 full runs in a row. Everything is in commit `485a11dd` and §21 of the pipeline doc.



Schematic. The flaky test wasn't slow code: the first group test also paid for loading LangGraph cold, inside its 15-second limit. Loading the module before the tests start moved that cost outside the timer.

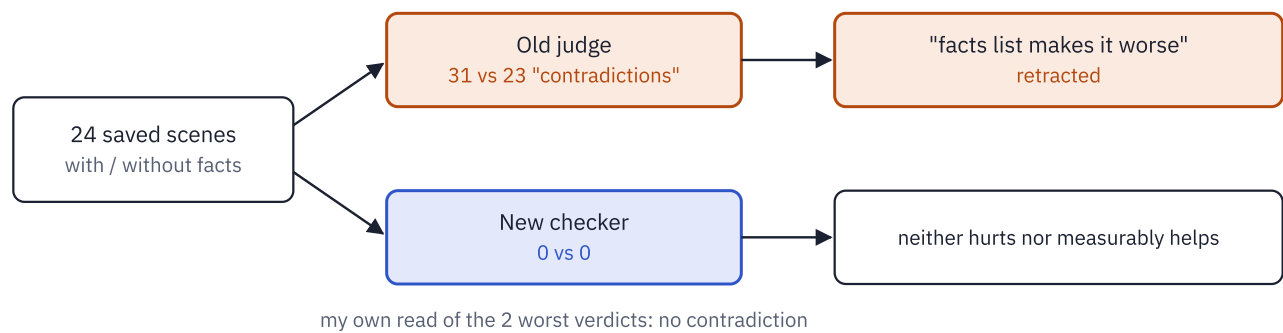
5e. A retraction, and a bug hiding behind it

The fact ledger did *not* make things worse

Last week's experiment concluded that giving the writer a list of established facts made it contradict the story **more** (+0.67 per scene). That experiment saved the 24 scenes it wrote, so I re-graded them with the new checker, the one that finds the planted contradiction 30/30 with no false alarms:

SAME 24 SCENES	WRITTEN WITH THE FACTS LIST	WRITTEN WITHOUT
Old judge's contradiction count	31	23
New checker's count	0	0

I read two of the old judge's worst verdicts myself (6 and 4 "contradictions") against their facts. Neither scene contradicts anything. The old finding was the judge's false alarms, not the writer, and it's now retracted in the doc (§22). The honest conclusion: at this depth neither version of the writer contradicts the story, so the facts list neither hurts nor measurably helps.



Same 24 saved scenes, two graders. The old finding came from the judge's false alarms, not the writer. Keeping the raw scenes is what made re-grading cheap.

That old judge was still running, and rewriting scenes

The same checker drives the **chapter-end audit**, which rewrites any scene it believes contradicts the story. I measured it on all 30 scenes:

Chapter audit, 30 scenes



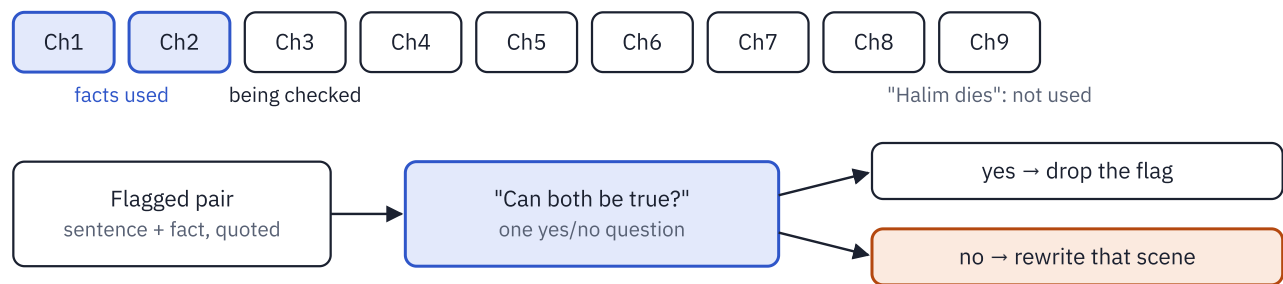
Left bars grow leftward: shorter is better. Right bars: longer is better.

The old audit accused good scenes and let the real contradiction through 29 times out of 30. Every accusation I read was false: "she collapsed under the salt in chapter 2, which contradicts her carrying it later"; "her clothes smelled of earth in one scene and of iron in another".

The fix (behind the same focused switch):

- **The audit now uses the isolated checker.** Each scene is checked only against facts from **earlier** chapters. Otherwise a chapter-9 fact like "Halim dies" would make every earlier "Halim is alive" look wrong. The quoted sentence tells the audit exactly which scene to rewrite.
- **Check the checker** (DeepSeekMath-V2's meta-verification). The last false alarm was a sentence that *agreed* with its fact in different words: "warns about the salt's strange properties" vs "warns of its unnatural qualities". Every flagged pair now gets one yes/no

question, "can both be true?", and only "no" survives. Result: **0 good scenes accused, 30 of 30 real contradictions caught.**

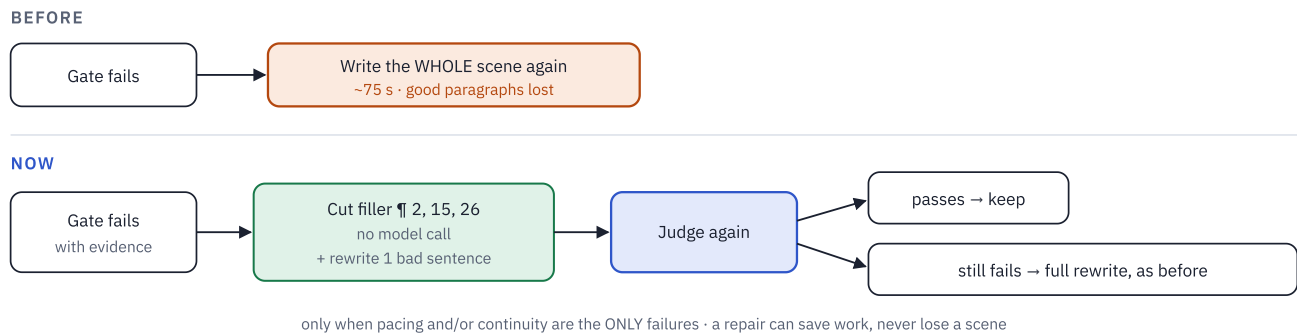


Two fixes to the chapter audit. Each scene is checked only against facts from earlier chapters, so a later death can't make earlier scenes look wrong. Every flagged pair then faces one "can both be true?" question, which removed the last false alarm.

Commits `31d0478f` (retraction, §22) and `2da1aa14` (audit fix, §23). Not claimed: how well it catches *subtle* natural contradictions. The planted one is blatant, and the 24 generated scenes contained none to find.

5f. Fix what's broken, keep what's good

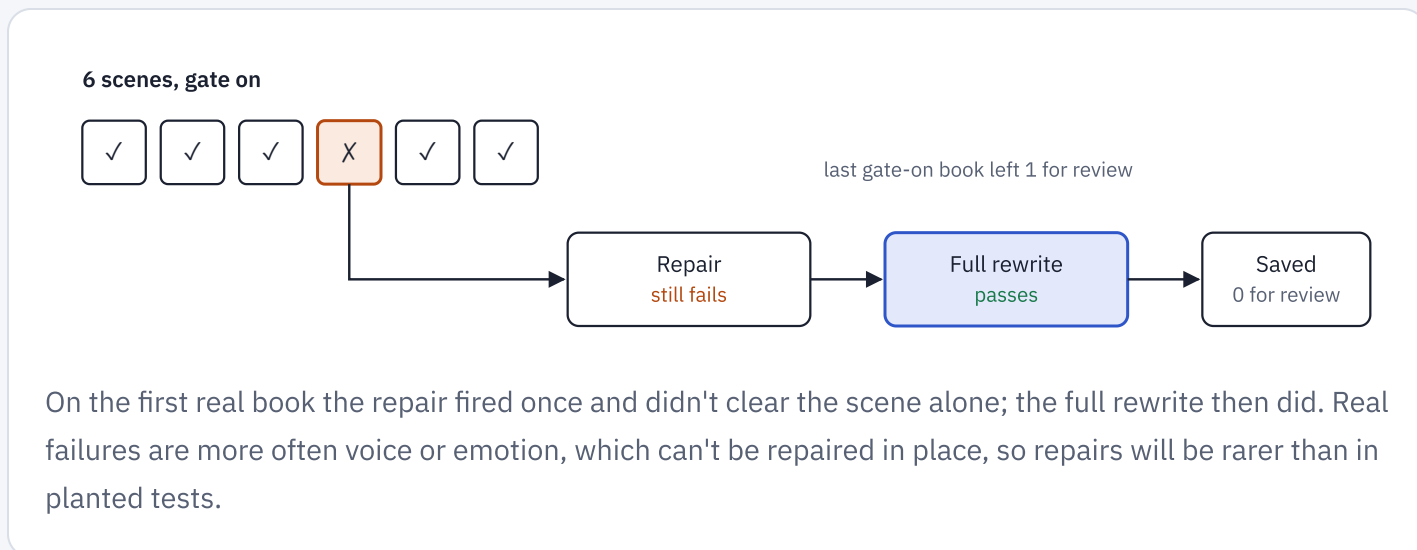
Until now, when the gate failed a scene the pipeline did one thing: it asked the writer for the **whole scene again**. That's about 75 seconds of writing. It throws away every paragraph that was fine, and the new draft can bring new problems. But for two kinds of failure the gate now says exactly what's wrong, so it can be fixed in place:



The gate's evidence (paragraph numbers, quoted sentences) becomes the repair plan. Voice or emotion failures still go back to the writer, because there's nothing precise to cut.

ON 30 PLANTED SCENES	RESULT
Padded scene passes after cutting the confirmed filler	27 / 30 (6 real paragraphs cut by mistake, against 69 planted)
Contradiction scene passes after rewriting one sentence	29 / 30

First real book: the repair fired once in 6 scenes and didn't clear that scene alone. The full rewrite then did, and **no scene was left for review** (the previous gate-on book left 1). Real failures are more often voice or emotion, which can't be repaired this way, so repairs will be rarer than in the planted tests. How much they save needs more books.



Now on both writing paths

At first the repair only ran in the default (legacy) path. The LangGraph path builds its gate from separate steps (draft, critique, then an Editor that decides) and never repaired. The repair is now **one shared step**, "repair, then judge the repair", and both paths call it, so they can't drift apart. In the graph it runs inside the critique step, on the critic's lane. The Editor then sees the repaired draft and can accept it, instead of queueing a whole new draft on the GPU. Two new tests cover it, and I confirmed both fail when the repair is switched off. A real LangGraph book (2 chapters × 3 scenes) finished cleanly: 8.7 min, 6 of 6 scenes passing. No scene needed a repair in that run, so this run shows the path works, and the tests show the repair itself. Commit [b8ee3788](#) , §26.

LEGACY PATH



LANGGRAPH PATH

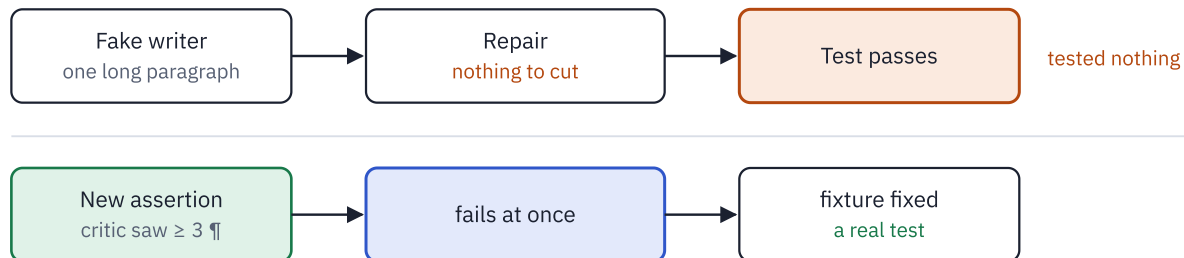


runs inside critique, on the critic's lane: no new GPU draft queued

The repair is one shared step that both writing paths call, so they can't drift apart. In the graph it runs inside the critique step, and the Editor sees the repaired draft instead of queueing a whole new one.

A testing lesson from this one: the first end-to-end test **passed without testing anything**.

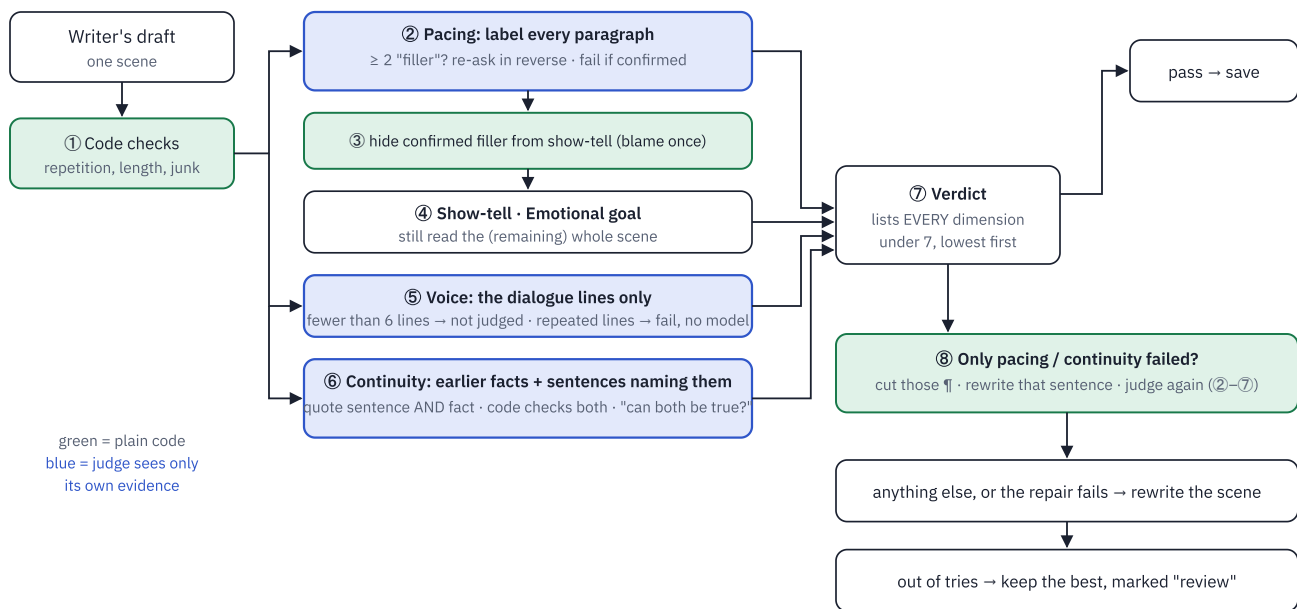
The fake writer's text arrived as a single paragraph, so there was nothing to cut. Asserting "the critic saw at least 3 paragraphs" exposed it. Commits [12a85df4](#) and [c470e248](#), §25.



The first end-to-end repair test passed because its fake input never reached the repair. Asserting the precondition, not just the outcome, exposed it.

5g. How the gate works now, step by step

Sections 5b–5f built the gate one fix at a time. This section puts it together: one scene, from the moment the writer finishes it to the moment it's saved. It then explains each of the four fixes that made the gate see, with the before-and-after that motivated it.



About 5 to 7 short model calls per judgement (the book runs measured 38–40 judge calls for 6 scenes, retries included), each asking one narrow question. Clean scenes skip the confirming passes and the repair entirely. The same steps run in both writing paths (legacy and LangGraph).

A scene walking through it

Take a 20-paragraph scene where the writer drifted: paragraphs 7 and 15 describe the light and the dust and nothing happens, and one line says Halim "had been dead two years", although the story bible says he's alive.

1 Code checks pass. No loops, no junk characters, and the length is fine.

2 Pacing flags ¶ 7 and 15. Two flags would fail the scene, so pacing asks again with the paragraphs reversed (¶ 20 first). The second pass also flags them, so the failure is confirmed. Pacing scores 5.

3 Show-tell reads the scene without ¶ 7 and 15. Those paragraphs are pacing's problem. Counting them again would also fail show-tell for the same flaw (they *are* flat telling), and the verdict would blame two things for one mistake.

4 **Voice reads only the 14 lines of dialogue.** The dust paragraphs and the dead-Halim line aren't in its input, so they can't drag its score down. It scores 8.

5 **Continuity gets the facts from earlier chapters, plus the 6 sentences that mention Halim or Nesrin.** It quotes "Halim had been dead two years..." against "Ch1: Halim is alive...". Code confirms both quotes exist. The last question, "can both be true?", gets the answer no, so continuity scores 3.

6 **The verdict says "continuity scored 3, pacing scored 5, below the minimum 7".** It names both dimensions, not just one.

7 **Repair.** Only pacing and continuity failed, and both come with exact evidence. So code cuts ¶ 7 and 15, and one short call rewrites the Halim sentence. The scene is judged again and passes. It's saved with 18 of its 20 paragraphs untouched, and without a 75-second rewrite.

The scene: 20 paragraphs



plus one line: "Halim had been dead two years" (the bible says he's alive)

Pacing	flags ¶ 7 and 15 · reversed pass agrees	5 · fails
Show-tell	reads 18 ¶: 7 and 15 hidden (blame once)	no double blame
Voice	reads only the 14 dialogue lines	8 · pass
Continuity	earlier facts + 6 sentences naming Halim or Nesrin	3 · fails
Verdict	continuity 3, pacing 5: both named	
Repair	cut ¶ 7, 15 · rewrite 1 sentence · judged again	pass · 18 of 20 ¶ kept

The worked example as a trace: what each judge reads and what it returns. Only two paragraphs and one sentence change. The other 18 paragraphs are saved untouched, with no 75-second rewrite.

The four fixes, explained

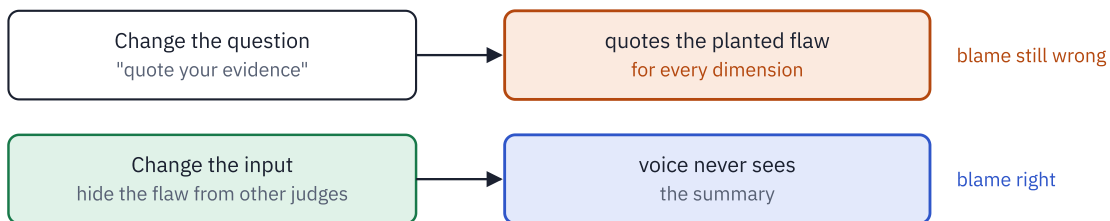
1 · Each judge sees only its own evidence

Problem: every judge read the whole scene, so one visible flaw dragged every score down. A scene turned into summary lost 2–3 points on voice, pacing and emotion as well as show-tell. The verdict then named whichever dimension happened to be lowest, often voice.

Fix: change what the judge *reads*, not what it's *asked*. Better instructions didn't help: asked for evidence, the judge quoted the planted flaw for every dimension. Hiding the flaw from judges it doesn't concern did help. Summary contains no dialogue, so the voice judge simply never sees it.

Result: filler named 30/30 at the exact paragraphs, contradictions 30/30, voice 18/18 where there's enough dialogue to judge.

Two ways to stop wrong blame



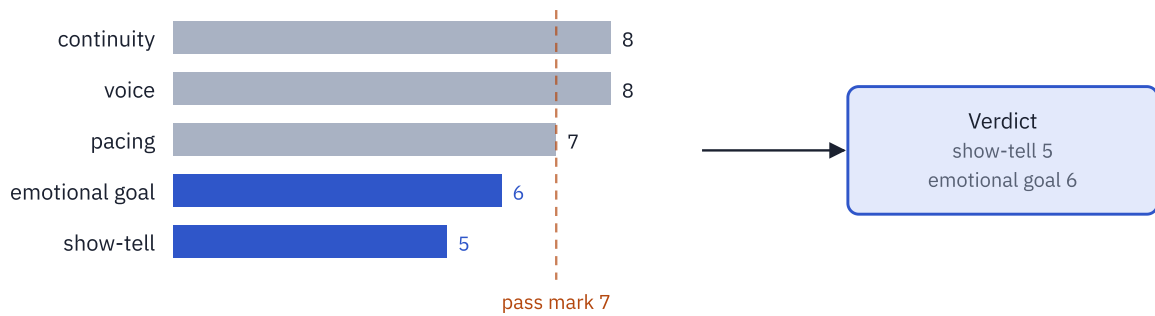
Better instructions didn't help: asked for evidence, the judge found the flaw and blamed it on whatever it was asked about. Narrowing what each judge reads did help.

2 · The verdict lists every failing dimension

Problem: the verdict named one "weakest" dimension. When two tied at 5, it picked whichever came first in a fixed list, and show-tell lost all 11 of its ties to emotional goal. The rewrite step was also told to fix only that one dimension.

Fix: a tie means two failures, so the verdict now lists every dimension under 7, lowest first, and the rewrite step gets all of them.

Example scores (illustrative)



Every dimension under 7 is listed, lowest first, and the rewrite is told about all of them, instead of one "weakest" picked by list order.

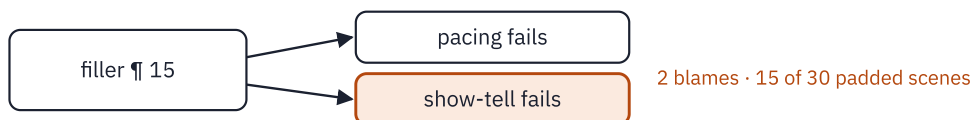
3 • Blame each flaw once

Problem: filler paragraphs failed pacing *and* show-tell, because filler really is flat telling (a separate test called 88 of 90 of them "reported"). One mistake was counted twice.

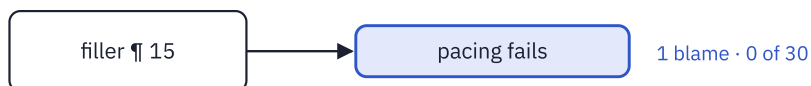
Fix: pacing runs first. When it fails, the paragraphs it named are hidden from show-tell.

Result: padded scenes that also failed show-tell went from 15 of 30 to 0 of 30.

BEFORE



AFTER



Filler really is flat telling, so one flaw was counted by two judges. Hiding the paragraphs pacing named from show-tell leaves one flaw with one blame.

4 • A setting meant for prose is switched off for judges

Problem: every call to Ollama used a "don't repeat yourself" penalty. That's good for prose, but a judge's answer *is* repetition ("ADVANCES, ADVANCES, ADVANCES..."), so the penalty pushed it toward "FILLER" further down the list.

Fix: judge calls use a neutral penalty. And before a pacing failure counts, it has to be confirmed by a second pass with the paragraphs reversed.

Result: clean scenes failing pacing went from 7/30 to 1/30 with the penalty off, and to 0/30 with the confirming pass.

Pacing judge's answer, one label per paragraph



clean scenes failing pacing: 7 / 30 → 1 / 30 (penalty off) → 0 / 30 (reversed re-check)

Schematic. A judge's answer is repetition by design, so the "don't repeat yourself" setting meant for prose pushed later labels toward FILLER. Judge calls now use a neutral setting.

5h. What's still imperfect, and what research says about it

Four gaps remain. For each one I looked for published work that fits this setup (local 8B judge, 8 GB GPU, no paid APIs), and ran one cheap test of my own. Citations were checked on arXiv or GitHub unless marked otherwise.

First, how sure are the numbers?

A fraction hides how much it could move with more scenes. The 95% range ("Wilson interval") is where the true rate probably sits:

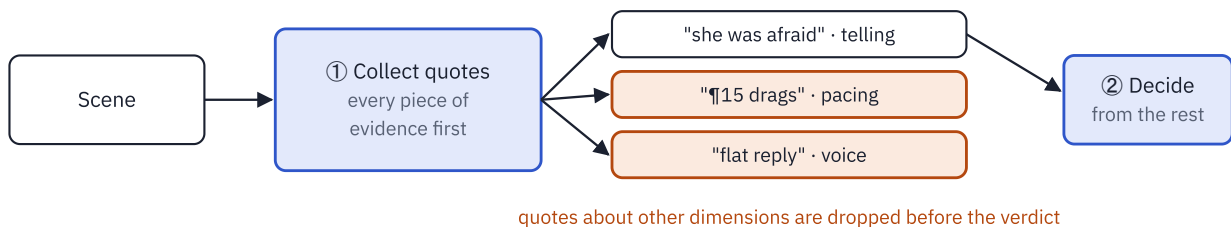
MEASURED	RATE	PLAUSIBLE RANGE (95%)	WHAT IT MEANS
Broken scenes caught	133 / 145	86 – 95%	Solid: a real, large effect.
Contradictions caught	30 / 30	89 – 100%	Solid, but only for blatant planted ones.

MEASURED	RATE	PLAUSIBLE RANGE (95%)	WHAT IT MEANS
Faithful telling caught	21 / 30	52 – 83%	The weakest spot, and it's wide.
Clean scenes wrongly failed	1 / 30	0.6 – 17%	Not established. 30 clean scenes can't prove a low false-alarm rate. It could be 1 in 6.

Gap 1 · Show-tell and emotional goal still fail alongside other problems [arXiv 2608.23783](#) · [2608.14684](#) · [2406.13439](#)

What research says: this is a known effect, called *inter-dimension dependence*. A judge's reasoning pulls in evidence about other qualities. A 2026 paper fixes it by **filtering the evidence** before the verdict ("DimCheck"): collect evidence first, delete anything about other dimensions, then judge from what's left. A second paper confirms that putting one question per call, which we already do, is the right first step. Any remaining co-failure comes from the scene itself, not the prompt.

For us: make show-tell and emotional goal two-pass: gather quotes, drop any quote about pacing, voice or continuity, then decide.



DimCheck's fix for judges that bleed into each other: gather evidence first, delete anything about another dimension, then judge only from what's left.

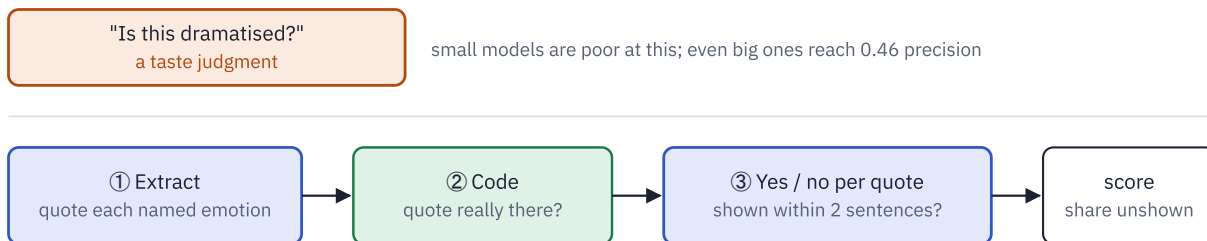
Gap 2 · Faithful telling caught only 21/30 [arXiv 2409.14509 \(LAMP\)](#) · [2309.14556 \(TTCW\)](#) · [1906.02402](#)

What research says: nobody has a reliable show-vs-tell detector. Even GPT-4-class models locate "unnecessary exposition" with only 0.46 precision. The lesson that fits our results: my failed per-paragraph test asked for a *taste* judgment ("is this dramatised?"). Small models are good at **extraction** ("quote every sentence that names an emotion") and

at narrow yes/no checks ("is that emotion also shown by an action or line within 2 sentences?").

My own test: I counted "telling fingerprint" words (named emotions like "she was afraid", filter verbs like "she realized") with plain code. They **don't** separate my telling fixture from clean prose: 18/30 scenes, close to chance. My fixture's telling is *summarised action*, not named emotion. So I need a second kind of test defect (named emotions) before I can measure the extract-then-check approach honestly.

For us: rebuild show-tell as extract, then check in code, then one narrow yes/no per quote. Test it against a new named-emotion defect and against LAMP's human-tagged exposition spans.

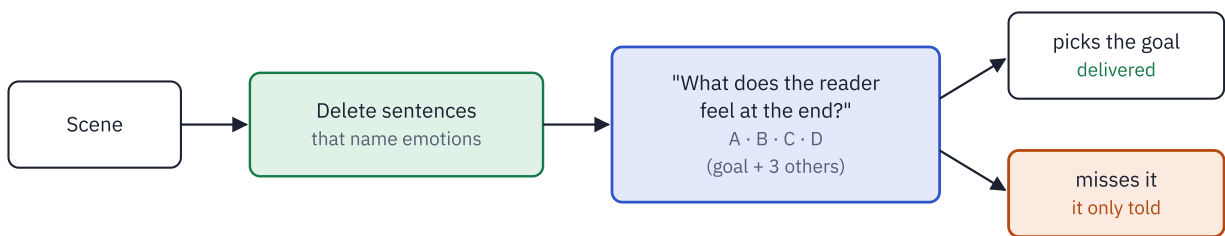


Small models are good at extraction and narrow yes/no questions, and poor at taste. So the show-tell judge becomes: quote, verify in code, then one narrow question per quote.

Gap 2b · Emotional goal is an absolute 1–10 judgment [arXiv 2406.12680](#) · [2609.13773](#) · [2507.00769](#) · [2402.12071](#)

What research says: LLMs judge "how moving is this?" poorly. The best setups reach about $p = 0.51$ against humans, and judges favour surface features like sentence length. **Pairwise** comparisons ("which version makes a reader feel dread more?") work better, asked in both orders.

For us: two cheap isolated checks. (a) Delete the sentences that name emotions, then ask a multiple-choice question: "what does the reader feel at the end?" If the right answer only comes from the named emotions, the scene tells rather than delivers. (b) Compare the scene against a deliberately flat version, in both orders.



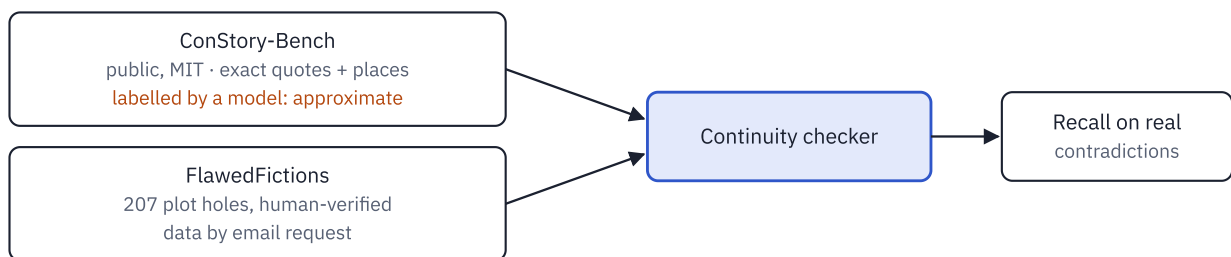
Emotional goal as a reader's question. With the sentences that name feelings removed, if the model can still pick the intended feeling, the scene delivers it. If not, only the named emotions carried it.

Gap 3 · Contradiction recall on real, subtle errors is unmeasured [arXiv 2603.05890](#) (ConStory-Bench) · 2504.11900 (FlawedFictions)

What research says: two ready-made test sets exist. **ConStory-Bench** is public (MIT licence, on Hugging Face). Its stories have contradictions with exact quotes and locations, though they were labelled by another model, so treat the labels as approximate.

FlawedFictions has 207 human-verified plot holes in real prose; its data has to be requested from the author by email.

For us: run the continuity checker on ConStory stories cut to scene size and measure what share of their contradictions it finds.



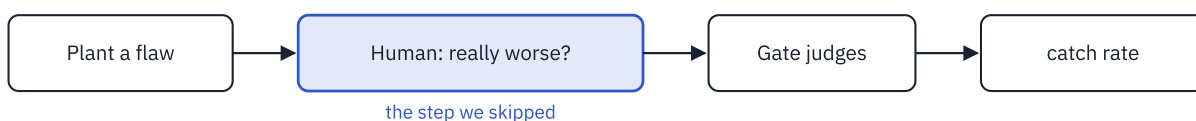
Two ready-made test sets of natural contradictions. They answer the question planted flaws can't: what share of real, subtle errors the checker finds.

Gap 4 · Everything was tested on planted defects, never on human labels [arXiv 2406.13439](#) (FBI) · 2511.21140 · 2404.12272

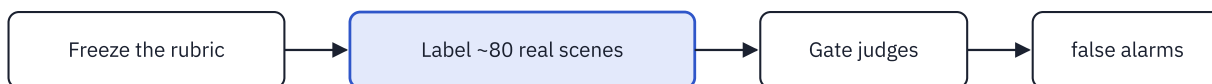
What research says: our "plant a flaw and see if the judge notices" method is an established technique (FBI, 2024). Their judges missed more than half of planted flaws, so ours doing better is meaningful. They add a step we skipped: **a human confirms each planted flaw really makes the text worse**. To report an honest false-alarm rate you need a human-labelled set: roughly **60–100 real scenes** labelled by you, with the rubric frozen first, because criteria drift as you grade. There's also a formula that corrects the judge's raw rate using its measured error rates.

For us: label about 80 real generated scenes yourself. It's the one step that turns "works on planted defects" into "works".

FBI-style planted test



Honest false-alarm rate

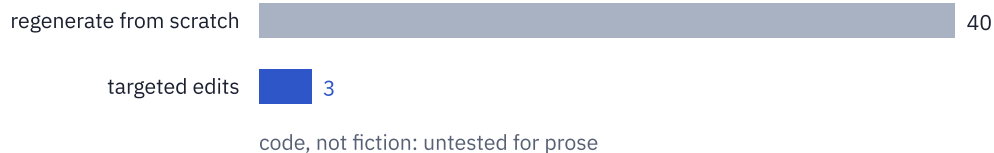


Planted flaws only show what the gate catches, and only once a human confirms each flaw really makes the text worse. A false-alarm rate needs real scenes labelled by a person, with the rubric fixed before labelling starts.

And a caution about the repair [arXiv 2504.07532](#) · [2609.00854](#)

Editing only the bad spans is well supported for prose. But a 2026 code study found that **re-generating from scratch beat targeted edits 40 to 3** at equal compute. Nobody has tested this directly for fiction. My first real book fits that doubt: the repair fired once and didn't clear the scene. It needs a fair head-to-head before I claim it saves anything.

A 2026 code study, equal compute: which fix won



The only direct comparison found (on code) favours regenerating, 40 to 3. Nobody has tested it for fiction, so the repair has to earn its place in a fair head-to-head (run later, in 5o).

5i. The real test: on real scenes, the gate is nearly blind again

Everything up to 5h was measured on flaws **I planted**. The plan's first step was to test the gate on **real scenes** against someone else's judgment. That changed the picture more than anything else today.

Planted test: can it see?



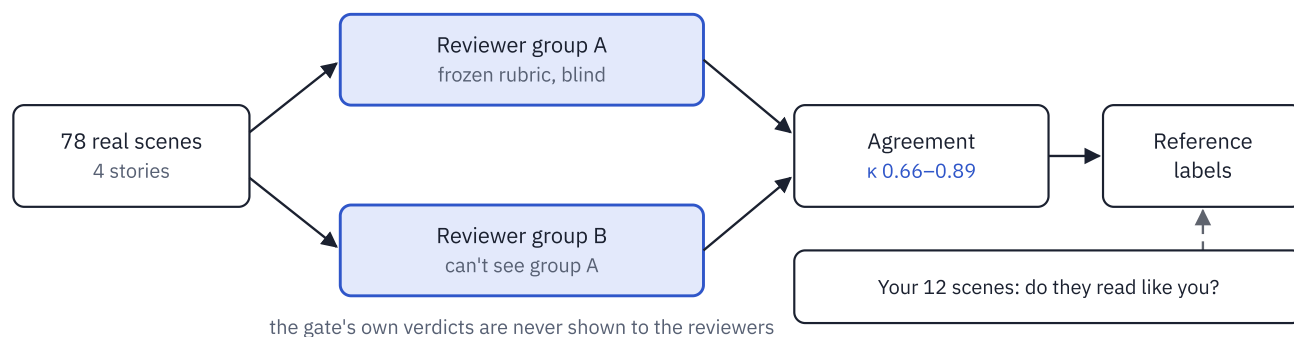
Real test: does it see?



Planted flaws answer "can the gate see a blatant flaw?". Real scenes graded by someone else answer "does it see the flaws that actually occur?". Only the second matters to a reader.

Who labelled what

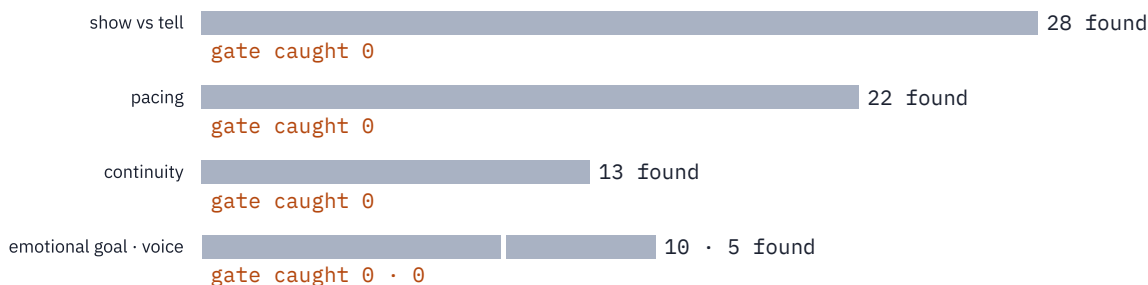
78 real scenes (about 48,000 words, four different stories) was too much for one person. So all 78 were labelled **twice**, by two separate groups of independent Claude reviewers. They used the same frozen rubric, never saw the gate's verdicts, and the second group couldn't see the first. Your share is **12 scenes**. Your labels will tell us whether the reviewers read scenes the way you do, so until you've done them, everything below is **reviewer labels, not human labels**.



All 78 scenes were labelled twice, by groups that never saw each other's labels or the gate's verdicts. Their agreement makes them a usable reference. Your 12 labels will show whether they stand in for you.

First check: do the two reviewer groups agree with each other? Yes. Kappa (agreement beyond chance, where 1.0 is perfect) is 0.66 to 0.89 per question, which counts as substantial to near-perfect. So they're a usable reference.

Real problems both reviewer groups agree on, and how many the gate caught



The gate failed 12 of 78 scenes. The reviewers would revise 46 of the 64 scenes they agree on. The 133/145 from the planted tests measured *blatant* flaws. Real flaws are quieter.

Are the reviewers just being harsh? Checked: no

Continuity is the one dimension where a claim can be verified, so I read what they flagged. It's all real and quoted:

- **Scene 14:** "It hadn't been warm since the day Halim died...", but the story's facts say Halim is alive and leading the caravan.
- **Scene 6:** a man "lying lifeless... the silence of death", who then coughs and dies two paragraphs later.
- **The Marseille scene:** she "closed the laptop... leaving only the faint glow of the screen".

The gate scored all of these 8/10 for continuity.

	Reviewers	Gate continuity
"since the day Halim died" facts: Halim is alive	contradiction	8 / 10 · pass
"lying lifeless... the silence of death" then coughs two paragraphs later	contradiction	8 / 10 · pass
"closed the laptop" "...the faint glow of the screen"	contradiction	8 / 10 · pass

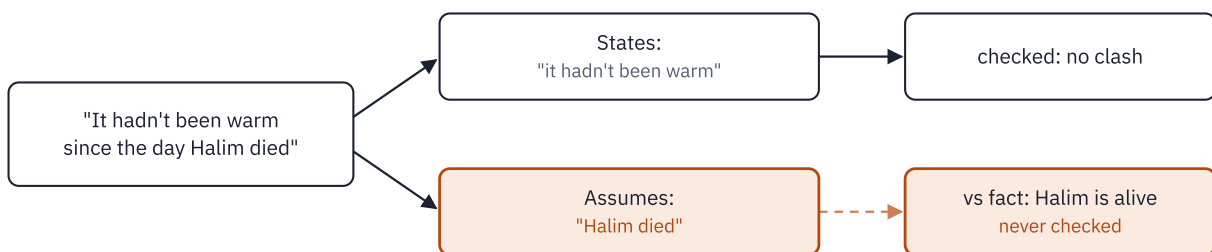
Three real contradictions, all quoted, all flagged by both reviewer groups. The gate scored each one 8/10 for continuity.

Why each judge misses: four different reasons

Continuity: it checks what a sentence says, not what it assumes

My planted flaw was a plain statement: "Halim had been dead for two years." The real one is tucked inside a clause: "since the day Halim died". The sentence *does* reach the checker, and the checker finds nothing. Linguists call this a **presupposition**. Separately, the gate has **no check at all** for a scene contradicting itself (the lifeless man who coughs).

Fix: first break each sentence into the facts it states *or assumes* ("Halim died"), then check those. The research brief pointed here (FActScore-style claim splitting). Add a within-scene pass that checks each scene against itself.



fix: extract what each sentence states or assumes, then check both

The checker tested what a sentence says, not what it takes for granted. "Since the day Halim died" states nothing false; the error is in what it assumes.

Pacing: a real signal, behind a rule tuned on fake filler

On scene 9 the gate flagged ¶13, and the reviewers said ¶13–15 could go. Real filler is milder than mine, so the gate usually flags one paragraph, and my "2 or more, confirmed" rule lets the scene pass. Problem scenes get a 7 twice as often as fine ones (32% vs 16%), so the signal is there.

Fix: recalibrate the pass mark on these labels, choosing on half the scenes and testing on the other half.

Share of scenes the pacing judge scores 7



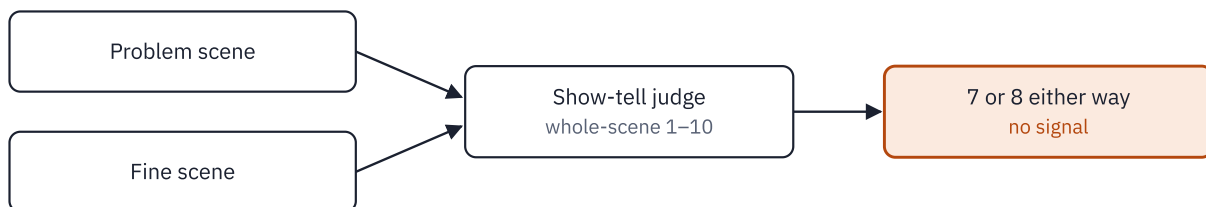
but the rule fails a scene only on ≥ 2 confirmed filler flags,
and real filler is milder: usually 1 flag, so the scene passes

The pacing signal is there: problem scenes get a borderline 7 twice as often. A rule tuned on blatant fake filler just sets the bar too high for real filler.

Show-tell: no signal at all

Problem scenes and fine scenes get the same mix of 7s and 8s. A whole-scene 1–10 judgment is simply not sensitive to real telling like "she felt the strain of keeping it all inside".

Fix: the extract-then-check design from 5h. That sentence is exactly the kind an extractor quotes.



Problem scenes and fine scenes get the same mix of 7s and 8s. A whole-scene score doesn't react to a single telling line like "she felt the strain of keeping it all inside".

Emotional goal: the constant score is back

It scores 7 on every problem scene and on 42 of 45 fine ones. That's the day-one "8/10 on everything" failure in a new place.

Fix: the isolated multiple-choice question from 5h ("what does the reader feel?").

Emotional goal scored exactly 7

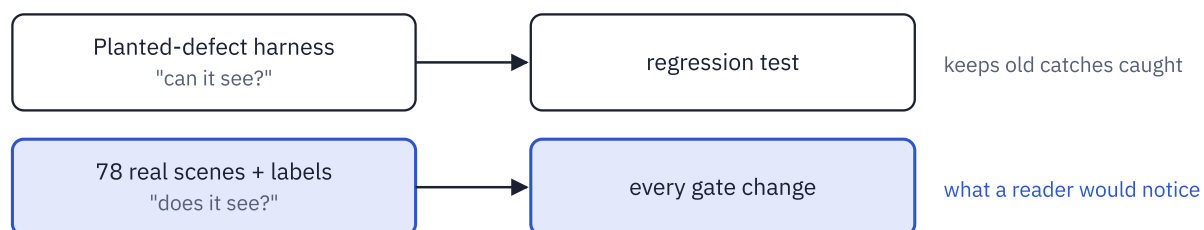


a constant score carries no information

The day-one "8/10 on everything" failure, in a new place. A score that barely varies can't separate anything.

The lesson, plainly

Planted flaws tested whether the gate *can* see a flaw. Real scenes test whether it *does*. Both tests were needed, and only the second one matters to a reader. The planted-defect harness stays as a regression test. From now on, every change to the gate is measured on these 78 real scenes.



Two test sets with two jobs. Planted flaws keep old catches from regressing. The 78 labelled real scenes decide whether a change to the gate is an improvement.

5j. Against the masterpieces: the gate prefers LLM style to Chekhov

You asked for a comparison with real masterpieces. It turned out to be the most revealing test of all. I cut 12 passages of about 550–850 words from six public-domain classics: Chekhov, Wharton, Joyce, Mansfield, Conan Doyle and Wells, two scenes each, covering our four genres. I gave them the same kind of brief as our generated scenes, then ran them through the gate and past two fresh reviewers using the same rubric.

Share of scenes each judge says need work

12 masterpiece passages

reviewers: revise 1 / 12

gate: fail 11 / 12

78 generated scenes

reviewers: revise 53 / 78

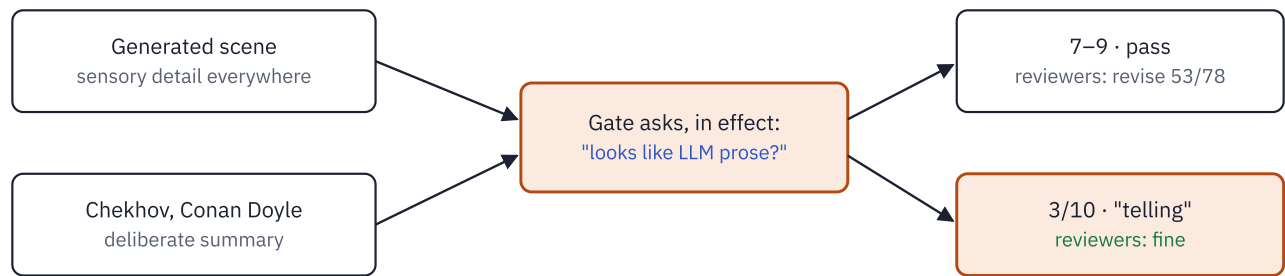
gate: fail 12 / 78

The reviewers pass great prose and fault generated prose, which is the right direction. The gate does the opposite. It failed "The Lady with the Dog" on show-tell (3/10) and gave every generated scene 7–9.

What the gate is really measuring

On show-tell and pacing, the gate isn't judging quality. It's judging **how much a passage looks like LLM prose**. Generated fiction has a house style: "the scent of damp earth clung to the air", "her pulse thudded against her throat". Every paragraph is stuffed with sensory detail. The gate rewards that. Chekhov and Conan Doyle use narrative summary on purpose and with great skill ("To Sherlock Holmes she is always *the* woman..."), and the gate reads that as "telling".

This is the surface-feature bias the research warned about. It explains 5i: the gate wasn't catching the reviewers' problems badly, it was **scoring something else entirely**.



On show-tell and pacing, the gate rewards the house style of generated fiction and punishes skilled narrative summary. It's measuring resemblance to LLM prose, not quality.

My own review of your 12

I read your 12 scenes cold and labelled them before looking at the reviewers. All three of us agree on **42 of 50** decisions. Where we disagree, I'm the *stricter* one (show-tell and pacing on two orchard scenes). Two limits: I'm the same model family as the reviewers, so this is a third careful reader, not a human check; and I'd already seen the reviewers' notes on one scene (the Marseille laptop one).

50 decisions on your 12 scenes, three readers

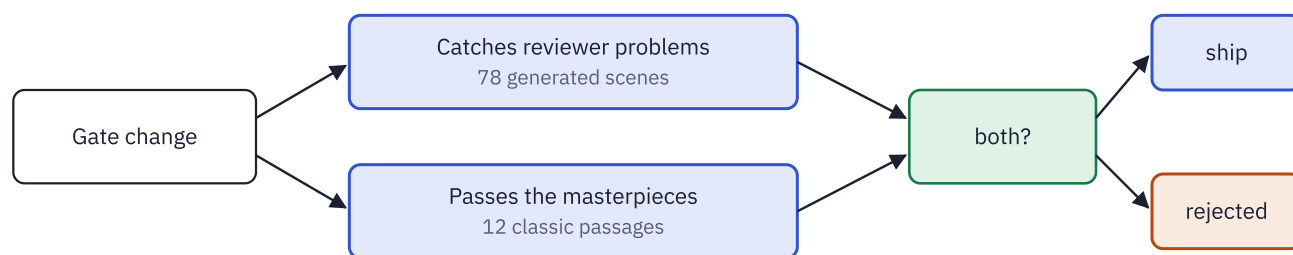


where we differ, I'm the stricter one (show-tell, pacing on two orchard scenes)

A third careful reader agreed with both reviewer groups on 42 of 50 decisions. It's the same model family, so this is a consistency check, not a human one.

The new rule for every gate change

Every change to the gate must do two things: **catch the reviewers' problems in the 78 generated scenes**, and **pass the 12 masterpieces**. A show-tell or pacing judge that fails Chekhov is disqualified, however well it does on planted defects.



The two-set rule. A judge that fails Chekhov is disqualified however well it does on planted defects.

5k. Acting on it: three judges stop failing scenes, continuity reads between the lines

5i and 5j said the gate was measuring the wrong things. This section is what I changed because of that, and how each change was checked. The rule from 5j applied to every step: **catch the reviewers' problems, and pass the masterpieces.**

First, a surprise in the labels

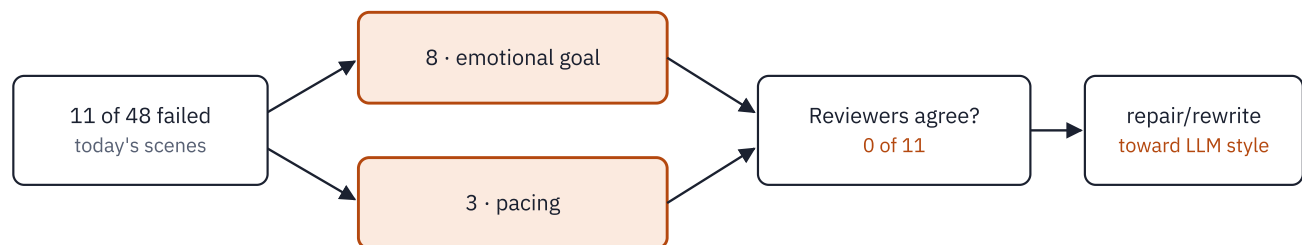
Before trusting any score, I split the reviewers' problems by where each scene came from. They aren't spread evenly. Almost all the pacing and show-tell problems sit in the **old Salt Road corpus**, whose scenes are twice as long (about 980 words) as what the pipeline writes today (about 410).

DIMENSION	OLD CORPUS (30 SCENES): PROBLEM / FINE	TODAY'S PIPELINE (48 SCENES): PROBLEM / FINE
continuity	11 / 12	6 / 30
voice	3 / 12	2 / 25
show-tell	24 / 3	4 / 34
pacing	22 / 4	0 / 45
emotional goal	7 / 13	3 / 32

Word count alone separates the reviewers' pacing problems from fine scenes almost perfectly (AUC 0.97). So a judge could "win" on pacing just by spotting the old corpus. On today's output, the reviewers found **no** pacing problems at all. Continuity is the one dimension with real problems left.

Change 1: show-tell, pacing and emotional goal now warn instead of fail

On today's output the gate failed 11 of 48 scenes. In **none** of those 11 had the reviewers seen a problem on the dimension the gate blamed (8 were emotional goal, 3 pacing). Every one of those failures bought a repair or a full rewrite, pushing the prose toward LLM style. The pacing judge is worse than useless: it scored all 22 reviewer pacing problems as passes.



Every failure blamed a dimension the reviewers saw no problem on, and each one bought a repair or rewrite that nudged the prose toward the style the judge prefers.

So these three are now **advisory**. They're still judged and still shown, but they can't fail a scene, count as major problems, pull down the average, or trigger a repair. Continuity and voice still decide. Both were validated on planted flaws, and neither ever failed a masterpiece. The end-to-end test found a second door: another gate took its own average of all five scores and would still have failed scenes. That's closed too.

The default gate, before and after (live, same scenes)

Masterpieces failed

before 11 / 12
after 0 / 12

Today's scenes failed

before 11 / 48
after 1 / 48 (voice)

Reviewer continuity problems caught

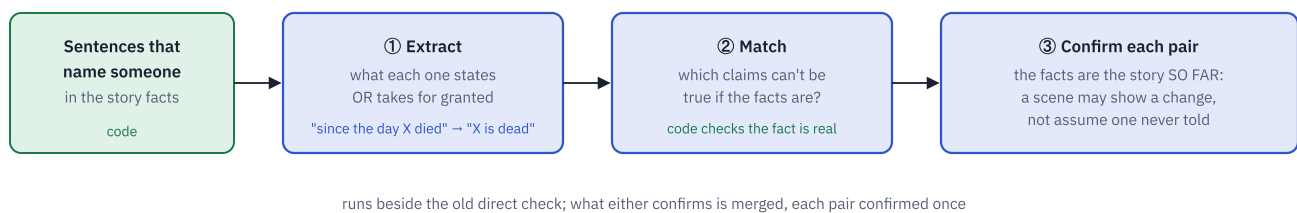
before 0 / 17
after 3 / 17 – and still 0 false alarms on 42 fine scenes

The cost: the critic now takes about 13–18 seconds more per scene (25.6 minutes for all 78, against 8.1), almost all of it the new continuity step below.

Change 2: continuity learns what a sentence assumes

The reviewers' story-fact contradictions were mostly *assumed*, not stated. "It hadn't been warm since the day Halim died" never says "Halim is dead", and the old check only looked for sentences that *state* a contradiction. It caught 0 of 9.

The new check asks three small questions in a row, the way 6④ proposed on day one:



Each step is a question a small model answers reliably. The hard part turned out to be the last one.

Three versions, each fixing a failure I could point to

- **Version 1 looked good (3 of 9), but the score was void.** Its prompt example was a sentence copied almost word for word from one of the test scenes. That's the answer leaking into the question. I replaced it with a made-up example.
- **Version 2 asked "can both be true?" of the bare claim. It dropped to 1 of 9.** The trace showed why. "Halim is alive" (Chapter 1) against "Halim is dead" came back as *both can be true*, since he could have died since. That's a fair answer to the wrong question.
- **Version 3 asks the right question.** The facts are the story so far. A scene may *show* a change happening, but it may not *assume* one the story never told. Result: **3 of 9 caught, 0 false alarms on 29 fine scenes.**

v1: leaked example 3 / 9 · void

v2: bare "both true?" 1 / 9 · wrong question

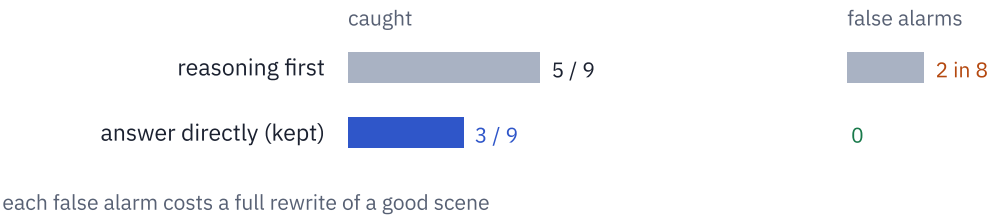
v3: "the story so far" 3 / 9 · 0 false alarms on 29

reviewer continuity problems caught, of 9

Same score for v1 and v3, very different meaning. v1's example was copied from a test scene. v2 answered a fair question about the wrong thing. v3 asks whether the scene assumes a change the story never told.

I also tried making the model write down its reasoning before answering. It caught more (5 of 9 on test pairs), but it also flagged "Élodie walked the orchard one last time" as contradicting the sale. That's 2 false alarms in 8, and each false alarm costs a rewrite, so I didn't adopt it.

Continuity confirmer: two ways to answer



Thinking first caught more, but flagged harmless lines such as "Élodie walked the orchard one last time". A false alarm costs a rewrite, so the quieter version stayed.

A test it was never tuned on

Because I debugged on those 9 scenes, their score flatters the new check. So I planted 12 new contradictions in clean scenes, each worded differently: a burial, a tax waved through, a debt denied, doubled oxygen rations. Two name nobody from the facts.

ON 12 PLANTED CONTRADICTIONS (LIVE, PRODUCTION CODE)	CAUGHT
Old direct check	4 / 12
New claims check	4 / 12 (a different four)
Both, merged: what runs now	5 / 12, every flag on the planted sentence

Only the new check sees the assumed death ("the only thing left of Halim after the sandstorm took him"). Only the old one sees "with the doubled oxygen rations". So both run. Neither sees the two plants that name nobody from the facts: a known blind spot.

What's still missing, honestly

- **Recall is low.** 5 of 12 plants, 3 of 17 reviewer problems. Every miss dies at the last step: the confirming question is too cautious. A better confirmer is the next lever.
- **Contradictions inside one scene** (8 of the 17) are parked. Extraction flattens "the dying man collapses" into "the Worker is dead" and merges "a worker" with "the Worker", so a death followed by a cough vanishes before anything checks it.
- **The show-tell replacement failed, and that's useful to know.** I tried "list the telling sentences, then ask if each is shown nearby", then counted telling per 100 sentences. It can't tell the reviewers' problems from fine scenes (AUC 0.46, where 0.5 is a coin flip). At its best pass mark it raised false alarms on 13 of 17 fine scenes and failed 7 of 12 masterpieces. Masterpieces simply *tell more* than generated prose. The reviewers weren't counting telling. They objected to telling *at the moment that matters*. A next try would first find that moment (from the brief's "what changes") and judge only that passage. Show-tell stays a warning.
- **Voice** still fails one of today's scenes that the reviewers passed, and has caught 0 of 5 reviewer voice problems. It stays a gate only because planted-flaw tests back it.



caught: 5 of 12 plants, 3 of 17 reviewer problems

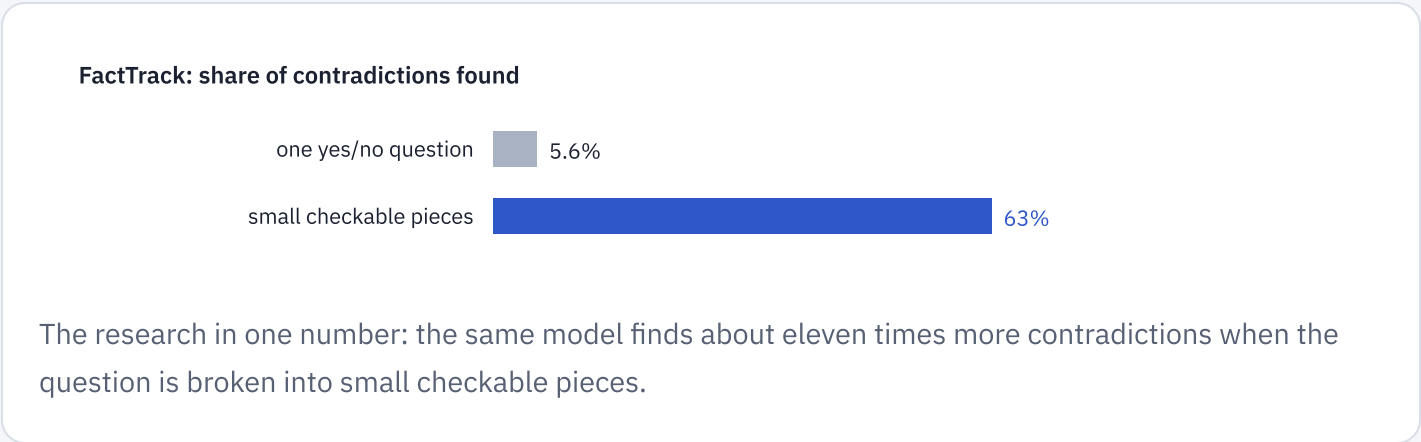
Recall is low for one reason: the misses get through extraction and matching, then the confirming question turns them down. A better confirmer is the next lever.

51. The four open problems: research, then a fair test for each

You asked for solutions to the four gaps left in 5k. I read the research first, then built one candidate per gap and tested it the same way as before: the reviewers' labels, the

masterpieces, and planted mistakes. **One small fix shipped. The rest either failed or backfired on real scenes.** Each result tells us something.

- What the research said, in one line each**
- **Asking a small model one yes/no question** gives high precision and tiny recall. In one study it caught 5.6% of contradictions. Breaking the question into small checkable pieces raised that to 63% (FactTrack).
 - **Asking an 8B model to review a whole story** is about as good as guessing (FlawedFictions).
 - **Telling speakers apart from single lines** is near chance, even in published novels.
 - **For show vs tell**, find the key moment first, then measure how much story time passes in it.



PROBLEM	WHAT I TRIED	RESULT	SHIPPED?
Continuity misses	Two extra checking questions: spell out what the fact means, and what the sentence takes for granted	Planted mistakes: 5→9 of 11 , and 6→9 of 14 on a fresh set. Real scenes: 3→4 of 17 caught, but false alarms 0→3 of 42 , and checking took 2.3× longer	No, reverted
Contradictions inside a scene	Collect exact quotes a few paragraphs at a time, then look for clashing pairs	Flags 3 of 8 (0 before); 0 of 42 false alarms; but fails 1 masterpiece (Joyce: "her mother was alive" then, "dead" now)	No, not yet safe
Voice	Measure each character's style with	AUC 0.53, a coin flip; with a scene's worth of dialogue it's	No

PROBLEM	WHAT I TRIED	RESULT	SHIPPED?
	numbers (sentence length, questions, common words)	noise	
Voice false fail	Found a counting bug: "You carry enough," he said, "but..." counted as two lines	That scene, plus 5 others with too little dialogue, is now left unjudged. No real voice problem affected	Yes
Show-tell	Find the turning point, then ask "is it shown?" and "how much time passes?"	"Shown" for almost everything. The time measure puts masterpieces among the problem scenes	No

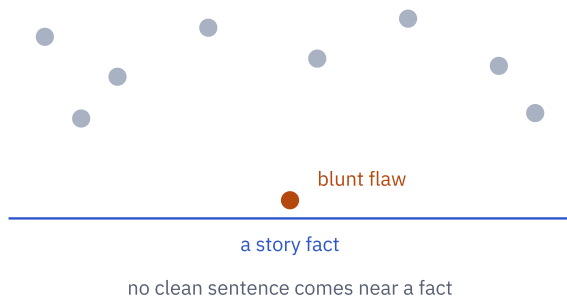
Every row was measured, not guessed. The two "no" rows with real gains (continuity, within-scene) are kept as scripts in `tools/judge-bench`, ready for the next attempt.

The most important lesson: planted mistakes hid the false alarms

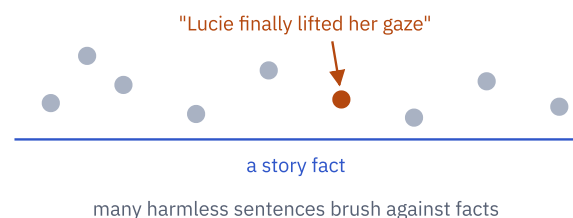
The continuity upgrade looked excellent on planted mistakes, including a fresh set I wrote before testing, which is the careful way to do it. Then I ran it on the 78 real scenes. It started flagging harmless sentences, like "Lucie finally lifted her gaze", as contradicting "Lucie hasn't been told about the sale".

Why didn't the planted sets catch this? A planted mistake is blunt, and the clean sentences around it rarely sit close to a story fact. Real scenes are full of sentences that brush against the facts. So **only real clean scenes can measure false alarms**. I reverted the change.

Planted scene



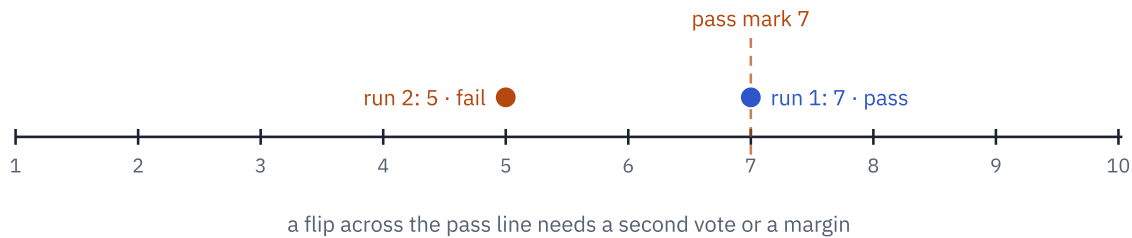
Real scene



Why the planted sets hid the false alarms. A planted flaw is blunt and the clean text around it rarely touches a story fact. Real scenes are full of sentences that brush against facts, which is where false alarms live.

A second finding: the voice judge wobbles

On a re-run, two scenes with *identical* dialogue went from 7 (pass) to 5 (fail). A score that flips at the pass line on the same input can't be trusted to fail a scene alone. It needs a second vote, or a margin, before it counts. That's the next voice step, and it's cheaper than any new judge.



What it looked like in 5l: the same dialogue scored on both sides of the pass mark. (Section 5m later found the input wasn't really identical: a fix of mine had changed how the lines were split.)

Where this leaves the gate

It no longer punishes good prose: 0 of 12 masterpieces fail. It catches a few real continuity errors with no false alarms on clean scenes. Show-tell, pacing and emotional goal stay as warnings, because three separate attempts found no judge that tells good from bad there. The honest summary is that a small local model can reliably check facts, but not taste.

Facts: checkable
continuity decides pass / fail
real errors caught,
0 false alarms on clean scenes

Taste: not yet
show-tell, pacing, emotional goal
warnings only
3 attempts, no judge that works

a small local model checks facts reliably, not taste

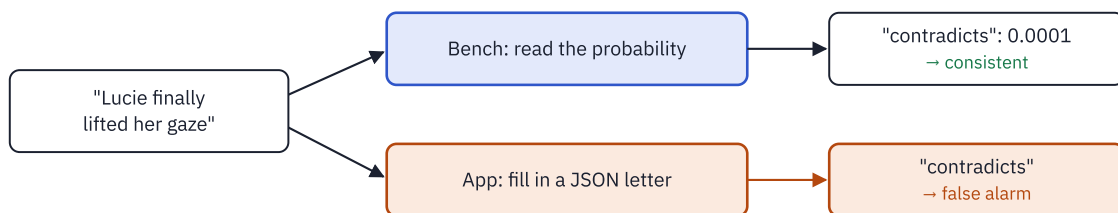
Where the gate stands after 5l: it decides only where claims can be checked, and only advises where the question is taste.

5m. Fixing it: three things I got wrong in 5l, and two fixes that hold

You asked me to fix everything. First, three corrections, because they change the story in 5l.

1 The continuity false alarms were my porting bug, not the idea.

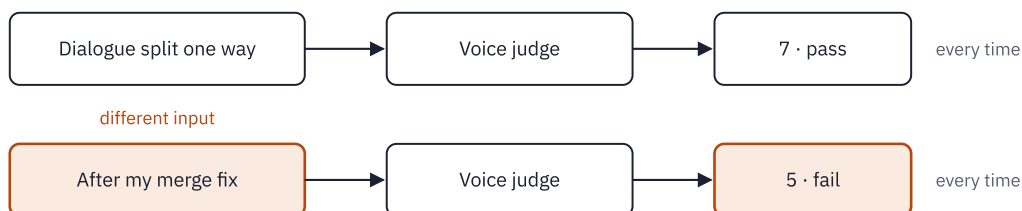
In testing, I read the model's *probability* for its answer. In the app, I asked it to fill in a JSON letter instead. For "Lucie finally lifted her gaze", the model's real probability of "contradicts" was 0.0001, yet the JSON answer said "contradicts". Forced-format answers aren't the model's true answer.



Same model, same question. Forced to fill a JSON field, it wrote the answer it gives almost no probability to. The app now reads the probability, as the bench did.

2 The voice judge wasn't wobbling.

The two "flipping" scenes had different input, because my dialogue fix had merged their lines. Split differently, the judge scored them 5 instead of 7, the same way both times. My fix removed one false alarm and created two.



The judge wasn't flipping. My dialogue fix changed how lines were split, so the two runs saw different input, and each input got the same score every time.

3 My timing numbers were polluted.

Jester's scheduled tasks use the same model every 15–30 minutes. Each time they took turns with Versatile, Ollama reloaded the model, about 18 seconds per call. The "59 minutes" figure in 5l is not a real cost. With your OK I paused Jester, measured cleanly, and turned it back on.

Jester running



Jester paused



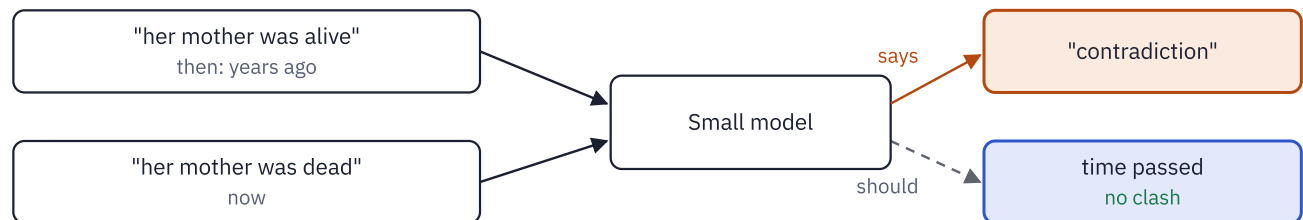
Two programs taking turns on one GPU forced Ollama to reload the model at every switch, about 18 seconds each. With Jester paused the same work ran without reloads, so the earlier "59 minutes" was not a real cost.

CLEAN LIVE RUN	YESTERDAY	NOW
Scenes from today's pipeline that fail the gate	1 of 48	0 of 48
Voice false alarms (reviewers said fine)	1 of 37	0 of 37
Continuity false alarms (reviewers said fine)	0 of 42	0 of 42
Real continuity problems caught	3 of 17	3 of 17
Planted contradictions caught (set A)	5 of 11	7 of 11
Fresh planted set B	6 of 14	9 of 14 , 0 of 6 traps flagged
Critic time, 12 scenes	5.1 min	6.0 min (+18%)

The two fixes: continuity's extra questions now read the model's real probability, and voice counts whole speeches to decide whether there's enough dialogue, while still judging the lines as written. One honest blemish: in set B, one clean sentence ("Lucie's laughter... light and careless") was flagged. That's 1 false alarm in 32 planted scenes, and 0 in 42 real ones.

Still not fixed, and why

- **Contradictions inside a scene:** the model firmly believes Joyce's "her mother was alive (then)... her mother was dead (now)" is a contradiction, however I ask. I built a fresh set of 12 masterpiece passages for the next attempt.
- **Show-tell and pacing:** three judges failed the masterpiece test. Today's pipeline has too few of these problems (4 and 0 in 48 scenes) to build a fourth. The next step is more labelled scenes, not another judge.

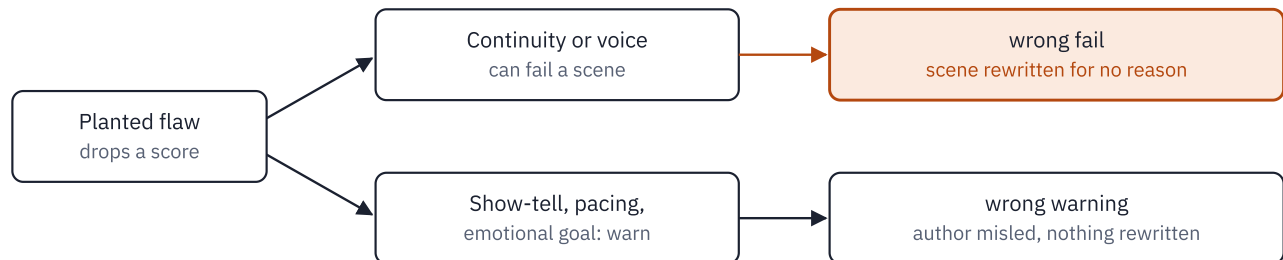


A real contradiction needs both statements to hold at the same time. The model ignores the "then" and "now" in Joyce however the question is asked, so the within-scene checker stays unshipped.

5n. "It blames the wrong dimension": re-measured on today's gate

You asked to finish the first-day item where the gate caught broken scenes but blamed the wrong thing. Much has changed since, so I re-ran the exact same test on today's code: 30 scenes, each also broken five different ways on purpose. I then counted two kinds of wrong blame separately:

- **A wrong *fail*:** continuity or voice (the two that can fail a scene) drops for a flaw that isn't theirs. The scene gets rewritten for the wrong reason, so this is the costly one.
- **A wrong *warning*:** one of the warning-only dimensions drops. The author is told the wrong thing, but nothing is rewritten.



Wrong blame costs different amounts depending on where it lands. On a gate dimension it triggers a needless rewrite. On a warning-only one it only misleads.

PLANTED FLAW (30 SCENES EACH)	RIGHT DIMENSION FLAGGED	WRONG <i>FAIL</i>	WRONG <i>WARNING</i>
All dialogue made to sound the same	18 of 25	0	11 (mostly show-tell)
Scene turned into empty summary	28 of 30	0	30 (pacing, emotion: <i>correct</i> , see below)
Scene turned into faithful summary	23 of 30	1	11 (mostly emotion)
Filler paragraphs added	30 of 30	1	4
A flat contradiction added	30 of 30	1	8

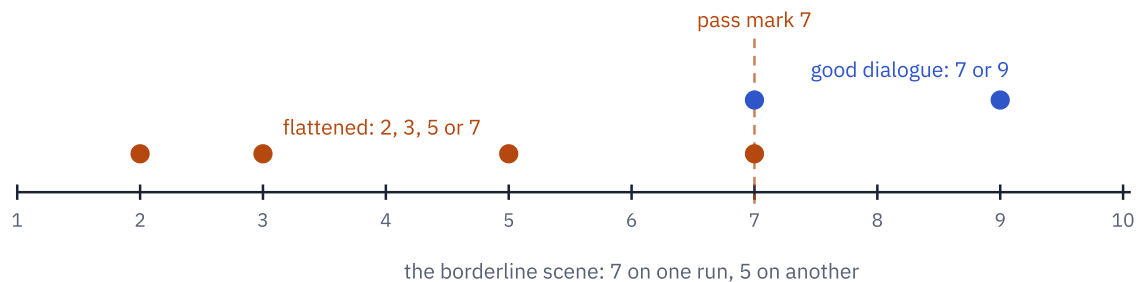
The day-one complaint ("summary failed on voice") is gone. Across 60 summary versions, voice never dropped. Only 3 of 145 broken scenes fail for the wrong reason.

Three things the re-run taught me

- **The "clean" test scenes weren't clean.** 4 of the 30 unbroken scenes now fail on continuity. All four are real: "the amulet Halim gave her before he died", "since the day Halim died", a dying man who is also already dead, and a "still" mule that should be injured. The

reviewers found three of them. The old gate had scored all four 8 out of 10, so this is the new continuity check doing its job.

- **Some "wrong" warnings are right.** The empty-summary flaw swaps paragraphs for sentences like "The business of the hour was transacted without incident". That really is filler with no emotion, so flagging pacing and emotion there is correct. The warnings that really are wrong (show-tell on flattened dialogue, emotion on faithful summaries) sit on dimensions that are warnings only, because their judges showed no signal on real scenes. I've left them alone.
- **The voice judge has one borderline scene.** Two of the three wrong fails are one scene whose dialogue scores 7 (pass) on one run and 5 (fail) on another. I tested the obvious suspect, the voice judge's randomness setting, directly: on its own, that dialogue scored 7 five times out of five either way, and the stricter setting added a false alarm. So I didn't change it. Voice scores come in coarse steps (7 or 9 for good dialogue; 2, 3, 5 or 7 for flattened), and 5 is also where real flattened voices land. No margin rule can separate the two, so it stays a known small error: 2 in 145.



Voice scores come in coarse steps, and 5 and 7 are used for both good and flattened dialogue. No margin can separate them, so one borderline scene stays a known small error (2 in 145).

Status

This item is closed. What's left is two known, measured error sources. Nothing in the gate changed in this round: the one change I tested (voice randomness) made things slightly worse, so it wasn't kept.

5o. Working through the plan (section 7): what's done, what it showed

You asked me to implement the plan in section 7. I took the steps cheapest-first. Each change was checked on the reviewer labels, the masterpieces, or a live run. Here's where each step stands.

PLAN STEP	WHAT I DID	RESULT
8a. The Editor's illegal answers	The Editor now picks <i>one plan</i> from a list of legal combinations ("gpu: revise #1 / cpu: critique #2"). Before, it filled in two free-form fields.	Legal answers on 24 test situations: 7 → 24 of 24
8b. A retrieval test that can fail	The old test was circular: it decided what counted as "relevant" with the same method it was grading. The new one asks the real question: for each story fact, can retrieval find the chapter it came from, among 30 scenes of the same book?	Score 0.67 (chance ≈ 0.21 ; the old test gave an impossible 1.0). The app's current settings were already the best of those tried.
8c. Spine from written prose	When writing chapter 5, the writer now sees chapters 1–4 <i>as they were written</i> (the last scene's summary and the facts found in the prose), not as planned.	Verified live: chapter 2's writer prompts carried "WRITTEN SO FAR", with chapter 1 summarised from its actual prose.
(found on the way)	A continuity fix-rewrite used to drop the scene's chapter number and facts, so fixed scenes vanished from the story's fact list.	Fixed, with a test that fails without the fix.
3. Emotional goal	The 1–10 score (nearly always 7) became a reader's multiple-choice question. "What will a reader most likely feel?" The options are the scene's goal plus 3 different feelings, written from the brief only. I read the model's own probability for the right letter.	Real problems flagged 1 → 6 of 10 ; masterpieces flagged 4 → 1 of 12; but good scenes flagged 2 → 9 of 45. Still a warning only, and it now names what a reader would feel instead.
6. Repair vs rewrite	34 scenes that fail on continuity (30 planted, 4 real). Each was fixed both ways from the same verdict.	See below

PLAN STEP	WHAT I DID	RESULT
5. Contradictions on real prose	21 MB sample of ConStory-Bench (GPT-4o's stories plus a checker's findings), downloaded with your OK. The within-scene checker read 20 stories with a known contradiction and 20 without.	Found almost nothing: 1 of 20 flagged, 0 of 20 at the right place. No false alarms (0 of 20). It stays unshipped. A real step up needs a stronger model or a trained checker, not another prompt.
7. Do gated books read better?	Not run as whole books, by your decision. The gate now steps in about once every 10–15 scenes, so a few books couldn't show a difference.	Answered scene by scene instead. Where it steps in, it's right (real contradictions, 0 false alarms), its fix works (blind read below), and it no longer harms good prose (masterpieces 11 → 0 of 12 failing).

Repair vs rewrite: the numbers

	REPAIR IN PLACE (TODAY)	FULL REWRITE
Passes the gate afterwards	20 of 33	32 of 34
Time to fix (median)	1 second	69 seconds
Original words kept	100%	about 60%
Why it still fails	continuity, all 13	1 voice, 1 continuity

Every failed repair had the same cause. The repair prompt said "change as little as possible", so the model changed "had been dead" to "had been leading a caravan" but kept "buried past the salt flats".

Fixing the repair, then a blind read

I tried two stricter versions of the repair prompt on the same 34 scenes:

REPAIR PROMPT	PASSES	SENTENCES DELETED	REAL CONTRADICTIONS FIXED (OF 4)
"Change as little as possible" (old)	20 of 34	0	2

REPAIR PROMPT	PASSES	SENTENCES DELETED	REAL CONTRADICTIONS FIXED (OF 4)
"Nothing may state or imply it"	31 of 34	32	3
Same, but rewrite rather than delete (now in the app)	31 of 34	0	3

The middle version fixed things by deleting the whole sentence, including a real scene's opening line. The adopted version keeps the sentence and removes the implication: "since the day Halim died" became "since the day the caravan left".

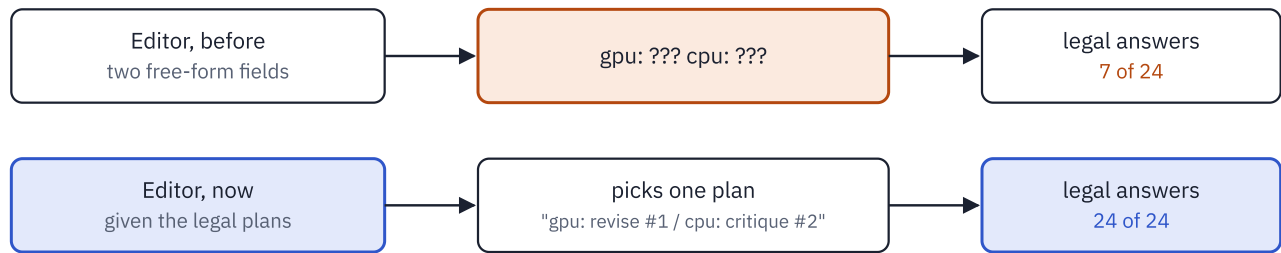
Passing the gate isn't the same as reading well. So six independent reviewers compared the repaired and the rewritten version of 30 scenes, without knowing which was which:

	REPAIR IN PLACE	FULL REWRITE
Judged the better scene (both reviewers agreeing)	11	14
Contradiction really gone	28 and 27 of 30	24 and 24 of 30
Time	~1 second	~69 seconds

On quality it's close to a tie, leaning slightly to the rewrite. But the repair removes the contradiction more reliably: six rewrites passed the gate yet still contradicted the story. The plan's rule was "keep repair only if it wins or ties at lower cost". It ties at about 1/70 of the cost, so the app keeps repairing first and rewrites only when the repair fails.

Two practical lessons from this round

- **Don't let a model fill in free fields when you have a list of valid answers.** Hand it the list. The Editor went from 7 to 24 of 24 legal answers just by being given the actual options.
- **Another program on the same GPU can wreck both.** Jester's scheduled jobs use the same model with different settings, so every alternation forced a reload of about 18 seconds. With your OK, I paused Jester for the measurements. Timings taken while it ran aren't trustworthy.



Given free fields, the 3B Editor invented combinations the pipeline can't run. Given the list of legal plans to choose from, it was legal every time.

5p. Quadtrees: would they help Versatile? (27 Sep)

You asked me to look into quadtrees and decide for myself whether they'd help the code.

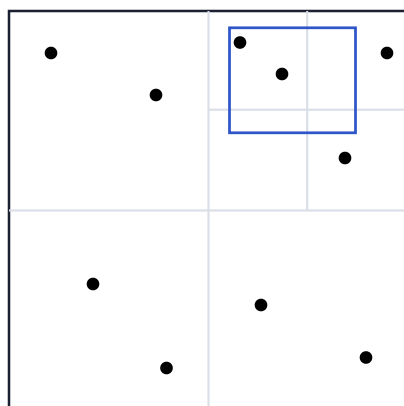
Short answer: not today. Checking gave you two real fixes anyway, just not quadtree ones.

What a quadtree is

A quadtree is a way of filing points (or boxes) on a 2D surface so you can ask "what's near here?" without checking everything. Start with one square covering the whole canvas.

When a square holds more than a few items, split it into four smaller squares, and repeat.

To find everything inside a rectangle, you skip every square that doesn't touch it. With N items, a query costs roughly $\log N$ steps instead of N .



a crowded square splits into four

query box
→

look only in squares the box touches: 2 of 7

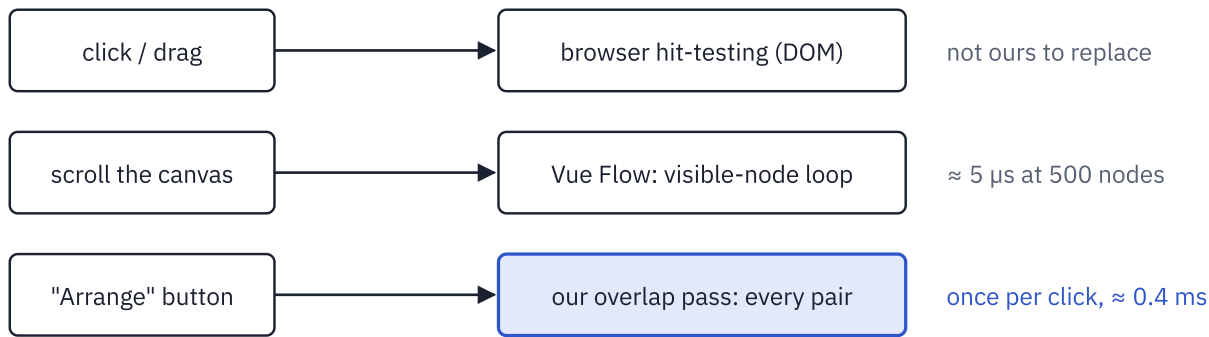
skip the rest without looking at their points

but: rebuild the tree whenever points move

A quadtree splits a crowded square into four, again and again. A search skips every square its box does not touch. The price is building the tree and rebuilding it whenever things move.

They pay off in four places: **hit-testing** (what's under the mouse), **viewport culling** (draw only what's on screen), **collision checks** (which pairs overlap), and **force layouts** (Barnes–Hut approximates distant nodes as one lump, so each step costs $N \log N$ instead of N^2 ; this is what d3-force uses). The catch is that the tree must be built, and rebuilt whenever items move.

Where Versatile does any of that



Three places a quadtree could help. The browser already does hit-testing, Vue Flow's loop is microseconds, and the one pair-by-pair loop we own runs once per click.

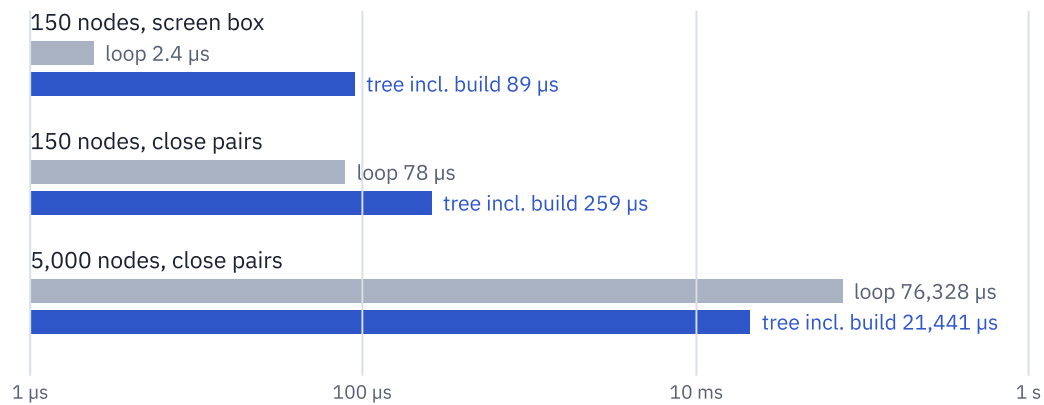
JOB	WHO DOES IT TODAY	WOULD A QUADTREE HELP?
Hit-testing (click, hover, drag)	The browser. Vue Flow draws nodes as HTML and edges as SVG, so the browser already knows what's under the mouse.	No. There's nothing to replace.
Viewport culling	Vue Flow's <code>only-render-visible-elements</code> (on in the Story Network). It checks every node against the screen: a straight loop.	No. It lives inside the library, and the loop is ~5 μs at 500 nodes (below).
Overlap fix after "Arrange"	Our code: every pair of ungrouped nodes, up to 5 passes (<code>StoryNetwork.vue:1735</code>). The only N^2 loop in the canvas.	Not at our sizes: it runs once per click, and the tree loses below ~1,000 nodes (below).

JOB	WHO DOES IT TODAY	WOULD A QUADTREE HELP?
Force layout	None. There's no d3, no simulation. Layouts are computed star and grid positions.	Only if we ever add a live force layout with thousands of nodes.
Which group a dropped node lands in	Our code: loops over the 3–15 groups.	No. 15 boxes is nothing. But this loop was wrong ; see below.

A real story network has about 20–150 nodes, 30–300 connections and 3–15 groups.

Seeing for myself: a benchmark

I wrote a plain quadtree and timed it against the simple loop on the two queries a canvas actually runs. The first is "which nodes are inside this screen-sized box". The second is "which pairs of nodes are too close", which is our overlap fix. Nodes are 160×96, spread at the same density at every size. Times are in microseconds (a 60 fps frame has 16,600).



Log scale. At a big story's size (150 nodes) the plain loop wins, because building the tree costs more than the searches it saves. The tree only pulls ahead in the thousands.

NODES	SCREEN BOX, LOOP	SCREEN BOX, QUADTREE	BUILDING THE TREE	CLOSE PAIRS, LOOP	CLOSE PAIRS, QUADTREE (INCL. BUILD)
50	1.6	1.7	102	7	233
150 (a big story)	2.4	3.6	85	78	259

NODES	SCREEN BOX, LOOP	SCREEN BOX, QUADTREE	BUILDING THE TREE	CLOSE PAIRS, LOOP	CLOSE PAIRS, QUADTREE (INCL. BUILD)
500	5.3	2.5	302	894	1,064
5,000	47	3.8	11,333	76,328	21,441
50,000	1,808	6	232,432	too slow	433,196

Node 24, one run, `scratchpad/quadtrees-bench.mjs`. The tree answers screen queries very fast at 50,000 nodes. But at our sizes the plain loop is already a few microseconds, and building the tree costs 30–100× more than the loop it would replace. On a canvas where you drag nodes, the tree has to be rebuilt on each move.

Verdict

Don't add a quadtree. At 150 nodes the loops it would replace take 2–80 μ s, well under 1% of one frame, and the tree is slower once you count building it. It starts to win around **1,000+ nodes** for the pair check and **5,000+** for screen queries. No novel's story network gets there.

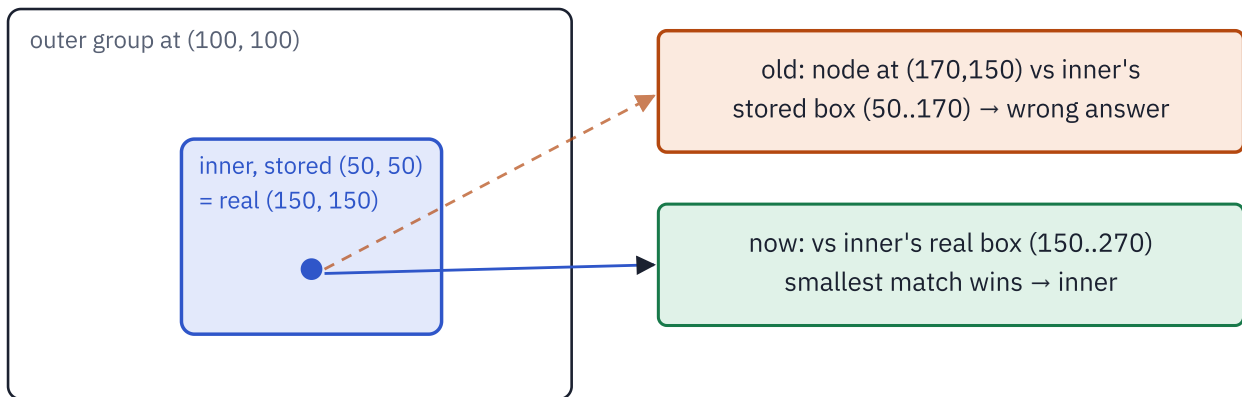
When to revisit: if the canvas ever shows thousands of items (every scene of a series as a node, say) or gets a live force layout. Even then, use a tested library (`d3-quadtrees` , or `rbush` for boxes) rather than writing one.

What the search did find: a real bug in the group code

Reading every "is this point inside that box?" check for the quadtree question turned up two bugs in how nodes land in groups:

- **Nested groups were measured in the wrong place.** A group inside another group stores its position *relative to its parent*. When you dragged or dropped a node, the code compared the node's real canvas position against that relative position. A node dropped into a nested group could land in the outer group, in no group, or in the wrong one. It also took the first group in the list rather than the innermost. Dragging a *group* was already correct. The code for nodes just never used the same method.
- **Dropped nodes landed shifted.** When you drag an entity from the sidebar onto the canvas, the code subtracted the canvas's position on the page and then called Vue Flow's

converter, which subtracts it again. Every drop landed off by the canvas's distance from the page's top-left corner. (This bug dates from the first commit.)

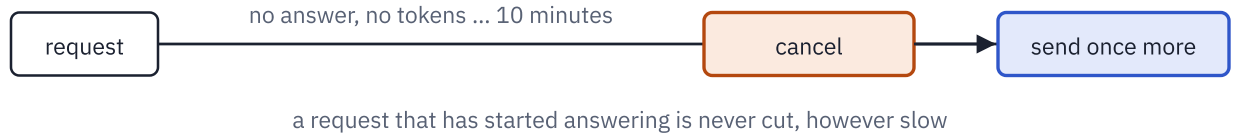


A nested group stores its position relative to its parent. The old check compared a real position against that relative one. It now compares real against real, and the smallest group containing the point wins.

Fixed (`f6366958`): one small helper, `innermostGroupAt`, finds the smallest group whose *real* box holds the point. It now handles node dragging, drag highlighting and dropping. The drop passes the mouse position straight to Vue Flow. A group dropped into a group now also records its new parent (before, it moved but kept the old one). 4 new tests, including "a point inside the nested group's *stored* box but outside its *real* one belongs to no group", which the old logic got wrong.

The two hardening tasks, also done

- **The hung-request guard.** Two live runs hung forever: Ollama accepted a request and never answered, and with time limits off (the default) nothing gave up. Now, if a request produces *nothing at all* for 10 minutes, it's cancelled and sent once more. A slow request that has started streaming is never cut, even when it's slow. 4 new tests, and they fail if the guard is removed.
- **The flaky test.** The map-upload test waited a fixed 20 ms for the browser to read a file. Under the load of the full suite that sometimes wasn't enough. It now waits until the write actually happens.



With time limits off, a request Ollama accepted but never answered hung forever. After 10 silent minutes it is now cancelled and sent once more.

Full suite: **3,062 tests pass**; typecheck, lint and repo policies clean. Committed locally, not pushed.

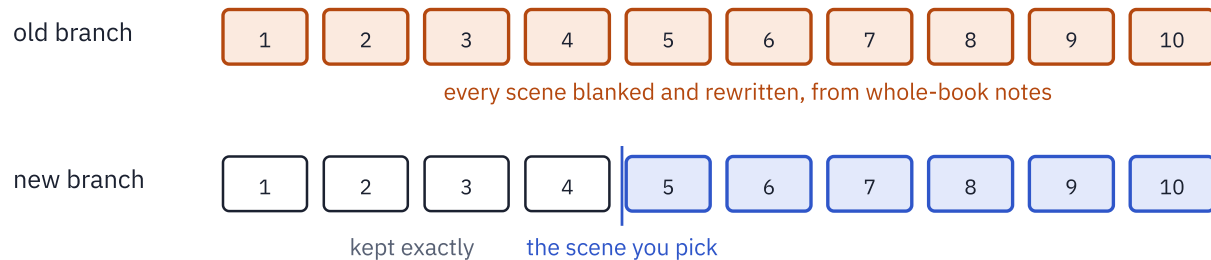
5q. What If on any novel: import it, understand it, branch it (27 Sep)

You asked for the What If feature to work on any novel, including one you import. So the first step was to check both halves: What If itself, and importing. Both had bigger problems than expected. The full plan is in `docs/WHATIF-AND-IMPORT-PLAN.md`.

What I found

- **There was no novel importer at all.** "Import" only restored a Versatile backup file, and even that was broken. It gave every row a new id but kept the old links, so scenes pointed at chapters that didn't exist.
- **The recovery backup had no prose in it.** Its list of tables had fallen behind the database and was missing chapters and scenes. Restoring it wipes the database first, so a restore would have deleted every scene.
- **"Rewrite the rest as a branch" ignored the scene you picked.** It blanked and rewrote the *whole* book, using a bare one-shot prompt instead of the real writer and gate.
- **Its "story so far" included the future.** It gave the model the whole book's notes as canon, including the events the What If is supposed to change.
- **Accepting a branch copied nothing back.** It matched scenes by the wrong chapter id, and the accept/delete list was never shown anywhere in the app.

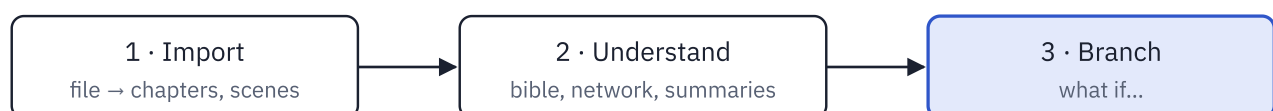
- **Branches in general were broken.** A new branch was an empty row, so switching to it showed an empty book. Opening a second project kept the first project's branch. Older, imported or template rows had no branch, so they were invisible to every branch-filtered read.
- **An imported or hand-written book gets almost no story knowledge.** Scene summaries and facts are empty, and nothing reads characters, places or relationships out of prose. The book's genre and arc exist only inside a generation run. "Refresh voice from manuscript" was broken for every project.



Old: the whole book was rewritten, whatever scene you chose. New: everything before the chosen scene is kept exactly.

The plan, in three moves

1. **Import.** Read the file, show the chapters found, and let you fix them before anything is saved.
2. **Understand.** Read the book once in the background, on your GPU, and build the same records a generated book has: summaries, facts, cast, the story bible, relationships, a story profile and the search index.
3. **Branch.** Pick a scene and describe the change. Versatile proposes, for each later scene, keep / rewrite / drop, and you edit that plan. It writes with the real writer and gate, using only what happened *before* the change as canon. Then you compare scene by scene and merge back, with a snapshot taken first.



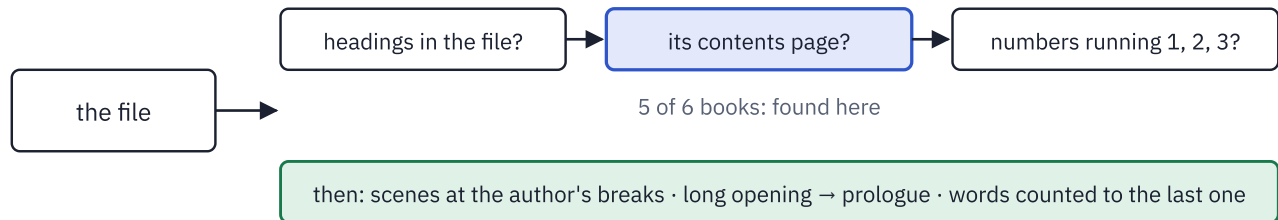
What If needs what the book knows about itself, so an imported book goes through import and understanding before it can branch.

Done so far

STEP	WHAT CHANGED	PROOF
1. Data safety (2c10308b)	Rows with no branch join the main branch when a project opens. A new branch copies the book, and every copy remembers which scene it came from. Deleting a branch cleans up its rows. Switching reloads the book. The recovery backup covers every table and never wipes a table the backup doesn't contain. The backup import relinks everything. Voice extraction reads the scenes.	11 tests on the real database; the import tests fail on the old code
2. Import a novel (2964895c)	.txt, .md, .docx, .epub, .html. Chapters come from the file's headings, else the book's own contents page, else numbered headings. Scenes split at the author's breaks. A preview lets you rename, merge or leave out chapters. It's on the projects page, in the project menu and in Ctrl+K.	6 real books (below); tried in the running app on <i>Ethan Frome</i> and <i>Dubliners</i>

BOOK	CHAPTERS FOUND	HOW	WORDS KEPT
Chekhov, <i>The Lady with the Dog</i>	9 of 9	contents page	every word
Mansfield, <i>The Garden Party</i>	15 of 15	contents page	every word
Doyle, <i>Sherlock Holmes</i>	12 of 12, with each story's parts I, II, III as scenes	contents page	every word
Joyce, <i>Dubliners</i>	15 of 15	contents page	every word
Wells, <i>The Time Machine</i>	17 of 17	contents page	every word
Wharton, <i>Ethan Frome</i>	10 of 10 (the frame story as a prologue, then I–IX)	Roman numerals	every word

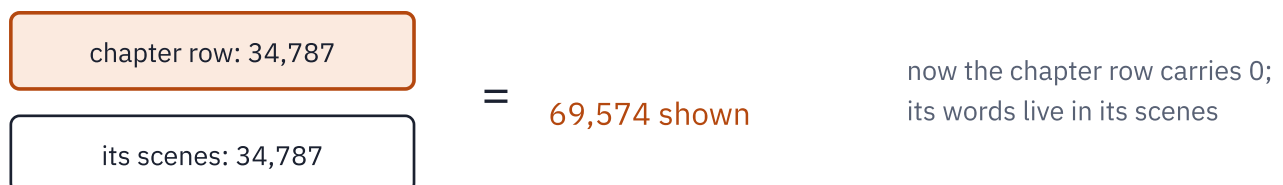
"Every word" is checked exactly: words in scenes + front matter + back matter + headings = words in the file. The test fails if a single word goes missing. It already caught one bug: "_maman_'s" is one word once the italics are removed, not two.



The detector tries the most trustworthy signal first. A book's own contents page beats any guess about what a heading looks like.

Three bugs the live try caught that tests had not

- **Double word count.** The first real import showed *Ethan Frome* as 69,574 words, exactly twice its length. The chapter rows carried their scenes' word count, and every counter adds chapters and scenes together. Fixed before the commit, and there's now a test for it.
- **Numbers vs text.** In step 1 the branch store looked up project ids as text, but real project ids are numbers, so it matched nothing. The step 1 tests used text ids and passed anyway. The import test with a real id caught it.
- **No way in for new users.** The projects page, the first screen a new user sees, had no import button. It now does.



Every counter adds chapters and scenes together, so an imported chapter that also carried its scenes' total was counted twice.

Next

Step 3, understanding the book, is in progress. For each scene: a summary, key facts, who is present, where it happens and whose point of view. For the whole book: characters merged across name forms ("Holmes" = "Sherlock Holmes"), relationships with the chapter they start in, a story profile and the search index. Then **step 4**, the What If branch itself. Jester is paused with your OK so the timings are clean; its six tasks are saved and will be restored exactly.

Still open from earlier: the within-scene contradiction checker needs a stronger model (a personal Groq key, or your OK to download a larger local model), and your 12 scenes still need labels.

5r. Step 3: Versatile reads an imported book, and should it use a message broker?

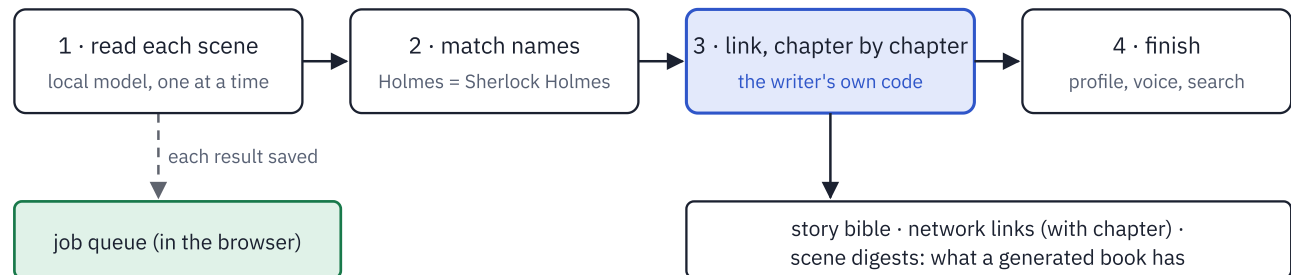
An imported novel arrives as plain chapters and scenes. A generated book arrives with much more: scene summaries, key facts, who is in each scene, a filled story bible, and a story network whose links remember the chapter they started in. What If and every other tool read those. Step 3 makes an imported book end up with the same things.

How it works, in plain words

1. **Read each scene.** The local model reads each scene (passages of up to 2,500 words) and writes down: a short summary, whose point of view it is, where it happens, the named people and places, up to 5 facts later scenes must respect, and the relationships it shows. Each result is saved the moment it's done.
2. **Match names across the book.** "Holmes", "Mr. Holmes" and "Sherlock Holmes" become one character. The rule: a short name fits the one full name that contains it. When it fits two ("Frome" could be Ethan *or* Zeena), code never guesses. The model picks between the two, and the merge only happens if it is at least 60% sure. Merging two people is worse than keeping one person twice.
3. **Link, chapter by chapter, through the writer's own code.** Each scene's reading is turned into exactly the record the writer produces for a scene it has just written. It then goes through the *same* functions a generated scene does. Those create the characters and places, add them to the network, stamp each relationship with its chapter, and write the

scene digests. So an imported book and a generated book really are handled alike, not merely look alike.

4. **Finish.** Chapter summaries, a story profile (genre, tone, premise, central conflict, themes, setting), the voice profile, and the search index.



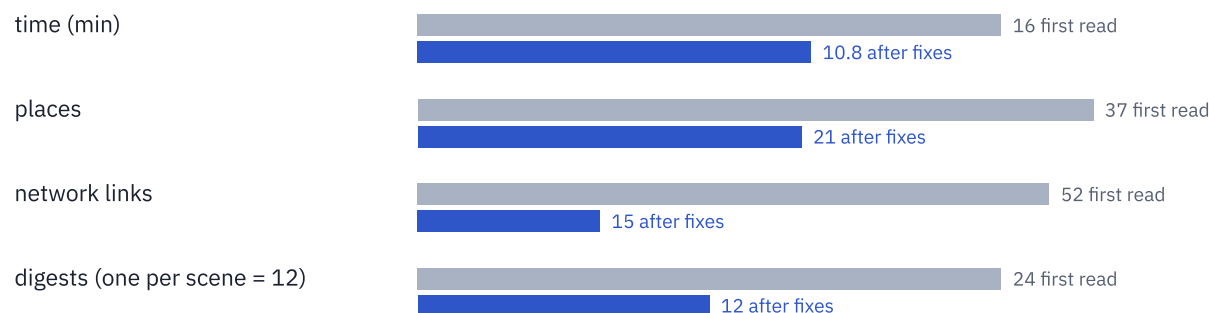
stopped or crashed? the next run resumes at the next unread scene

Each scene's reading is saved the moment it is done, so a stop loses nothing. The linking step feeds every scene through the same code that fills the bible for a generated book.

It starts from a bar that appears on an imported book ("Versatile has not read this book yet... Read the book"). For any book, Ctrl+K → "Understand this book" does the same. The bar shows scene N of M, the time left, and a Stop button. Stop, a closed tab or a crash loses nothing: the next run carries on from the next unread scene. I tested that for real by accident, because my own edit reloaded the page mid-run, and it resumed at scene 1 of 12 without re-reading the prologue.

The real test: Versatile reads *Ethan Frome*

34,787 words, 12 scenes, on your RTX 4060 with the local model (qwen3:8b), with Jester paused so the timing is clean. The first read worked from start to finish, and it also exposed eight defects that no test had caught. I fixed all eight, each with a test, and read the book again.



Ethan Frome, first read versus after the fixes. Every bar that shrank was a defect: repeated places, churning links, duplicated digests, and 30-second pauses.

	FIRST READ	AFTER THE FIXES
Time for the whole book	≈ 16 min	10.8 min
Characters in the story bible	7, but "Ned" (her husband) merged into "Mrs. Ned Hale"	7, all correct: Narrator, Ethan Frome, Zeena Frome ("Ethan's wife"), Mattie Silver, Denis Eady, Jotham Powell, Mrs. Ned Hale
Places	37, many repeated ("Frome farm" / "The Frome Farm" / "Farm") and bare nouns ("Kitchen")	21 , all named places
Links in the network	52, including "Ethan <i>father of</i> Mattie" and "Ethan <i>married to</i> Mattie"	15. "Ethan married to Zeena" runs from chapter 1 and is never closed. "In love with Mattie" runs chapters 2–9, then becomes "formerly close to" in chapter 10. Mattie is Zeena's cousin.
Scene digests	24 (each scene twice)	12, one per scene , each with a summary and 5 facts
Story profile	"Realistic fiction · melancholic, tragic · a man trapped in a loveless marriage grapples with forbidden affection · themes: love vs duty, isolation, fate, sacrifice". Accurate both times.	

What remains is the model's own misreadings (for example, Zeena "recommending" Denis as a match). An 8B model reads a novel the way a hurried student does: well enough to build the notes, not well enough to trust every line.

The eight defects the live read found

1. **Every scene got two digests.** Scene ids are numbers, and I passed them as text. The database treats "5" and 5 as different, so it kept both.
2. **The book was linked twice.** When I edited code mid-run, the page reloaded, and the old copy of the reader kept running next to the new one. Both took jobs from the queue. That's a dev-server accident, but two browser tabs would do the same thing. Fix: taking a job is now atomic (one database transaction), and only one reading per book can run at a time, across tabs.
3. **"Zeena (his wife)" became a name.** The model's note in brackets is now removed.
4. **"Ned" was merged into "Mrs. Ned Hale".** Titles are no longer thrown away: "Mrs." says she isn't Ned.
5. **Places were repeated and generic.** Places are now merged the way names are. Bare nouns ("the kitchen") only enter the bible if three or more scenes happen there.
6. **Family misreadings.** A family link now needs two scenes that say it. Marriage is also exclusive: the model married Ethan to Mattie in two scenes, against five that married him to Zeena, and now the five win.
7. **The network churned.** Each chapter worded the same relationship differently ("loves", "has feelings for", "emotionally attached"), and the network closes a link when its type changes. Labels now fold into about 15 types, and the book is walked once per pair. A marriage stays; a passing mood doesn't replace it.
8. **A link to someone not yet in the bible vanished silently.** Both ends of a link are now always added first.

Ethan ↔ Zeena, chapters 1 → 10

1 2 3 4 5 6 7 8 9 10

before

married

neglect

married

cares

married

married

after

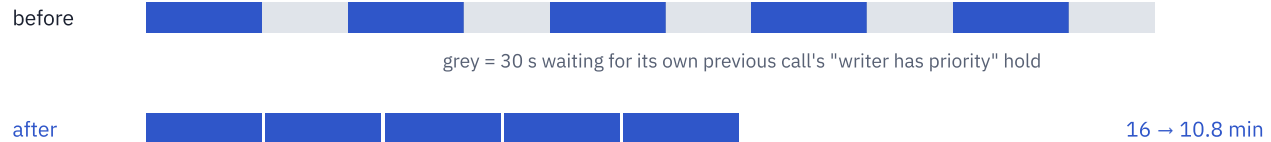
married to, chapter 1 onward, never closed

labels folded into ~15 types · a mood never replaces a marriage · a family link needs two scenes

Before, each chapter's differently worded description of the same marriage closed the previous link. Now, one link lasts until something actually changes.

And one speed-up, found by timing it

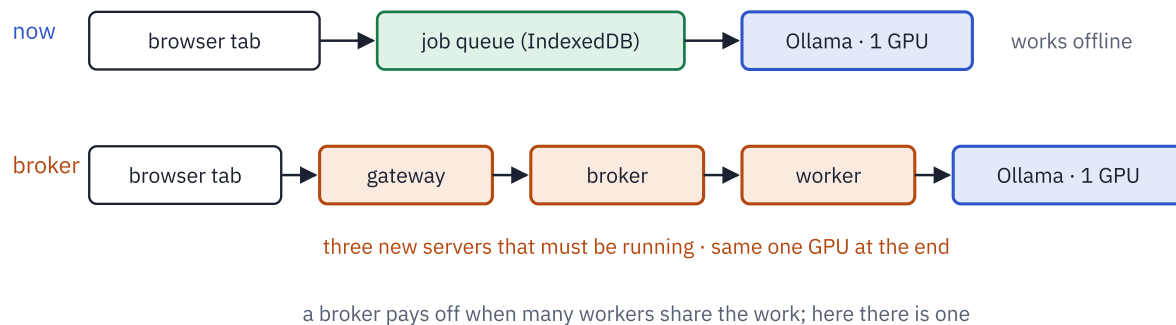
The log showed the GPU sitting idle for exactly 30 seconds between scenes. The app gives the writer priority: after any "foreground" call, background work waits 30 seconds. The reader's own calls were counted as foreground, so it spent a third of its time waiting for *itself*. Background calls now take their turn on the GPU without claiming priority. Result: 16 → 10.8 minutes.



Every call the reader made marked the GPU as busy with priority work for 30 seconds, and then the reader waited for that mark to clear. Background calls no longer set it.

Your idea: a message broker like RabbitMQ or Kafka

A fair question: processing a whole book is exactly the kind of long, many-step job brokers exist for. I checked it against how Versatile actually runs.



Both paths end at the same single GPU. The broker adds three servers that must run, and takes away working offline, without adding any speed.

	RABBITMQ	KAFKA	WHAT VERSATILE USES NOW
What it is	A job-queue server: messages, acknowledgements, retries, dead-letter queues, priorities	A distributed, append-only event log: partitions, replay, many consumer groups	A job queue stored in the browser's own database (IndexedDB, <code>analysisQueue</code>), already in the app since schema v46
Built for	Many producers and many workers sharing tasks across machines	High-volume event streams that many systems read and re-read	One writer, one GPU, one machine, working offline
Would it make a book faster?	No. The limit is one GPU that serves one request at a time. That's on purpose: the code records that running calls side by side made 2-second requests take 270 seconds. A broker hands work to workers, and here there is only one.		Same speed; nothing extra to run
Keeps work across a crash, retries, a pile of failed jobs, ordering, stages	Yes	Yes	Yes. I used it for this and added the two missing pieces: jobs claimed by type, and each job keeps its result
Cost of adding it	A server process that must be running. A browser can't speak its protocol directly, so it also needs a gateway (RabbitMQ's Web-STOMP plugin, or a Kafka REST proxy). An offline feature would come to depend on the backend. Kafka also needs a JVM broker service.		None

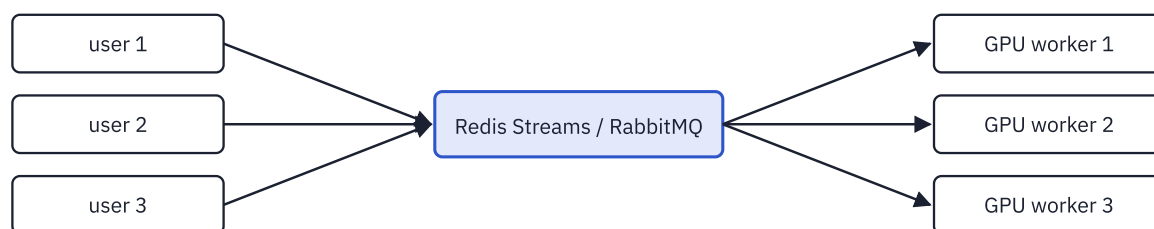
Everything a broker would give this job (durability, resume, retry, dead letters, ordering, stages) the browser-side queue already gives, without adding a server.

Verdict: not now. Here is when it would be right.

Keep the in-browser queue for one writer on one machine. A broker becomes the right tool the day book analysis moves to the server: many users importing at once, or a pool of GPU workers. Even then, the backend already has what to build on. It has a Postgres "outbox" table with a background worker, and it runs Redis. So the natural first step is

Redis Streams or **RabbitMQ** through **MassTransit** (.NET), with the browser just showing progress. **Kafka** only makes sense at event-stream scale (many services reading the same history), which Versatile doesn't have.

What I took from your idea is its core: the pipeline is written as **stages with durable jobs** (read → match names → link → finish). That's exactly the shape that could move behind a real broker later without being rewritten.



The day analysis moves to a server, with many users and several GPU workers, a broker earns its place. The backend's Postgres outbox and Redis are the natural starting point.

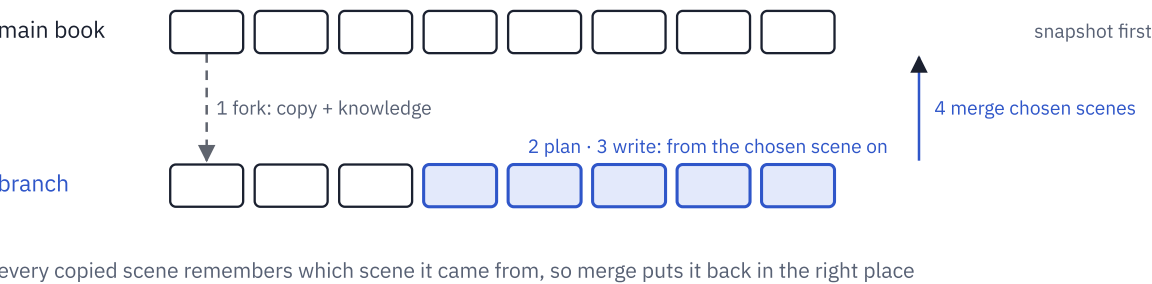
5s. Step 4: What If, rebuilt, and tried on *Ethan Frome*

The old "rewrite the rest as a branch" ignored the scene you picked, rewrote the whole book from notes that included the future, and its "accept" copied nothing back. What If is now four steps you can see and control.

1. **Fork.** The whole book is copied into a new branch, together with what Versatile knows about it (scene summaries, facts, the timeline). Every copied scene remembers the scene it came from. Story knowledge now lives per branch, so a What If can never overwrite what the main book knows (schema v55).
2. **Plan.** For each scene after the change, Versatile decides: **keep**, **rewrite** (with a one-line brief of what it must now show), or **drop**. You see every decision and can change any of them before anything is written.
3. **Write.** Dropped scenes are removed. Rewritten scenes are written in order by the real writer and quality gate. Each is written against the prose before it and against "the story up to the change + the change as a fact". It never sees the notes about the book's original

future. Kept scenes are then checked against the change, and any sentence that contradicts it is repaired.

4. **Compare and merge.** Each changed scene shows before and after. You tick the ones you want. Each scene it replaces is saved to that scene's history first, so you can restore it.

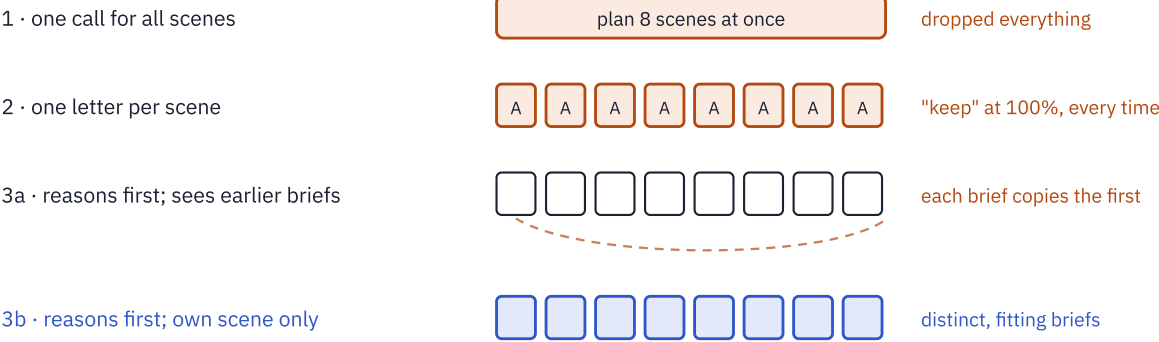


A What If is a full copy of the book. Only the scenes from the change on are planned and rewritten, and only the scenes you tick go back into the main book.

It works on any book. An imported book only needs to have been read (step 3) so the planner has scene summaries to work from.

Getting the plan right took three tries, each measured on the real book

The test: *"What if Zeena never goes to Bettsbridge, and stays home the night Ethan and Mattie were to be alone?"*, diverging at chapter IV.



Four ways to ask the same model the same thing. Only the last, one question per scene with room to reason and no view of the other answers, produced a usable plan.

TRY	HOW THE PLAN WAS MADE	WHAT CAME BACK
1	One call plans every later scene at once	Every scene after the change dropped. Scene V's brief ("free from Zeena's presence") contradicted the change it had just stated.
2	One question per scene, answered as a single letter read from the model's probabilities	"Keep as written" with probability 1.00 for all seven scenes, including the evening that only happens <i>because</i> Zeena is away. It gave the same answer with the options reordered, so this wasn't position bias: the model had no room to think.
3a	One question per scene, reason first ("what does the scene need? what does the change make false? so: keep/rewrite/drop"), with briefs seeing the new version so far	Sensible decisions. But from chapter V on, every brief was a copy of the first one ("a charged, intimate moment in the kitchen").
3b	Same, but each brief sees only the change and its own original scene	Each brief grows from its own scene. Zeena's hired-girl plan goes away. The money problem stays. Mattie leaves, and there's no sled crash. Planning takes about 50 seconds.

Three rules this taught (they generalise beyond What If)

- **One small question per call.** Asking an 8B model to plan eight scenes at once got nonsense. Asking eight questions got sense. The gate work found the same (§16–17).
- **If the answer needs thinking, let it think first.** Reading one letter's probability worked for look-ups ("do these two quotes contradict?", §33). It failed here, where the answer needs a step of reasoning.
- **A small model copies whatever example it sees.** Showing it earlier answers made every later answer the same. A question about one item should show only that item.

The real branch: writing it

With the plan from try 3b, and two briefs corrected by hand (the plan is editable for exactly this), Versatile wrote the branch with its real writer and quality gate on your GPU.



identical to the book (4/4)

rewritten: Zeena at home

IX-2 dropped

"after Zeena left"

kept: sled crash

The live Ethan Frome branch. The scenes before the change were untouched. The change carried through five of six rewritten scenes. Chapter VIII and the kept epilogue did not follow it.

CHECK	RESULT
Scenes before the change unchanged (gate G6)	Yes: 4 of 4 byte-identical to the original book
Time	Plan $\approx$ 50 s. Writing 6 scenes ($\approx$ 23,000 new words): 54 min , with Jester sharing the GPU
Does the change happen?	Yes. Chapter IV opens with Zeena at the breakfast table; "the hope crumpled underfoot, crushed by the weight of Zeena's decision."
Do later scenes respect it? (gate G7)	Mostly. Zeena is present in V, VI, VII and IX ("Zeena sat at the table, her back stiff..."). Chapter VIII fails: "Zeena's absence was like a door left open...", "after Zeena left". It even gives Mattie's broken pickle dish to Zeena.
The kept epilogue	Its check found nothing, yet it still tells of the sled crash, which the rewritten chapter IX no longer has.

Why, and the fix (in the code now)

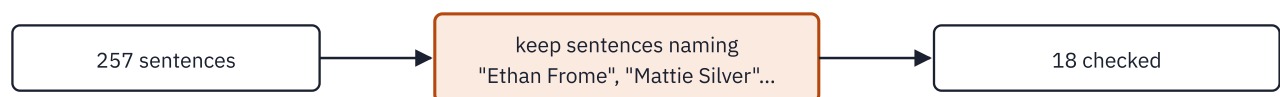
Two gaps. **Rewritten scenes were never checked against the change:** the writer's own quality gate doesn't know about it. And **kept scenes were checked against the change alone**, not against what the rewritten scenes now say happened.

Now, after writing, Versatile reads each rewritten scene (which also updates the branch's own knowledge). Then it checks **every** scene after the change, in order, against the change *plus* the facts of the rewritten scenes before it, and repairs any contradicting sentence. A "Check again" button runs this again after you edit.

Result on this branch (28 Sep, 16 minutes): it read the six rewritten scenes and checked all seven scenes after the change.

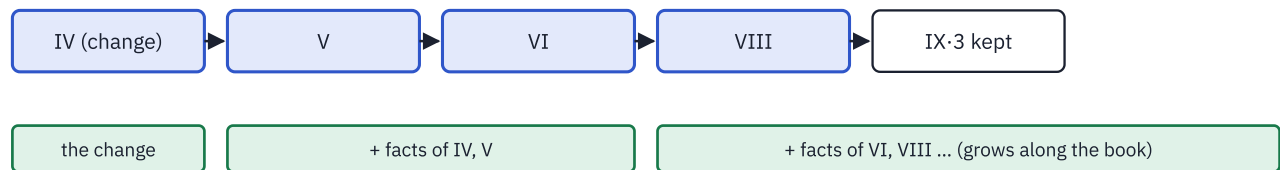
- **The kept epilogue was caught.** The check found its contradictions with the new ending and marked it *needs review*. The automatic sentence repair couldn't be placed, so it was flagged for you rather than silently kept.
- **Chapter VIII was not caught.** It still says "Zeena's absence..." and "after Zeena left", and the check passed it. Diagnosed, with two causes:
 1. **The stated change was too narrow.** You asked for Zeena to stay home "*the night Ethan and Mattie were to be alone*". The planner boiled that down to "Zeena decides to stay home instead of going to Bettsbridge" and dropped *that night*. Against that sentence, "after Zeena left" (the room?) isn't a strict contradiction, and the checker is built to stay silent when in doubt (zero false alarms was its design goal). **Fix:** the stated change must keep the premise's time and place.
 2. **Only 18 of the chapter's 257 sentences were checked at all.** The checker only reads sentences that name a character the way the bible spells it ("Ethan Frome") or as the facts it is given name them. Against the one-line change, that left 18. Measured in 5t: widening it adds false alarms, not catches.

Honest state: the check catches some contradictions (it caught the epilogue) and misses others. The compare screen shows every rewritten scene before you merge, and that is where the last catch happens.



the prose says "Ethan" and "Mattie": most sentences never reach the checker
(measured before changing it: see 5t)

The checker only reads sentences that name a character the way the bible spells it.



each scene is checked against everything the new version has already said, then contradicting sentences are repaired

Before, only kept scenes were checked, and only against the change. Now every scene after the change is checked, in order, against the change plus the facts the rewritten scenes before it established.

Also found on the way

- **The plan didn't appear after planning.** The page copied it with a function that refuses Vue's reactive objects, and the error was swallowed. Fixed.
- **Imported books count as words you wrote.** The projects page says "312,896 words this week", which is the six imported books. **Diagnosed:** the statistics count each project's day-to-day growth starting from zero, so the day a book is imported looks like one enormous day of writing. It also inflates the streak and "best day". **Fix, being made now:** the import records how many words the book arrived with, and the statistics start that project from there. Books imported earlier start from their first recorded day.
- **A test that timed out under load** (it loaded every AI module to reach one fake). Fixed.

Where it stands

All four steps are built and pushed ([a2c84aef](#)). You can import any novel, have it read, branch it at any scene, see and edit the plan, write it with the real writer, compare, and merge scene by scene with a snapshot of each replaced scene. Its honest limit is the 8B model: about 2 of 7 plan briefs needed a correction, and one of 6 rewritten scenes contradicted the change until the new check existed. The plan and the check are what make that acceptable. You see what it will do before it does it, and what it did is checked afterwards.

5t. Fixing what the live branch showed (28 Sep)

Three items were open after the live What If run. Each was fixed or measured, and the code is pushed ([4a45a386](#)).

1. Imported books counted as words you wrote. Fixed.

The projects page said 312,896 words "this week": the six test imports. The statistics store each project's total per day and count the difference from the day before, starting from zero. So an import day looked like a whole book written in a day, and it inflated the streak and "best day" too. The import now records the size the book arrived with, and the statistics start from there.

stored per day: the project's total

import day: 35,000

next day: 35,400

words written = total – previous total

before $35,000 - 0 = 35,000$ "written" on import day, then 400

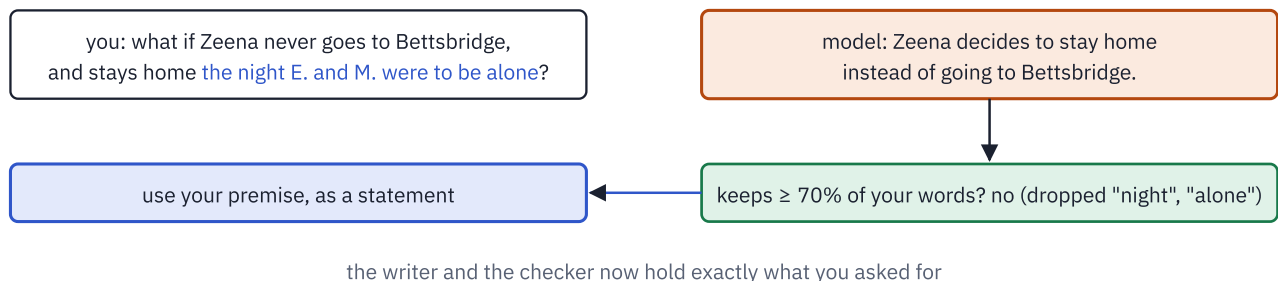
now $35,000 - 35,000$ arrived with = 0, then 400

streak, best day and "this week" all follow from the same subtraction

The statistics subtract yesterday's total from today's. For an imported book, "yesterday" is now the size it arrived with, not zero.

2. The stated change was too narrow. Fixed.

The planner turned your "stays home *the night Ethan and Mattie were to be alone*" into "stays home instead of going to Bettsbridge". Versatile now keeps the model's wording only if it keeps most of your premise's words. Otherwise it uses your premise itself, as a statement.



The planner's one-line version of the change is used only if it keeps your premise's key words. Otherwise your own sentence, turned into a statement, is used.

3. The name filter. Measured, and left as it is.

The continuity check only reads sentences that name a character the way the bible spells it. I measured widening that to first names and aliases, on the original *Ethan Frome* text, where any flag is a false alarm. It read 6% more sentences and added one false alarm, and it caught nothing new. Most first names already reach the check through the facts it is given. So the change wasn't shipped. (The "18 of 257 sentences" in 5s was chapter VIII checked against the one-line change alone; with the full premise it reads 50.)

full bible names (kept)

429 sentences read

0 false alarms

+ first names, aliases

453 sentences read

1 false alarm

clean original chapters I–VIII, each against the true facts of the chapters before it · new catches: 0

Widening the name filter read 6% more sentences, raised one false alarm where there had been none, and caught nothing new. So it was not shipped.

And why chapter VIII still passes: a limit, stated plainly

Even against your full premise, chapter VIII ("after Zeena left") is not a contradiction. The premise is about one night, and chapter VIII is a later day. What's wrong is that the new version never shows Zeena leaving: a **missing event**, not a contradicted fact. A fact checker can't see a missing event. Catching it needs a check on who is where from scene to scene (the entity timeline Versatile already keeps), and that's the next thing on the list. Until then the compare screen, which shows every rewritten scene before you merge anything, is the safeguard.

IV: stays

V: home

VI: home

VII: home

??? leaves

VIII: gone

the premise covers one night

nothing shows her leaving

no fact says "Zeena is still at home", so a fact check has nothing to hold VIII against

Chapter VIII does not contradict any stated fact. It skips an event, Zeena leaving, that never happens. Catching that needs tracking who is where, scene by scene: built in 5u.

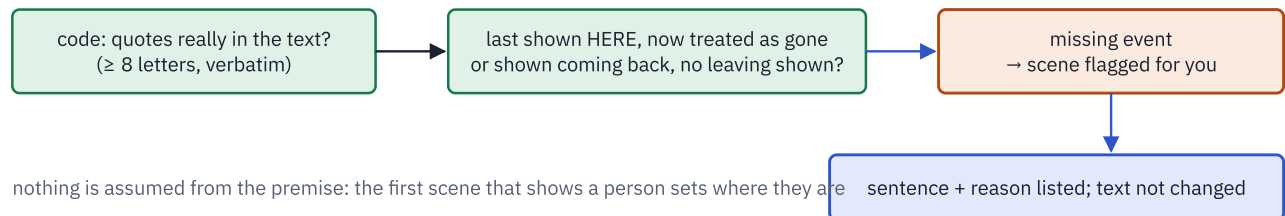
5u. Who is where: catching a missing event (28 Sep)

5t ended on a limit. In the *Ethan Frome* branch, chapter VIII says “after Zeena left”, but the new version never shows her leaving. No stated fact is broken, so the fact check can't see it. So I built a second check that follows each person the change names through the scenes, in order.

one person (here Zeena), the scenes after the change, in order



each scene: ONE question (reason first) → here? where? a leaving shown? a return shown? a sentence treating them as away?



The check follows each person the change names through the later scenes. Code, not the model, decides that an event is missing: last seen here, now treated as gone, and no scene shows the leaving. It flags the scene; it does not edit it.

The model answers one narrow question per scene and quotes the text. Code checks the quotes and makes the call. A flagged scene goes to “needs review”, and the branch screen lists the sentence under it with the reason. It took four live runs to get here, and the last one changed the design.

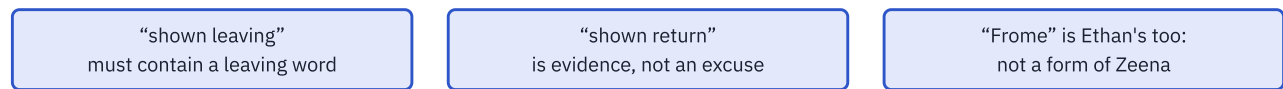
The first live run missed it, and why

I ran it on the real branch before calling it done. Chapter VIII passed. Replaying Zeena's scenes one by one showed the reason: asked for “a move shown”, the model quoted “Zeena's return came late”. A return is exactly the proof that she left off-page, but the rule counted it as the move that explains her absence. The model also quoted plain presence (“Zeena stood at the door”) as moves.

first live run: chapter VIII passed



now



the model had also quoted "Zeena stood at the door" as a move; the leaving-word rule stops that

One question about "a move" let the model excuse the missing event with the very sentence that shows it. Leaving and return are now separate questions, and code checks that a quoted leaving says one.

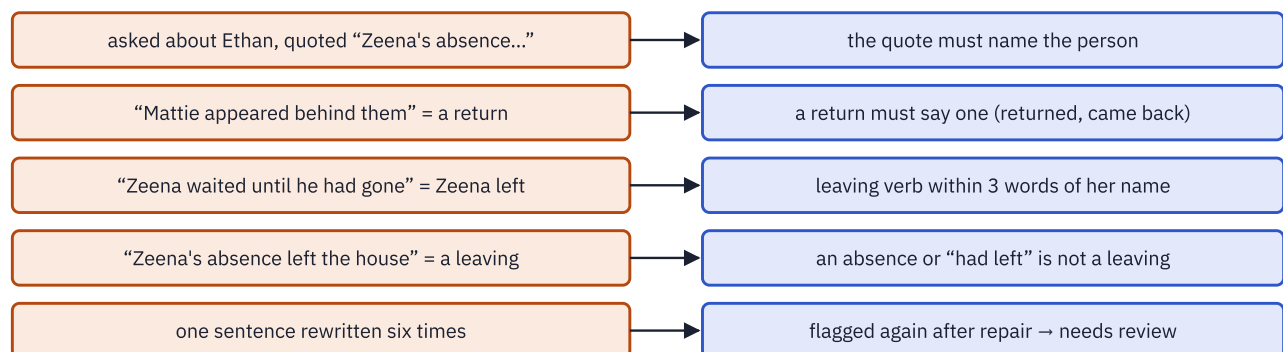
So I split the question. A leaving and a return are now asked for separately, and code checks that a quoted leaving really says one. "Frome" is Ethan's surname too, so it no longer counts as a name for Zeena.

The second run: three false alarms, and still a miss

The next run failed in new ways, each one visible only on real text. Every failure is now a rule in code, not in the prompt, with a unit test that fails without it.

what went wrong in run 2

code check now



The second live run failed in five different ways. Each is now a check in code and a unit test that fails without it.

Then I ran just the question on the branch, with no changes made to the text, to see every flag. There was one flag, in the right place, and no false alarms in 20 other reads.

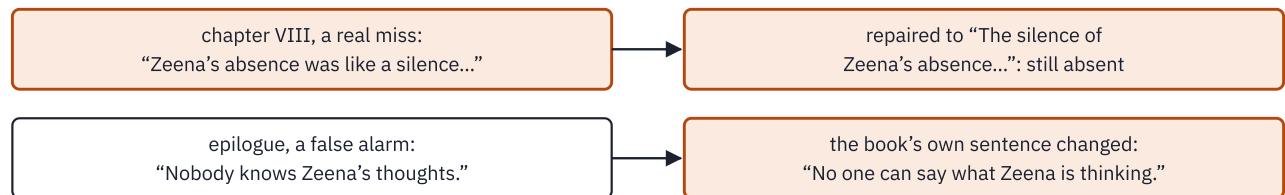
	IV	V	VI	VII	VIII	IX	epilogue
Ethan	here	here	here	here	here	here	not here
Zeena	here	here	here	here	flagged	here	not here
Mattie	here	here	here	here	here	here	not here

21 reads, 2.4 minutes, repairs off · 1 flag, where the book really breaks · 0 false alarms

On the real branch, after the fixes: the only flag is Zeena in chapter VIII. The epilogue, set years later, has people away without a flag, because nothing there claims a leaving the story skipped.

Runs 3 and 4: it catches chapter VIII, and repairing is the wrong response

Run 3 caught chapter VIII but couldn't place the fix, because the model had wrapped its quote in quotation marks. Run 4 settled the design. The one-sentence repair kept Zeena absent in new words, round after round. On one false alarm in the kept epilogue, it changed the book's own sentence. A scene built on an event that never happened needs a rewrite or the author, not a patched sentence. So the check now **flags and never edits**. I put the two epilogue sentences back from the original on the test branch.



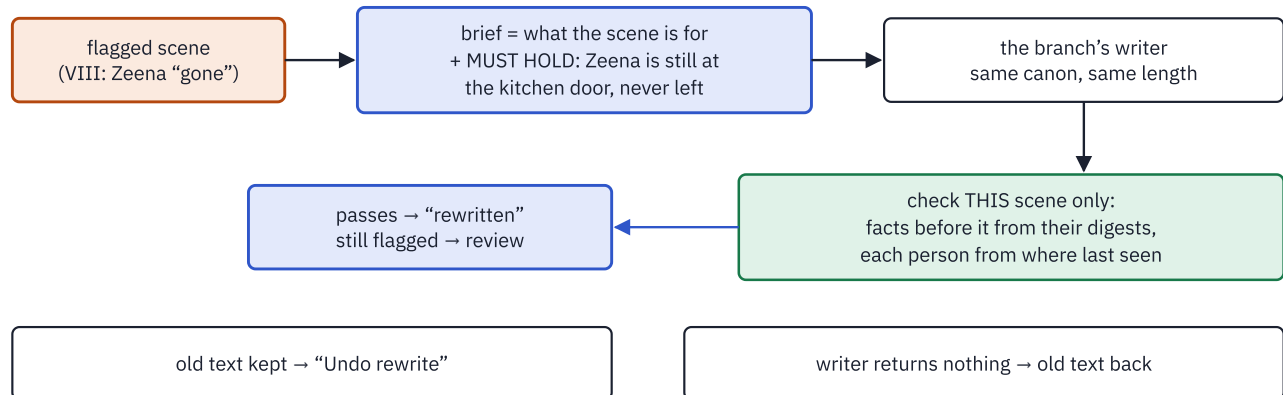
Run 4: patching one sentence cannot remove an absence a whole chapter is built on, and on a false alarm it edits the original book. So the check now only flags.

Where this leaves What If. Chapter VIII no longer passes silently: it shows as “rewritten, needs review”, with the sentences and the reason. The “Rewrite this scene” button that fixes it is in 5v. The live verify on this branch takes 10–30 minutes on the local model.

Code: `bec67744` , pushed.

5v. “Rewrite this scene” (28 Sep)

5u could find chapter VIII’s missing event but not fix it. Patching one sentence kept Zeena absent in new words. The tool that can change a chapter built on an event that never happened is the writer itself, told that it didn’t happen. So every flagged scene now has a **Rewrite this scene** button.



One click writes the scene again, knowing the missing event never happened, then checks only that scene. Nothing else in the branch is touched, and the old text can be put back.

The brief is what the scene is for, plus one rule per person the check flagged, taken from the check’s own finding. The writer is the same one the branch used, with the same canon, at the old scene’s length. Only that scene is checked afterwards. The facts of the scenes before it come from what was already stored, and each person starts from where the last check saw them, so the check costs a few questions instead of a full 10–30 minute pass. A test fails if a rewrite ever re-checks the whole branch.

Live: chapter VIII, one click

I clicked it in the app on the real branch. The new chapter opens with Zeena at the kitchen door and keeps her in the house. Every sentence that treated her as gone is gone, and the check passes it. It took 9.9 minutes, mostly the writer. This first rewrite also walked Zeena’s earlier scenes, because this branch’s last full check ran before sightings were saved. From now on that step is skipped.

	before	after “Rewrite this scene”
opening	“Zeena’s absence was like a door left open...”	“Zeena stood at the kitchen door...”
“Zeena gone” sentences	6	0
Zeena in the scene	away all evening	in the house throughout
length	3,629 words	3,502 words
check	flagged: needs review	passes: rewritten

Chapter VIII of the Ethan Frome branch, before and after one click (9.9 minutes on the local model). One scene shows the path works; it is not yet a rate.

What this does and doesn’t show. The whole path works on a real book: find the missing event, flag it, rewrite the scene with the rule, check it, and undo if you don’t like it. One scene is one example, not a success rate. 5w runs it on more branches and counts. The epilogue’s flag from 5u is a known false alarm. Its button is there, but the right move is to leave it alone. Code: `4ac1854c` , pushed.

5w. More branches, counted (28 Sep)

5v was one example. So I ran the whole What If path on more branches and counted. There were four branches on two books: *The Time Machine*, imported and read for this, and *Ethan Frome*. Everything went through the app’s own steps. I judged each flag real or false by reading the sentence against the branch.

branch	book	what if...	written
B10	Ethan Frome	Zeena stays home	from 5u–5v
T1	The Time Machine	Weena is never lost	3 rewritten · 23 min
T2	Ethan Frome	Zeena stays home (again)	7 rewritten · 51 min
T3	Ethan Frome	Zeena lets Mattie stay	5 rewritten · 34 min

Four branches on two books, run only through the app's own steps: branch, plan (as the planner chose), write and check, then "Rewrite this scene" on each flag. *The Time Machine* was imported and read for this (15 minutes).

A hole in the rule, fixed and re-scored on the same text

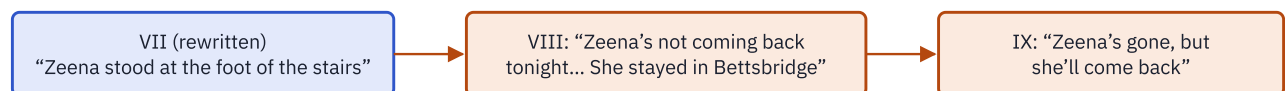
A fresh *Ethan Frome* branch wrote "Zeena wasn't coming" into chapter VIII, and it passed. The model had noticed her absence but quoted "Her things remained in their place, untouched", which doesn't name her, so 5u's rule (every quote must name the person) threw it away. Now code falls back to the scene's own sentence that puts her name right next to an absence word. It must be right next to it, so "Ethan felt Zeena's absence" never makes Ethan the one who's away, and "Zeena only looked away" isn't an absence. I scored both rules on the same saved text, so the comparison is fair.

	real	false alarm
rule as shipped (5u)	none	☐ ☐
+ name-next-to-absence fallback	☐ ☐	☐ ☐
same saved text, one model answer per person per scene, 46 reads		
new flags: "Zeena wasn't coming." and "...the wind had carried Zeena's absence all winter long."		

The model saw Zeena's absence but quoted a sentence without her name, and 5u's rule threw it away. Now code takes the scene's own sentence that puts her name right next to an absence. On the same text: two more real flags, no new false alarm.

The "control" that caught the writer

The fourth branch ("What if Zeena lets Mattie stay?") was meant as a control where nobody leaves, so any flag would be a false alarm. It got three flags, and all three were real. The writer showed Zeena at home in chapter VII, then slid back to the original book, where she was away in Bettsbridge.



the premise keeps Zeena home; the writer slid back to the original book, where she was in Bettsbridge

3 flags, 3 real, 0 false alarms

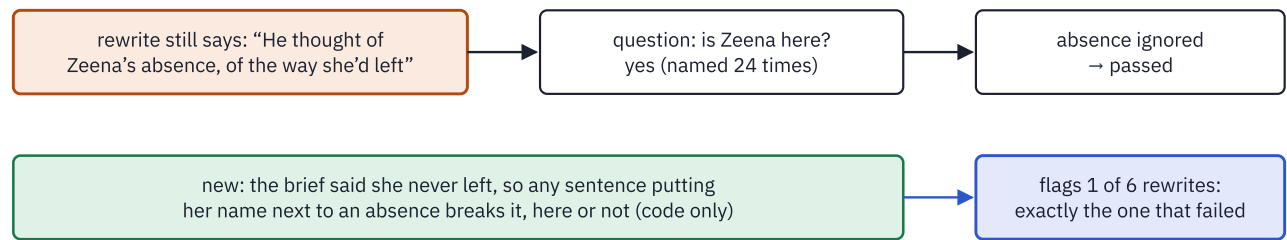
The third branch was meant as a control where nobody leaves. The writer made Zeena leave on its own, and the check caught all three places.

Does one click fix it?

scene	flag was	after "Rewrite this scene"	minutes
B10 VIII	real	fixed	9.9
T2 VIII	real	fixed	7.3
T2 IX :1	real	fixed	4.8
T3 VIII	real	not fixed → 2nd try fixed	8.0 + 8.1
T3 IX :1	real	fixed	7.0
T3 IX :2	real	fixed	7.2
T2 IX :2	false (a memory)	harmless	4.1

Six real flags, six fixed: five on the first rewrite, one on the second. The rewrite of a false alarm did no harm.

The one rewrite that failed exposed a blind spot in the re-check. The new chapter had Zeena present *and* still said “He thought of Zeena’s absence”. The who-is-where question only counts an absence when the person isn’t there, so it passed. After a rewrite that’s the wrong test, because the brief said she never left. A code check now looks for that directly. With it, the failure showed as “needs review”, and a second click fixed it.



The question only counts an absence when the person isn’t in the scene. After a rewrite that’s the wrong test: the rule was “she never left”. The new code check flags exactly the rewrite that failed and none of the five that held.

The tally

- **Real missing events (found by reading): 9. Flagged: 6. Fixed: 6** (5 on the first rewrite, 1 on the second), each taking 5–10 minutes.
- **Missed: 3.** *The Time Machine*'s kept chapter only *implies* Weena is gone (“where I had saved Weena, and that suddenly gave me a keen stab of pain”). Two *Ethan Frome* scenes say Zeena is gone while she's in the room. That's a contradiction inside one scene, which this between-scenes check doesn't look for.
- **False alarms: 3** across four branches: a habit (“a day when Ethan's off somewheres”), a memory (“how she had laughed when he first brought her home”), and someone merely in the next room. Each is only a “needs review”. Nothing is edited unless you click.

What these numbers can and can't say. Nine events on four branches are enough to find the holes (two are now fixed, both measured on saved text), not to quote a rate. The pattern is clear, though. When the check flags a real missing event, a rewrite fixes it, and the new rule check catches the rewrite that doesn't. The writer itself drifts back to the original plot, which makes this check more useful than expected. What was left: false alarms about habits, memories and the next room (fixed in 5x), and contradictions inside one scene. Code: `c157c02c`, `d55adb65`, pushed.

5x. Fixing the false alarms (28 Sep)

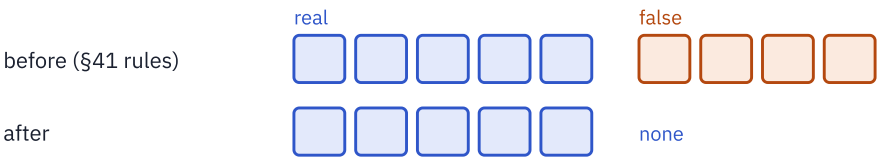
5w counted three false alarms. Recording the answers found a fourth. Every one of them is about the person at *another time*, not now. The prompt already asks the model to leave habits and memories out, and an 8B model doesn't, so code now does.



set aside → the scene's own “Zeena’s absence...” stands in, if it has one

All four false alarms describe the person at another time: a habit, the past, a memory, a moment ago. The prompt already told the model to skip these, and it didn't, so code does. A plainly-gone sentence is never set aside, so "He thought of Zeena's absence" still counts.

Measured before the fix, scored after it



74 answers recorded before the fix (four branches, incl. text before each rewrite), re-scored after it · in-sample

Before changing any code I recorded every answer the model gave on the four trial branches, then scored those same answers with the new code. All five real flags stay and all four false alarms go. The rule was written from those four sentences, though, so this proves it works on them, not beyond them.

A fresh branch the rule had never seen

The Time Machine, "What if Weena stays behind with the Eloi?", 59 minutes, run with the new rule. There were no habit, memory or past false alarms. It raised three flags, and they found something new: the planner had **kept** two chapters where, in the original, Weena runs at the Traveller's side. The story really was inconsistent, but the flags blamed the rewritten chapters, which are the ones following the change. A rewrite there would have undone the change.



before: flag on *In the Darkness*, the side that follows the change; a rewrite would force Weena back in

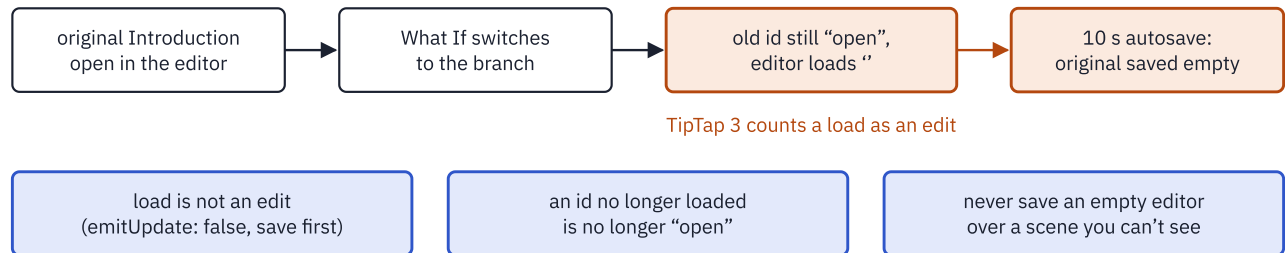
now: the sighting came from a KEPT scene → the flag says "the Palace of Green Porcelain is the scene to rewrite if the change means Weena is gone", and it is never a rule for rewriting *In the Darkness*

on every flag so far: 5 real flags → last seen in rewritten scenes (unchanged) · this branch's 3 → last seen in kept ones

A fresh branch, never seen by the rule: no habit or memory false alarms. Its three flags found a real conflict, but in the planner's kept chapters, not the rewritten ones. The flag now names the kept chapter as the one to fix.

A data-loss bug, found on the way

That same branch began by “rewriting” the Introduction, a chapter before the change. The reason was serious: the **original book’s** Introduction had been saved empty, ten seconds after an earlier run switched the editor to its branch. It’s restored now (1,680 words, from the branch copy made at the fork), and fixed.



live: same sequence, 12 s later the original is untouched; typing still saves to the right scene

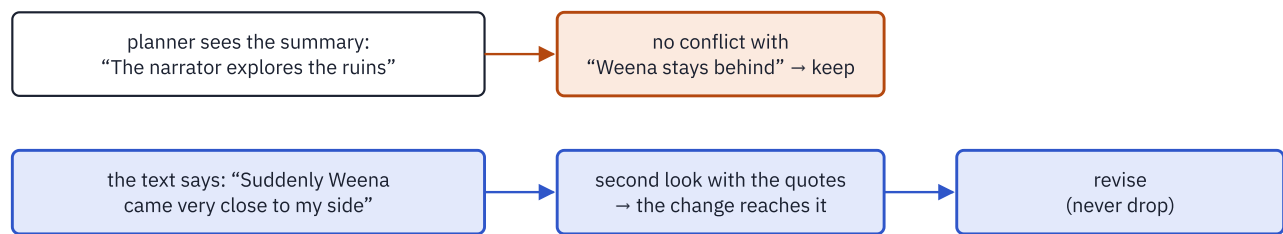
Found because a fresh branch started by “rewriting” a chapter before the change: the original book’s first chapter had been saved empty during an earlier run. It was restored from the branch copy, and three small fixes make sure it can’t happen again.

Where this leaves it. The habit, memory and past false alarms are gone on the text they came from, and none appeared on a fresh branch. The new failure the fresh branch showed, blaming the wrong side of a real conflict, now points at the right scene. The weak link it exposed is upstream: the planner kept chapters the change clearly reaches, and the fact check didn’t catch them. That’s fixed in 5y. One fresh branch isn’t a rate. Code:

14bd436d , 348c8ca7 , pushed. All 3,358 tests pass.

5y. Fixing the planner (28 Sep)

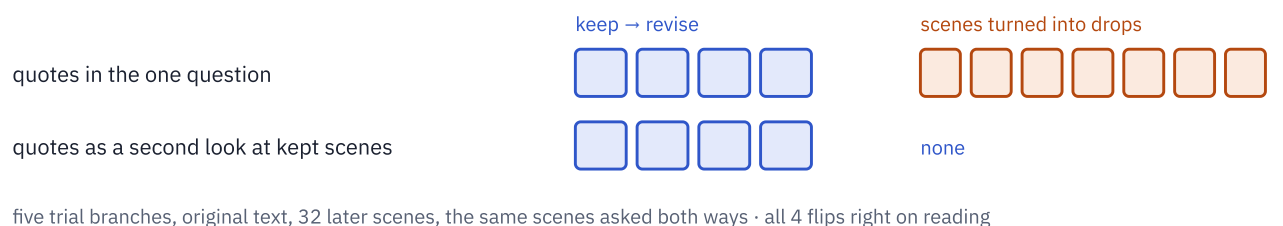
5x found the weak link upstream: for “What if Weena stays behind?”, the planner kept two chapters where Weena runs at the Traveller’s side. The cause was simple. It decided each chapter from a one-line summary, and summaries leave out the people.



A one-line summary drops the people. A scene the planner wants to keep now gets a second look at its own sentences about the people the change names.

Two ways to show it the text, measured

The first question stays as it was (summary only), so nothing the planner already gets right changes. A chapter it wants to keep is asked again with its own sentences about the people the change names, quoted. I scored both designs on the same 32 chapters from the five trial branches, using the original text.



Shown the quotes in its only question, the 8B model found the same four scenes, but it also turned seven scenes it meant to revise into drops, and a drop deletes a scene. So the quotes are a second look, and it can only move keep to revise.

The Time Machine · Weena never lost	Trap of the White Sphinx	"a keen stab of pain" — last round's miss
The Time Machine · Weena stays behind	The Morlocks	"Little Weena ran with me"
	The Palace of Green Porcelain	"Weena came very close to my side"
	Trap of the White Sphinx	grief for a Weena who is alive

The four scenes the second look moved from keep to revise. The first is the chapter the who-is-where check could only miss, because it implies the loss without stating it. It is now caught one step earlier, when planning.

Live

Explanation: revise	The Morlocks: revise ← was keep
When Night Came: revise	The Palace: revise ← was keep
In the Darkness: revise	Trap of the Sphinx: revise ← was keep
The Further Vision: keep	Traveller's Return: revise
After the Story: revise	Epilogue: keep

The app's planner on a fresh branch with the same premise, 1.4 minutes. The three chapters it used to keep are now rewritten. The two that never mention Weena stay kept, and nothing is dropped.

Where this leaves What If. The chapters the change reaches are now found at planning, including one the later check could only miss. The flags that blamed the wrong side (5x) came from exactly these kept chapters, so their cause is fixed. The kept-scene warning stays as a safety net for what the planner still misses. Four flips on 32 chapters show the fix works, not how often it will. A planning run adds one question for each chapter it keeps that names the people (a few seconds each). Code: `61e34ce3` , pushed.

5z. The fixed planner, written and counted (28 Sep)

I wrote the branch the new planner planned in 5y (eight chapters, 78.6 minutes) and ran the full check on it.

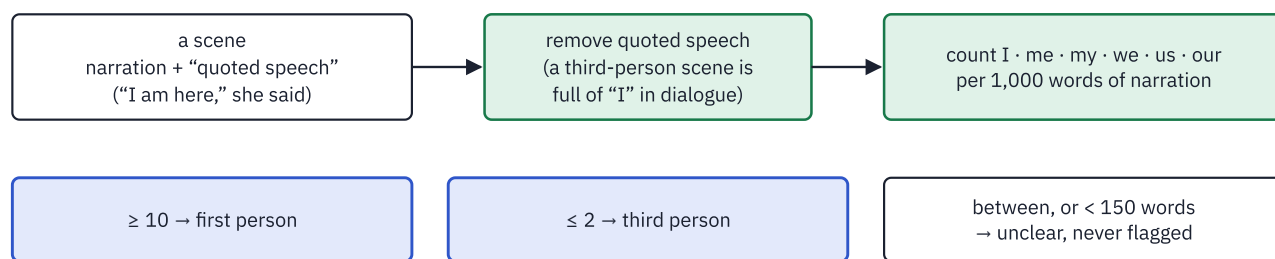
	old planner (5x)	new planner (5y)
chapters kept that contradict the change	2	0
chapters rewritten	5	8
flags	3 (wrong side)	0
Weena with the Traveller after the change	yes, in kept chapters	nowhere (28 sentences read)
time	59 min	78.6 min

The same premise, “What if Weena stays behind?”, planned and written twice. With the new planner, nothing that contradicts the change is kept, and the check has nothing to flag. Zero flags here is correct: every Weena sentence is a memory, an absence or an echo.

- **0 flags, and correctly so.** I read all 28 Weena sentences in the ten later chapters: “No trace of Weena”, “Weena’s absence was a wound”, “the memory of Weena’s touch”. She is never with the Traveller again, so there’s no missing event to find.
- **Not tested here:** the false-alarm filter from 5x. No chapter after the change shows Weena present, so memories had nothing to be flagged against.
- **Writer slips that no check looks for:** several chapters drift from the book’s first person into third (“his throat”), and “Weena’s dress lay folded in his hands” is odd if she stayed behind. That points to the next kind of check: voice (point of view) and objects, not people.
- **Also verified:** the branch switch left all 17 original chapters untouched (5x’s editor fix, live).

5aa. Point-of-view drift (28 Sep)

The branch written in 5z showed a flaw none of the checks looked for. Chapters of *The Time Machine*, which the Traveller tells as “I”, came back as “he” (“the scent of burning fruit lingered in his throat”). A reader notices at once, yet every check passed it, because none of them asks *who is telling the story*.



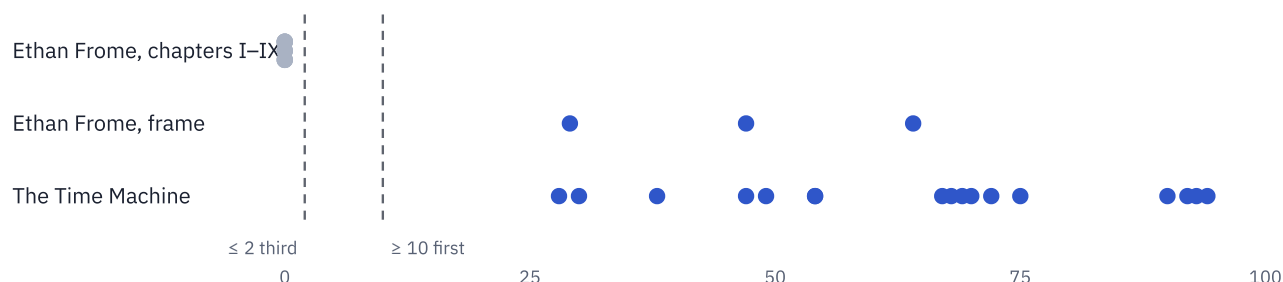
compared with the scene's OWN original: Ethan Frome has a first-person frame around third-person chapters

Plain code, no model call. The rewrite of a scene is compared with that scene's original, never with the book as a whole.

Does a pronoun count really tell first from third?

Before trusting it, I measured it on the original books, where the answer is known.

first-person pronouns per 1,000 words of narration, each original chapter one dot



On all 29 original chapters of both books the measure separates cleanly: third-person chapters score exactly 0 once speech is removed, first-person ones 28 to 94. Nothing lands in between.

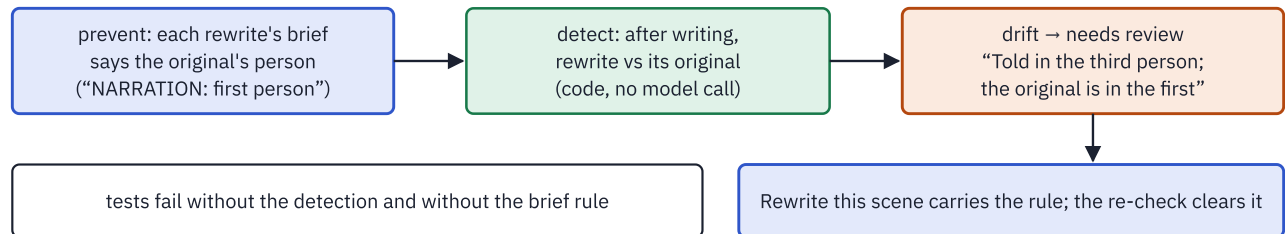
What it finds in the rewrites



34 rewrites on six trial branches · 11 drifts, 23 unchanged, 0 unclear · every drift real on reading

Run on every rewrite the trial branches produced: eleven drifts, all real, and nothing flagged that should not be. One sample: the epilogue, told by the narrator as “I”, came back as “He reached for it but hesitated”.

In the app



Branch 16's seven drifts came from the first write, so the rule now goes into every first write too, not only into a repair.

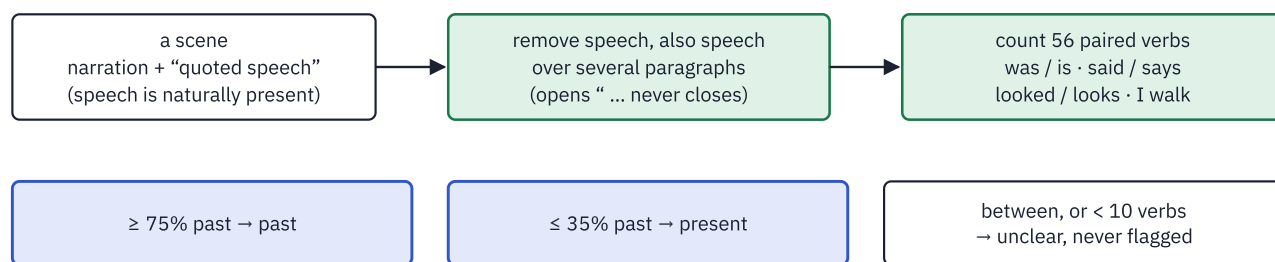
before (third)	after “Rewrite this scene” with the rule (first)
The Morlocks 0 per 1,000	74 · 10.8 min “The cold bit through my boots as I stepped over the broken stones”
The Palace of Green Porcelain 0 per 1,000	76 · 11.2 min “rough under my fingers even through the wool of my coat”

Two of branch 16's drifted chapters, rewritten live with the rule in the brief: both back in the first person, and both pass the single-scene re-check. Two out of two is an example, not a rate.

Where this leaves it. Point of view is now checked the way it should be: by code, cheaply, against each scene’s own original. On the text we have it makes no mistakes, and a rewrite with the rule fixes a drift. The same approach (a plain measure, calibrated on the originals) fits the next writer slips that nothing checks yet, such as tense and objects that shouldn’t exist (“Weena’s dress lay folded in his hands”). Code: `43f292c3`, pushed. All 3,370 tests pass.

5ab. Tense drift (29 Sep)

The companion of 5aa. A rewrite told in the present where the book tells it in the past (“She walks to the door”) reads wrong at once, and so does a past-tense scene that slides into the present for a few paragraphs. No check looked for either.



compared with the scene's OWN original, as for point of view: some scenes really are told in the present

Plain code, no model call. The same verbs are counted in both tenses, so neither gets more chances.

One trap in the old books

A speech that runs over several paragraphs used to be printed with a quote mark at the start of each paragraph and a closing one only at the end. The first version read those middle paragraphs as narration, and “It is a law of nature we overlook” looked like a slip into the present. They now count as speech. The exception is a scene where most paragraphs look like that: *The Time Machine* is the Traveller telling his story aloud, so there it is all narration.

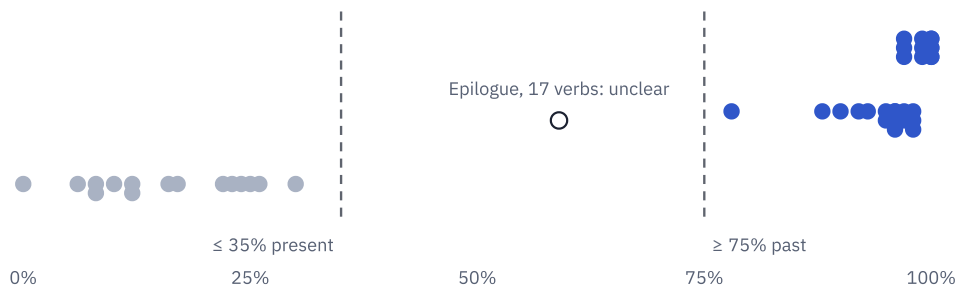
Does it tell past from present?

share of past-tense verb forms in the narration, one dot per scene

Ethan Frome, all chapters

The Time Machine

present-tense corpus scenes



The original chapters of both books are 78% to 100% past; the lowest, “The Sunset of Mankind”, muses in the timeless present. The corpus scenes told in the present are 0% to 30% past; the higher ones

remember things (“She had known this moment would come”). Nothing falls between 30% and 78%. Of the 117 other corpus scenes, all read as past.

A stretch, not only a whole scene

each box a paragraph · shaded: told in the present (≥ 4 verbs, $\leq 20\%$ past)

original	walked	walked	walked	walked	walked	walked	walked	0 present paragraphs
rewrite	walked	walked	walked	walks	walks	walked	walked	2 more than its original → needs review

why “more than its original”: The Time Machine has 3 such paragraphs of its own (“It is a law of nature we overlook...”)
the 117 past-tense corpus scenes have 0

The usual slip is a stretch, not the whole scene: the rewrite reads as past overall, so only a paragraph-by-paragraph count sees it.

What it catches

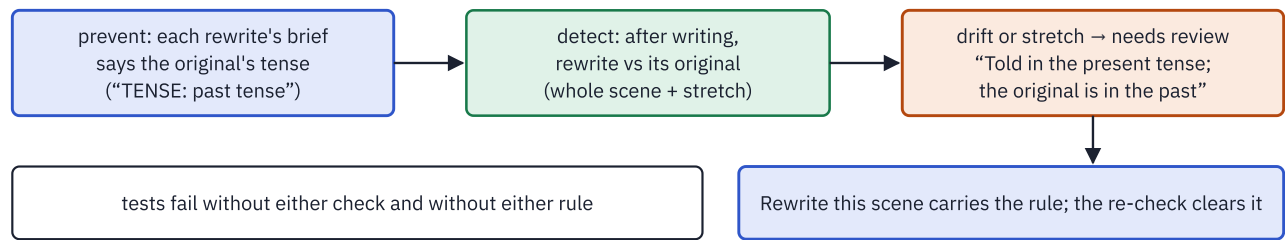
slips planted into the 29 original chapters of both books (verbs swapped: was → is, walked → walks)

whole chapter put in the present		28 / 29 caught
3 middle paragraphs put in the present		23 / 26 caught
unchanged copy of each chapter		0 / 29 flagged

missed: the Epilogue (too short to judge) and 3 stretches whose paragraphs are mostly speech (0–3 verbs of narration each)

Planted, not real: the trial branches of 5w–5aa lived in the in-app browser's storage, which was reset, so there were no stored rewrites left to count. How often the writer really slips tense is the next number to take.

In the app



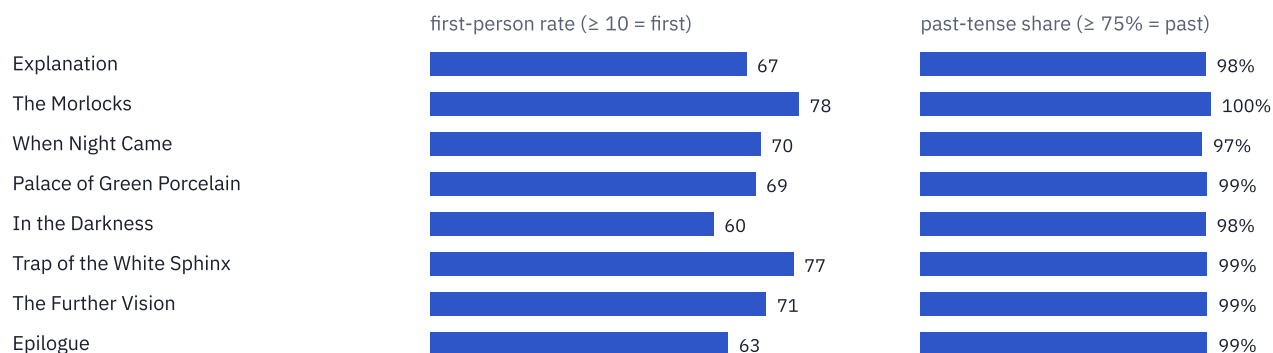
The same shape as the point-of-view check in 5aa, and it runs right after it, at no model cost.

Where this leaves it. Tense is checked like point of view: by code, against each scene's own original, with a rule in the brief to prevent it and Rewrite this scene to fix it. On the books and the corpus it raises no false alarms, and it catches almost every planted slip. What we don't know yet is how often the writer really slips tense. Branch 16's read-through found person drift, not tense drift, so the next branch written should be counted for both. All 3,378 tests pass.

5ac. A new branch, and repetition across chapters (29 Sep)

The in-app browser's storage had been reset, so both books were imported and read again (*The Time Machine* took 16 minutes: 12 characters, 22 places, 23 links). Branch 16's premise was then written again on the new code: "What if Weena stays behind with the Eloi instead of following the Time Traveller?" Planning took 7 minutes and writing 60 (branch 16 took 79). Eight chapters were rewritten, one kept and one dropped.

each rewritten chapter of the new branch · branch 16 (same premise, no brief rules): all 7 in the third person



Once each brief said “NARRATION: first person” and “TENSE: past tense”, all eight rewrites kept both. On branch 16, written on the same premise without those lines, all seven rewritten chapters had slipped to “he”. That is one branch against one branch: a strong sign, not a rate.

The only other flag was the who-is-where check on the Epilogue: “Weena’s absence gnawed through the quiet”. It is a false alarm. The next sentence is “The Eloi had taken her, or she’d chosen them”, which is exactly the premise: she is away from the narrator, still among the Eloi.

What no check saw

Reading the chapters showed something else. “When Night Came” opens with the same two paragraphs as “The Morlocks”, word for word. Measuring every pair showed it is not one slip:

longest passage two rewritten chapters share, word for word (words) · any two chapters of the original book: ≤ 12



Every chapter after the first reuses long passages of the one to three chapters written just before it: “When Night Came” opens with the first 208 words of “The Morlocks”. The writer carries its own recent

text forward. Only the first chapter written is clean.

Is 25 words the right line?

longest shared passage (words); a dot per book or run (the highest pair), per pair on this branch

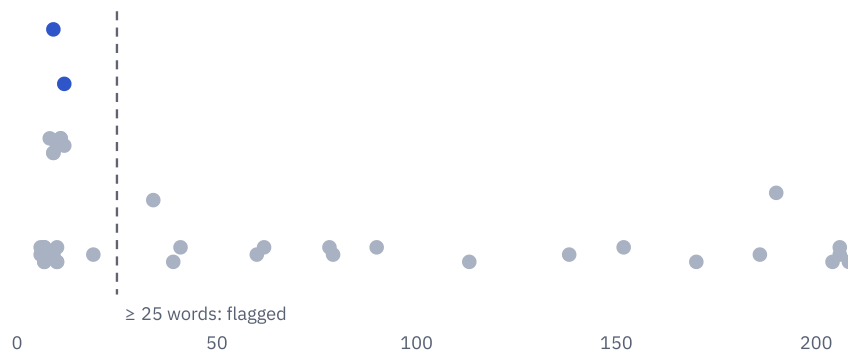
Ethan Frome, 132 chapter pairs

The Time Machine, 272 pairs

generated books, 7 runs

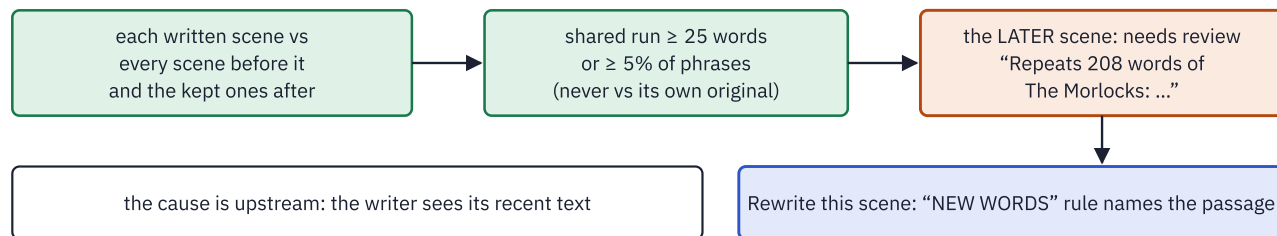
one generated book

this branch, 28 pairs



A book's own chapters never share more than 12 words in a row. The generated books mostly don't either, but one of them has two chapters sharing a 190-word passage, so the main writer copies too, not only What If. The limit (25 words, or 5% of all phrases) flags none of the original pairs.

In the app



Flagging only the later scene of a pair means one rewrite fixes it. The check is a net; the fix is in what the writer is shown, which comes next.

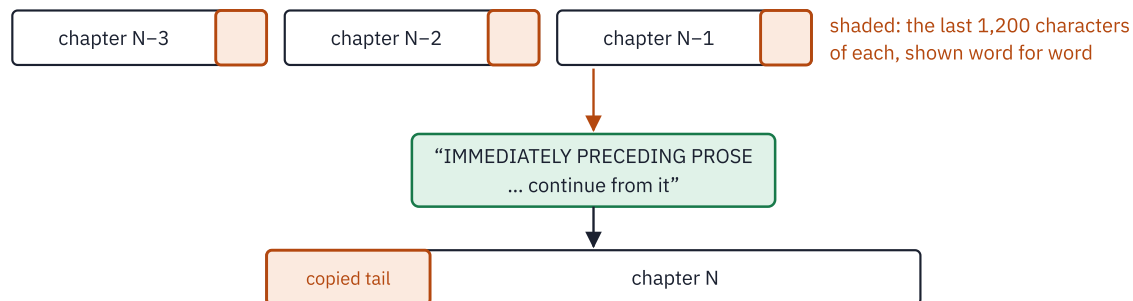
Where this leaves it. Person and tense now hold on a real branch, and copying between chapters is found by code with no false alarms on the books. But copying is the biggest flaw on this branch: 7 of 8 chapters. A net that flags seven chapters for rewriting is the

wrong fix. The next step is to find where the writer is shown its own recent chapters and stop it from copying them. All tests pass.

5ad. Stopping the copying, and a critic that never ran (29 Sep)

5ac found the writer repeating long passages of the chapters before it. The first job was to find out where it gets them.

what the writer is shown before it writes a chapter



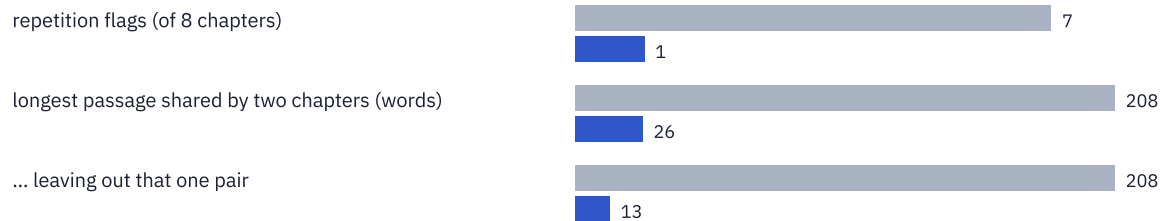
every copied passage on the branch sat 92–98% of the way through the chapter it came from: inside the shown tail

The writer was told to “continue from” three chapter endings, and it did so literally: it wrote them out again. Book drafting shows the same kind of excerpt (“Ending of Preceding Scene”), which is where a generated book’s 190-word copy came from.

The fix

There are two parts. The headers now say the text is already in the book and the new scene starts after it, without repeating it. And a guard removes, by code, any sentence of a new draft that is mostly a 15-word-or-longer passage of what the writer was shown. The guard sits in the one function every writing path goes through. On both original books and nine generated runs it removes nothing. Branch 4 is the same premise written again with the fix on:

branch 3 (before) against branch 4 (after), same premise, same book

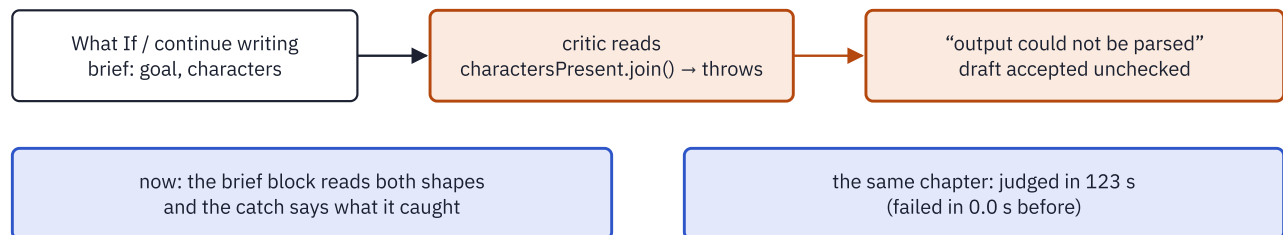


grey: branch 3 · blue: branch 4 · the original book's chapters share at most 12 words

the guard removed 158 copied sentences (2,511 words) in 12 of branch 4's 24 sections; on the books: 0

With the fix, the chapters no longer repeat each other, apart from one 26-word passage that the repetition check flags. But the guard did the work, not the new wording: it removed about as much as the old branch had copied. The model copies what it is shown, however it is labelled.

Found on the way



Found while checking the copy fix: all 24 sections of the branch were “accepted unchecked”. The cause was not the model. Continuation scenes use a different brief shape, and the critic crashed on it before asking anything. So the quality gate, and the repair step behind it, had never run on a What If or “continue writing” scene.

Where this leaves it. Chapters no longer repeat each other: one flag instead of seven, and the check catches what gets through. The cleaner fix is still to show the writer less to copy: the last paragraph of the chapter before and summaries of the rest, instead of three long endings. That can be measured by how much the guard still has to remove. Also, every branch so far was written with the critic switched off by this bug, so the next branch is the first one the quality gate will actually judge. Code: `2a1632f9` and `66d94ead`, pushed; 3,391 tests pass.

5ae. Scenes that open on the same picture (2 Oct)

A reader noticed it first: "cold wind biting into her skin like an old wound" opened three of ten scenes. The repetition check from 5ac can't see this. It looks for six-word phrases two scenes share, and a reused picture is reworded every time. What survives the rewording is the pair of content words that carries the picture: "damp earth", "fingers brushing", "boots sinking".

How often, measured

The measure is plain code. Take a scene's first sentence (20 to 40 words), drop small words and the story's character names, and keep each pair of neighbouring words. Two openings reuse a picture when they share a pair. Run over everything on disk, generated books and the chapter openings of six published ones:

openings that reuse a picture from an earlier opening

generated books (8 runs, 70 openings)

30 / 70 · 43%

of which the salt-road book (30 scenes)

15 / 29 · 52%

six published books (114 chapter openings)

14 / 114 · 12%

most repeated in the salt-road book: damp earth ×6 · fingers brushing ×4 · coarse fabric ×3 · stood at the edge ×3

the published "repeats" are mostly names and stock phrases: time machine, great deal, long time

the same measure later names what the writer reuses; it is shipped code (openingReuse.ts), not a script

Generated books reuse an opening picture three to four times as often as published ones, and the published hits are mostly not pictures at all. The measure has a human baseline before it is used to judge a fix.

Two causes

The planner hands every scene a "sensory anchor", and it repeats its own: "the scent of damp earth" is the anchor of six scenes. The writer opens on its anchor in 16 of 30. But only 5 of the 15 reusing openings take the picture from their own anchor. The rest are the writer's habits: kneeling beside something, fingers brushing, boots sinking.

Tried: tell the writer which openings are taken

The obvious fix puts the eight nearest openings in the writer's prompt under "do not open on the same picture". It was built and measured with the paired design used throughout: same six scenes, same brief, same everything, with and without the list, four tries each.

6 scenes × 4 tries, qwen3:8b · grey: no list · orange: list in the prompt

reuses an earlier opening	16 / 24	21 / 24
---------------------------	---	---

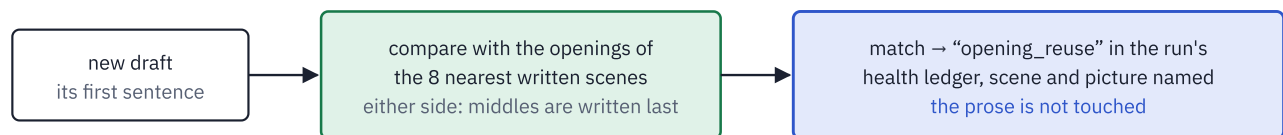
reuses one of the 8 it was shown	11 / 24	15 / 24
----------------------------------	---	---

longest stretch copied from a shown opening 2.7 words on average → 4.4, up to 19

no scene got better, 3 got worse · not significant ($p = 0.25$), but every scene that moved, moved the wrong way

Told what not to write, the writer wrote it: "sharp enough to cut through the morning haze and the scent of damp earth", 19 words lifted from a shown opening. Same lesson as 5ad: this model copies whatever it is shown, however it is labelled. The list was taken out again.

What shipped: say so, change nothing



replayed on the same books, as a run sees them: 28 / 70 generated openings flagged 12 / 114 published

on the first watched run (#68, 2 Oct): 2 flags, both real repeats; the clean scene left alone

"Ilse's boots clicked ... each step" vs "Ilse's boots crunched ... each step" · "Ilse's heel caught ..." vs "Ilse's heel caught ..."

The check is a warning, like every gate here: it never rewrites. On its first live run it caught both repeats the writer produced, and named them by scene title (an early version said "scene 1", which in a continued book meant the wrong scene; fixed).

Where this leaves it. The writer's reused openings are now counted and named, not fixed. Telling the writer made it worse, so any real fix has to change the writer without showing it the openings. The planner's repeated anchors were the remaining cause to test (5af).

Code: `e44c8883` (the experiment, kept so it can be re-run), `6b164d11`, `c829e028` ; pushed.

5af. The planner's repeated anchors are not the cause (2 Oct)

5ae left one lead: the planner gives scenes the same sensory anchor again and again, 14 of 29 in the salt-road plan. The cheapest fix needs no model at all. After each chapter is planned, clear any anchor that shares a picture with an earlier one. Before building it, the same paired test asked whether the anchor causes the reuse in the first place.

6 scenes whose anchor repeats an earlier one × 4 tries · grey: planned anchor · blue: anchor removed

opening carries the repeated picture



opening reuses an earlier opening



what it reached for instead

“stepped over the threshold” 6 → 8 · “sound cutting through” 4 → 4
“fingers brushing” 3 → 3 · “damp earth” still opens one scene with no anchor

no scene better, 2 worse · p = 0.5 · GPU to itself, 48 drafts, 0 errors

The fix does what it aims at: the repeated picture leaves the opening. It does not reduce reuse. With the anchor gone, the writer falls back on its own stock openings, and one scene still opened on "the scent of damp earth", picked up from the earlier prose in its context.

Decision: nothing shipped. Clearing repeated anchors would cost scenes a planned detail and buy no fewer reused openings. Reuse comes from the writer, not the plan. The `opening_reuse` report (5ae) stays the only mechanism, and any further work starts at the writer, without showing it what to avoid. Found on the way and fixed: the planner's rules named a character from one test story ("not every scene is Ines alone thinking") in every book's prompt. Code: `73c89282` (the probe), `affda224` ; pushed.

5ag. Two fixes the UX audit was still holding, and housekeeping (2–3 Oct)

After 5af I rechecked every item the UX audit still listed as open, against the code rather than the audit's own word. Four were already done and only needed marking: the header jargon, the organization button, the chapter and scene words in the outline, and adding a chapter by title only. Two were real, and both turned out to be more than the audit said.

The daily goal counted the whole book

The audit listed this as a lag: the goal bar caught up with the header's word count only at the 10-second save. Tracing it found the bar was not showing today's words at all. It showed a copy of the **whole manuscript's total**. Any book longer than its goal read "goal reached" before a word was written: the sample story opened at 459 / 500. A second lag sat underneath: in a chapter or scene, even the header's own count waited for the save, because a scene's words reached the store only when it was written to disk.

the sample story (459 words), opened, then 12 words typed into a scene

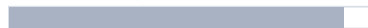
before

on opening



459 / 500 · nothing written

1 s after typing



still 459 / 500 · moves at the 10 s save, to 471

after

on opening



0 / 500

1 s after typing



12 / 500 · before "Saved" · header 471 words · streak 1

today = the live total – the last total saved on an earlier day (0 for a new book; what an import or the sample arrived with)

Measured in the app. Today's words are now the same day-over-day difference the workspace's writing history already used, so the header and the workspace agree. The open scene's count is live, and the streak counts today from the first word, not the first save.

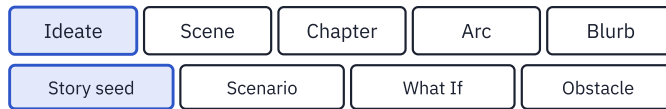
Two smaller things fell out of it. The editor counted words in a way that ran paragraphs together ("sky.Ilse" was one word), so the live count and the saved one disagreed; both now count the same way. And the sample story now records the words it arrived with, as an imported book does, so its 459 words are never counted as written on the day it was opened.

The Generator had two levels of mode

The panel opened on Ideate, with five tabs. Ideate then had its own four-way "Prompt type" switch: two choices of mode before anything happened, both forgotten when the panel closed. The fix was already agreed in the UX backlog (example 08).

the first screen of the Generator panel

before



mode 1

mode 2 · both forgotten on close

after



one row of modes, then a brief and a button

Ideate · Blurb

the More button then names the mode;
the prompt type is chips inside Ideate

reopens on the last mode used in each project; a panel asking for a mode still wins

Ideate and Blurb are not ways of writing the book and are used far less, so they moved under More. On the way, a shared control was fixed: with no mode checked in the row (while Ideate or Blurb is open), the keyboard could not reach it at all.

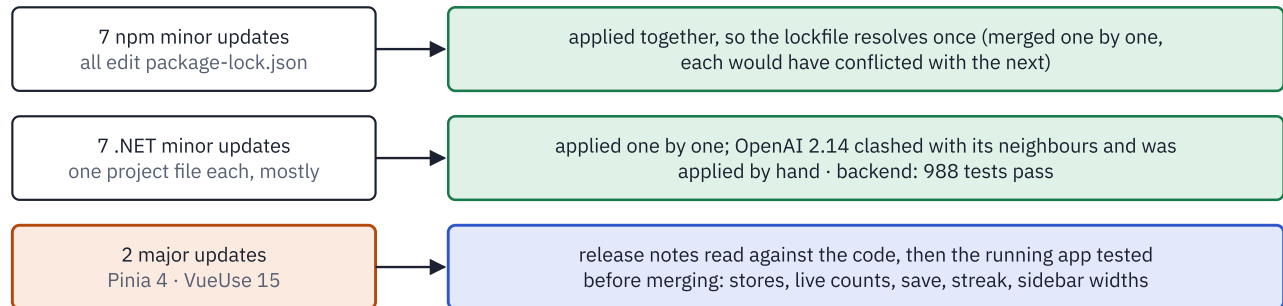
Housekeeping

- **No open issues.** The seven weekly dependency reports were stale; `npm audit fix` moved four packages up a patch version and `npm audit` now finds nothing (`9bbfee03`). With #63–#68 and #103–#107 closed earlier, the repo has no open issues.
- **Fewer untyped values.** The `any` baseline tightened for four files and lost three entries for files deleted long ago (`b957c411`).
- **Docs current.** The changelog, the UX audit (its "next pass" list now has nothing open) and the design-system catalogue are updated.

Where this leaves it. Both fixes were checked in the running app, not only in tests: the goal bar at 0 / 500 on opening and 12 / 500 a second after typing; the Generator opening on Scene, Ideate under More, remembered across a reload. Neither touches the writing pipeline, so nothing in 5a–5af changes. Code: `43c10823` , `cbbbadaa` ; pushed. All 3,468 tests pass.

5ah. Sixteen dependency updates, and a bug they surfaced (3 Oct)

Sixteen automated dependency updates were waiting, some for two weeks. Their CI had passed, but on a master that had since moved by dozens of commits. So none was merged as it stood. Each was applied to the current master and checked there.

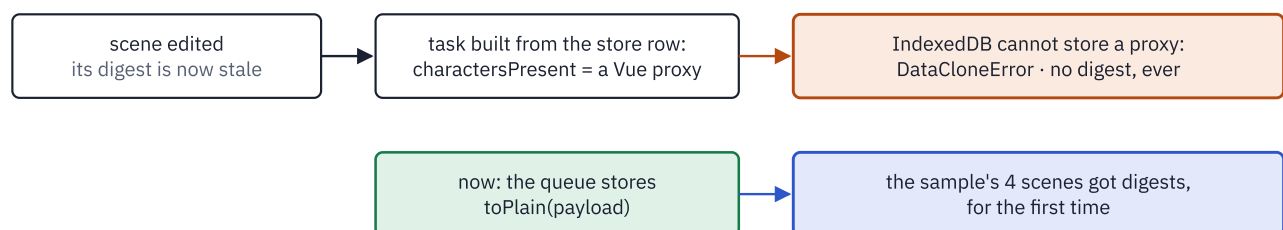


every one: typecheck, lint, policy, build, Storybook build, all front-end tests · 0 open pull requests after

None of the sixteen was merged as it stood. The two majors had small, checkable breaking changes: Pinia 4 is ESM-only and needs a newer devtools package, VueUse 15 removed functions this app does not use.

What the console showed while testing

Testing the majors in the running app meant reading its console, and two things were there. Neither came from the upgrade. A deprecation warning on every navigation: the sign-in guard used vue-router's old `next()` callback. It now returns its redirect, with the same routing. And an error on every load after a scene was edited: "Digest backfill failed: DataCloneError".



reproduced with a plain Vue ref and no Pinia: older than the upgrade

the existing tests replaced the queue with a stand-in, so a real database write never happened in them

A digest is the short summary of a scene that the later checks and the timeline read. No hand-written or edited scene had been getting one. The new test writes through a real (in-memory) IndexedDB and fails on the old code; so does the router test.

Where this leaves it. Every dependency is current, and Versatile has no open issues or pull requests. The bug that mattered was not in the upgrade but found by testing it in the running app: the tests had mocked the one call that failed. It is section 8's lesson again, "run the real thing before you believe the tests", this time on the app's own plumbing. Still open: a new security advisory on `braces`, a dev-only package with no fixed release; the only fix offered is Tailwind 4. Code: `410cfc6f ... ee3dc853`, `92cb18af`, `cee4dd0b`, `5d913324`; pushed. All 3,473 tests pass.

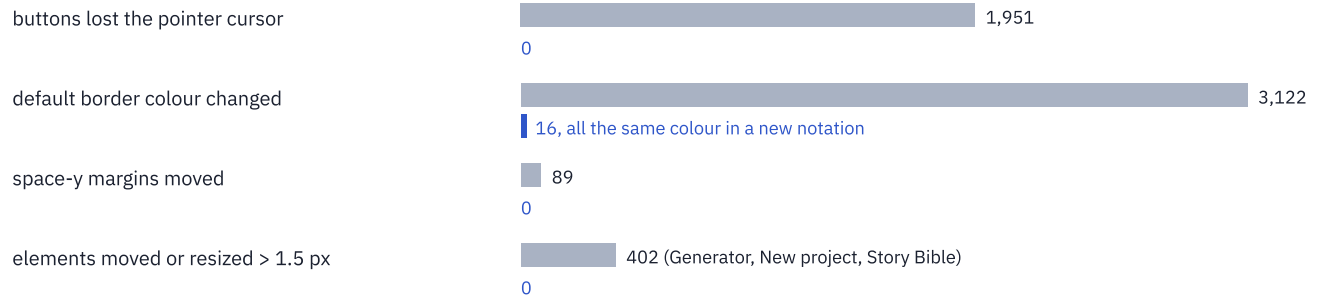
5ai. Tailwind 4, measured to the pixel (3 Oct)

One security advisory was left after 5ah: `braces`, a dev-only package with no fixed release. It reached the app through Tailwind 3 and plop, a code generator whose templates had gone stale. Plop was removed. Tailwind had to move to version 4, which changes how a dozen classes behave, and every screen in the app is built from those classes.

How "nothing changed" was checked

Screenshots compared by eye miss a 3-pixel shift. Instead, master (Tailwind 3) and the upgrade ran side by side. A script drove both through twelve screens and recorded the computed style and on-screen box of every element, up to 518 per screen. Two runs of the same version differed in nothing, so every difference between versions is real.

element-property differences, 12 screens, bars to scale · grey: straight from the upgrade tool · blue: after the fixes



left by design: colours written as oklab (same values) · divider lines drawn from the other side of the gap (same place) · one 150 ms hover curve

The fixes restored two Tailwind 3 defaults (the grey border, the button cursor) and kept `space-y` on its Tailwind 3 rule. Tailwind 4 puts the gap on the earlier child and lets a child's own margin replace it, which shifted three screens by 3–4 px.

The upgrade tool broke code

Tailwind's official upgrade tool renames classes across the codebase. It cannot tell a class from an ordinary string, so it also renamed words that only looked like classes. Every one of those edits was found by reading the diff and reverted.

rewritten by the tool

Spark store type 'outline' → 'outline-solid'

status shape 'ring' → 'ring-3'

panel id and button variant 'outline'

word lists: 'shadow' → 'shadow-sm'

copy: "the model returned no outline-solid"

what it would have broken

outline generation

the status dot's ring shape

the Outline panel, a button style

the heuristic analyser's matching

what the writer reads

A test file's data changed too ("the ring-3 of power"). None of these would fail a build, and some would pass the tests. They were found by grouping every change the tool made by old and new value, and reading the odd ones in context.

Classes that never worked, now working

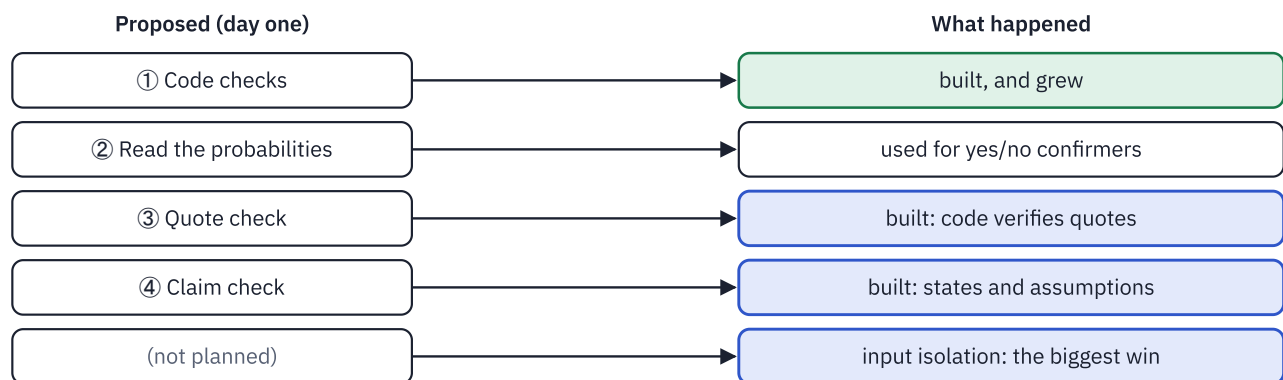
Comparing which classes each version actually generates found the opposite problem: classes written in the code that Tailwind 3 silently ignored. Faint borders such as `border-border-subtle/50` in the Research, Revise, Activity and entity panels fell back to a light

grey, and now render as written. A `z-1` on the editor was removed instead: honouring it would change how the editor's pop-ups stack.

Where this leaves it. No security advisories, current dependencies, and no element on the twelve screens moved by more than 1.5 px. Two lessons carry over. A codemod is a draft: read its diff by grouping changes, because the dangerous ones are few and look like the safe ones. And "looks the same" can be measured: a style-and-box snapshot of every element turns a visual review into a diff, with a noise floor to prove the instrument (section 8's "measure the instrument before you measure with it"). Code: `12309c27` , `f85d3d19` ; pushed. All 3,473 tests pass, and CI passed on the upgrade: lint, backend, unit tests and the end-to-end suite, its first run on Tailwind 4 (and on Playwright 1.63, whose browser was never installed locally).

6. The day-one proposal: a judge made of small, checkable pieces

This was the plan on the first morning. Section 5g shows what was actually built. The main difference: "read the probabilities" turned out to matter less than controlling what each judge reads.



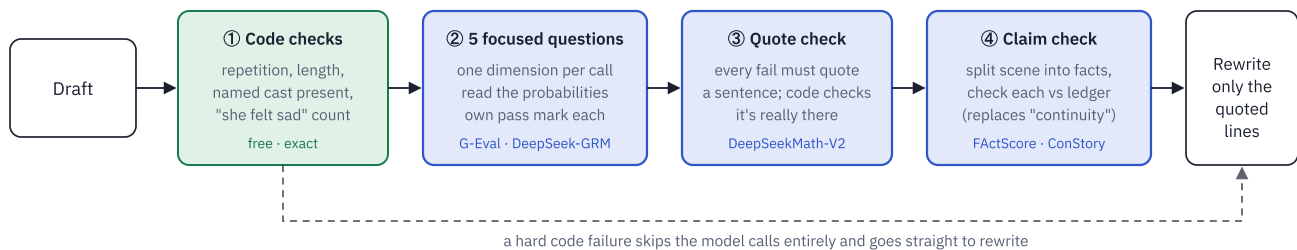
The day-one plan against what was built. Most pieces landed. The surprise was that controlling what each judge reads mattered more than reading its probabilities.

Here is what the scene judge becomes. The rule behind every piece: **ask small questions, prefer code over opinion, and demand evidence**. Top row is today, bottom row is the proposal. Both start at the same draft and end at the same decision.

TODAY



PROPOSED



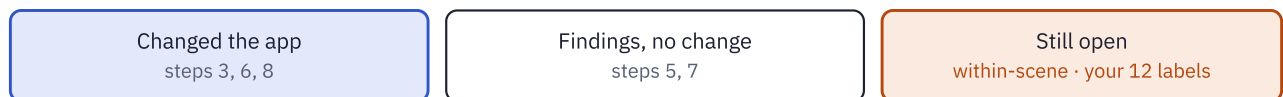
Blue pieces are new. Green is the code layer you already have, which should grow. The continuity dimension moves out of the scene critic into step ④, where your own §14 calibration already showed the contradiction checker can see a planted death.

7. How to move from here: the plan (updated 27 Sep)

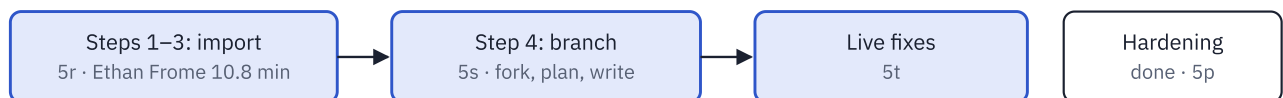
Status on 27 Sep (see 5o): every step of this plan now has a measured answer. Steps 3, 6 and 8 changed the app: a new emotional-goal judge, a better repair prompt, Editor plans, the spine as written, and a lost-facts bug fixed. Steps 5 and 7 produced findings, not changes. Still open beyond this plan: within-scene contradictions (needs a stronger checker) and the 12 scenes for you to label. **New since 27 Sep (5q):** What If on any novel, which means importing, understanding and branching. Steps 1–3 of that plan are done (5r: Versatile now reads an imported book and fills its story bible and network; *Ethan Frome* in 10.8 minutes). Step 4 is done too (5s): What If forks at a scene, plans keep / rewrite / drop per scene, writes with the real writer, checks every later scene against the change, and merges with snapshots. It was tried live on *Ethan Frome*. Fixes after the live run, and a measured limit, are in 5t. The limit is now a check: 5u follows who is where across scenes and flags chapter VIII's missing event. 5v adds a button that rewrites a flagged scene with the missing event as a rule; chapter VIII now keeps Zeena home. 5w runs it on four branches and counts: 9 real missing events, 6 flagged, 6 fixed, 3 false alarms. 5x removes

the habit, memory and past false alarms, points flags from kept chapters at the kept chapter, and fixes a data-loss bug the runs uncovered. 5y fixes the planner: a chapter it would keep gets a second look at its own sentences about the people the change names. 5z writes that branch: nothing contradicting the change is kept, and 0 flags, correctly. 5aa adds a point-of-view check: a rewrite told in “he” where the original says “I” is found by code and flagged, and the rewrite brief now names the person to keep. 5ab does the same for tense: a rewrite told in the present where the original is in the past, or one that slides into the present for a stretch, is found and flagged. 5ac writes a new branch on these checks: person and tense hold in all eight chapters, but seven of them copy long passages of the chapters before, which a new check now finds. 5ad traces the copies to the chapter endings the writer is shown and removes them by code (one flag instead of seven), and fixes the critic, which had never judged a What If scene. 5ae measures scenes that open on the same picture (43% of generated openings, 12% in published books): telling the writer which openings are taken made it worse, so the app now counts and names them instead. 5af tests the planner's repeated anchors as the cause, and they are not: removing them leaves reuse unchanged. 5ag fixes two things the UX audit still held open: the daily goal, which showed the whole book's total and lagged the save, now counts today's words as you type; and the Generator has one row of modes instead of two. 5ah lands sixteen dependency updates, each checked on the current master, and the bug that testing them in the running app found: no hand-written or edited scene had ever been getting its digest. 5ai moves to Tailwind 4 to clear the last security advisory, and measures every element on twelve screens before and after: nothing moves by more than 1.5 px. The two hardening tasks (a hung-request guard, a flaky test) are done, along with the group-drop bug found while checking whether quadtrees would help (5p: they wouldn't, at these sizes).

JUDGE PLAN



WHAT IF ON ANY NOVEL



Where the plan stood on 27 September. The judge plan has a measured answer for every step, and the What If track has gone from importing a novel to branching it live.

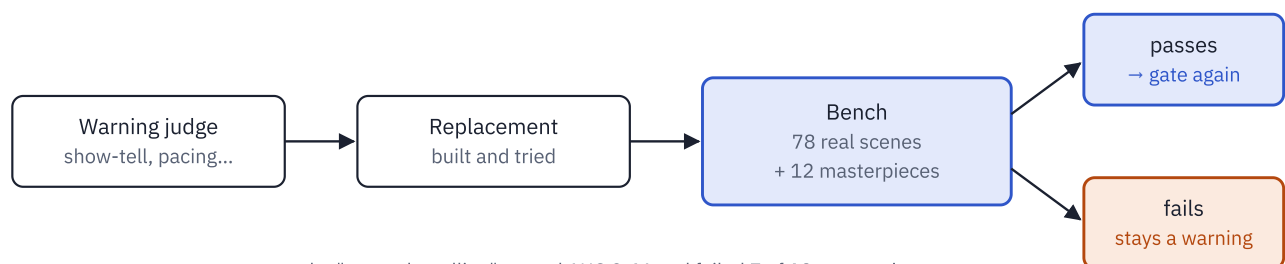
Update after the four experiments (5l): next, in order. (1) Make the voice judge vote twice, and fail only if both votes agree: cheap, and it fixes a measured flip. (2) Continuity: test the extra questions only on sentences that name someone in a fact *and* share a key word with it, and measure false alarms on real clean scenes first. (3) Within-scene: fix the "then vs now" error and re-check the masterpieces. (4) Show-tell and pacing stay as warnings until someone labels more current-pipeline scenes. Today's pipeline had only 4 show-tell problems in 48 scenes, too few to test a judge on.



cheapest first; false alarms measured on real clean scenes first

The order set after 5l. (Section 5m later showed the voice flip came from changed input, so step ① became a counting fix instead.)

Update after acting on it (5k): show-tell, pacing and emotional goal now only warn, and they come back as gates only when a replacement passes the bench. Continuity gained the "what does a sentence assume?" check. The next levers, in order: **a better continuity confirmer** (every miss dies there); **show-tell judged at the key moment only** (the "count the telling" replacement failed: AUC 0.46, 7/12 masterpieces failed); **within-scene contradictions** (parked, needs a new extraction design); pacing only if today's pipeline starts producing pacing problems, since it produced none in 45 scenes.



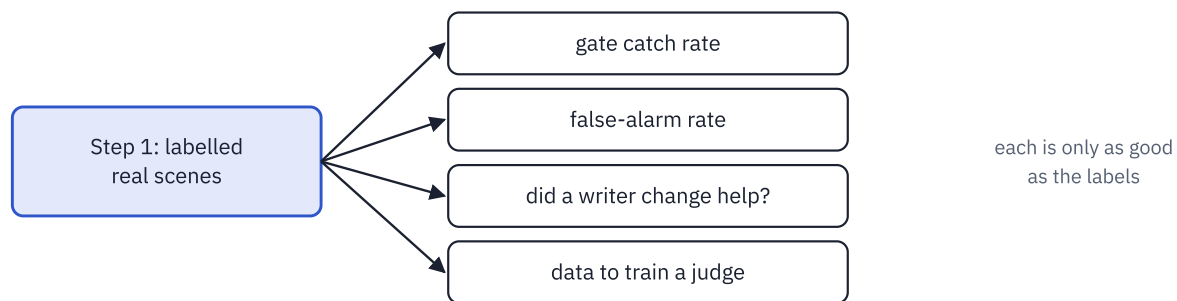
example: "count the telling" scored AUC 0.46 and failed 7 of 12 masterpieces

A warning-only judge comes back as a gate only through the bench. The first show-tell replacement didn't make it.

Update after the masterpiece control (5j): the show-tell and pacing judges reward LLM style and fail Chekhov, so they aren't being tuned any more; they're being replaced. Every step below is judged on two sets: catch the reviewers' problems in the 78 generated scenes, and pass the 12 masterpieces.

Update after the real-scene test (5i): steps 2–4 below now come first, because each one fixes a measured miss on real scenes. Continuity is first of all: presupposed contradictions plus a within-scene check, since its misses are checkable facts. Pacing thresholds get recalibrated on the reviewer labels. Every step's "done when" is now measured on the 78 labelled real scenes, not only on planted flaws.

The original plan's first five steps are done (see the box below). This is the plan from here. Order matters: step 1 comes first because **every later number is only as good as the ground truth it's measured against**. Right now that's planted defects and 30 clean scenes.



Every later number is measured against step 1's labels. If they're wrong, everything built on them is wrong too, which is why they come first.

Done since the first plan

- **Benchmark:** fails loudly and no longer grades itself (1785e9aa).
- **Gate:** input isolation, verdict lists all failing dimensions, blame once, judge calls without the repeat penalty, pacing confirmed in reverse (03b6b5f0 , 485a11dd). Caught 133/145, and on by default (b9befad8).
- **Contradictions:** the checker quotes both sides, code verifies the quotes, and a "can both be true?" confirmation follows. The chapter audit no longer rewrites good scenes (2da1aa14). The ledger result was retracted (§22).
- **Repair in place:** both writing paths (12a85df4 , b8ee3788).

1

Build a labelled test set of real scenes done: 78 scenes labelled twiceyour 12 still open (~30 min)

Status: independent reviewers labelled all 78 scenes twice (κ 0.66–0.89 between passes). Results are in 5i. Your 12 decide how far their labels can stand in for yours.

Status (25 Sep): the pool is built: 78 scenes, about 48k words, from 4 stories (the Salt Road plus new crime, sci-fi and family-drama books written with the gate *off*, so the judge under test didn't pre-filter them). The rubric is frozen in `docs/LABELLING-RUBRIC.md`. The labelling desk is live at claude.ai/artifact/3Ba7tdgwn8QAacCRdrg29W, and the gate's own verdicts are computed separately and never shown there, so your labels stay blind. `tools/labelling/compare.py` turns the two into the numbers.

Generate about 80 real scenes, then freeze a one-page rubric (what counts as filler, telling, flat voice, missed emotion, contradiction) *before* labelling. Label each scene pass/fail per dimension yourself. Re-label 20 of them a week later to measure your own consistency (a stand-in for a second human). While you're at it, confirm each planted defect type really makes a scene worse (FBI's step).



the rubric is fixed before labelling, because criteria drift as you grade

The labelling recipe. Freezing the rubric first stops the criteria drifting. Re-labelling 20 scenes later stands in for a second human.

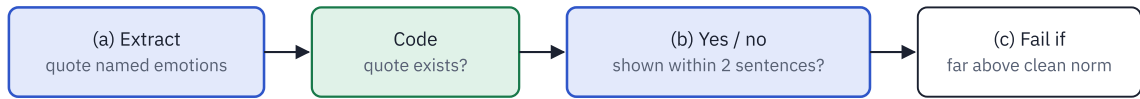
Done when: every gate number is reported against your labels with a 95% range, and the false-alarm rate's range is under ± 8 points (it's 0.6–17% today).

2

Show-tell: extract, then check 1–2 days

Add a second telling defect, **named emotions** ("she was afraid"), since today's fixture only covers summarised action. Then rebuild the judge in three steps: (a) the model quotes every sentence that names an emotion, and code checks each quote exists; (b) for each quote, one yes/no question: "is this emotion also shown by an action, gesture

or line within 2 sentences?"; (c) fail on the share of named-but-never-shown emotions, compared with what clean scenes normally have, not a fixed number.



tested on a new "named emotion" defect and on human-tagged spans

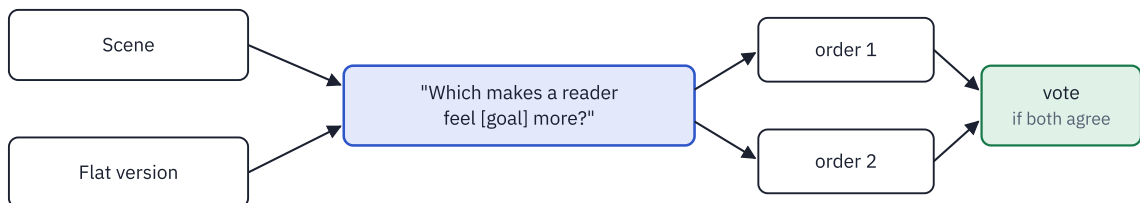
Step 2's judge: extract, verify in code, ask one narrow question per quote, and fail on the share of named-but-never-shown emotions compared with what clean scenes normally have.

Done when: named-emotion telling caught at $\geq 85\%$, faithful telling above 21/30, no rise in clean fails, measured on the held-out half and on LAMP's human-tagged exposition spans.

3

Emotional goal: judge what a reader would feel 1 day, then 1 more

First: delete the emotion-naming sentences from step 2, then ask a multiple-choice question: "what does the reader most likely feel at the end?" (the goal plus 3 wrong answers). Then, as a second vote: compare the scene against a flat version of itself, "which makes a reader feel [goal] more?", asked in both orders.



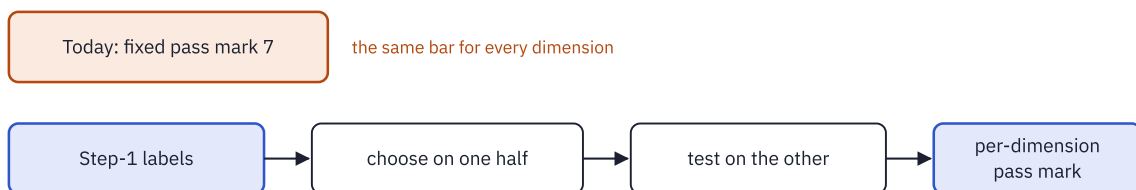
Step 3's second vote: compare the scene with a deliberately flat version of itself, asked in both orders, so the vote counts only when position bias cancels out.

Done when: agreement with your step-1 labels beats today's 1–10 score, and emotional goal no longer co-fails on the telling defects.

4

Filter off-topic evidence from whole-scene judges 1–2 days

Whatever still reads the whole scene works in two passes: collect quoted evidence, drop any quote about another dimension, then decide from what's left (the DimCheck idea). Read the model's yes/no probabilities where the question is yes/no, and set pass marks from the step-1 labels instead of a fixed 7.



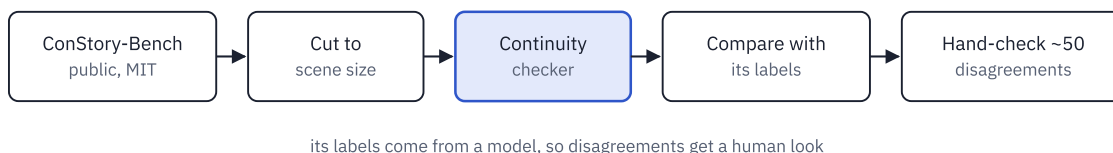
Step 4 also replaces the fixed 7 with a pass mark per dimension, chosen on half of the labelled scenes and checked on the other half.

Done when: co-failures on planted defects drop well below today's (show-tell 8/25 on flattened voice, emotional goal 26/30 on empty summaries), with catch rates unchanged.

5

Contradictions on real prose 1–2 days

Download ConStory-Bench (public, MIT), cut its stories to scene size, and measure what share of its character and fact contradictions the checker finds. Hand-check about 50 of the disagreements, because its labels come from a model. Email the FlawedFictions author for their human-verified plot holes.

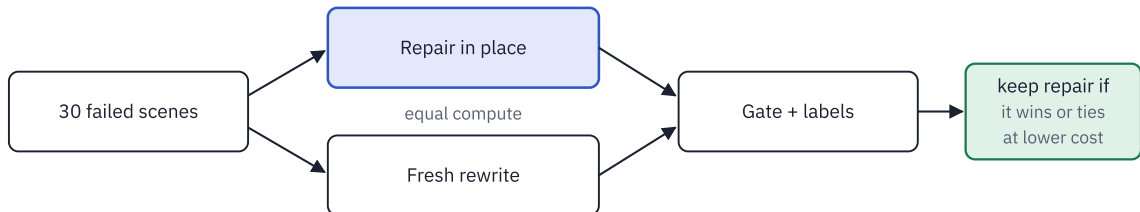


Step 5: a recall number on natural contradictions. The benchmark's labels come from a model, so the disagreements are checked by hand. (Run in 5o: the within-scene checker found almost none.)

Done when: a recall number on natural contradictions exists, next to today's 30/30 on planted ones.

6 Repair vs rewrite, fairly 1 day, mostly waiting

Take 30 scenes that failed the gate. Fix each one twice at equal compute: once by repair, once by a fresh rewrite. Judge both with the gate and with the step-1 labels. The only direct evidence so far (from code) favours rewriting.



Step 6's fair test. Result in 50: the repair ties on quality at about 1/70 of the time, so it stays first.

Done when: keep repair on only if it wins or ties at lower cost.

7 Only then: do gated books read better? 2–3 days, mostly waiting

Write 3–4 small books with the gate on and 3–4 with it off. Compare their committed scenes in pairs using a judge that *isn't* the gate (so the gate doesn't grade itself), and spot-check with your own reading.



Step 7 compares gated and ungated books with a judge other than the gate, so the gate never grades itself. (50 answered it scene by scene instead.)

Done when: a quality difference with a confidence interval, or an honest "no measurable difference".

8

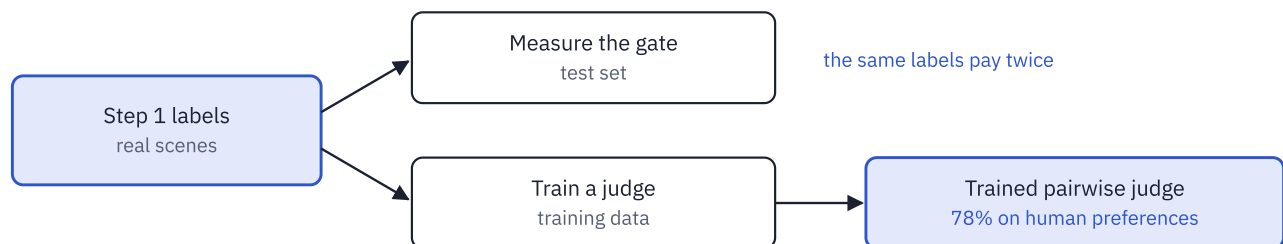
Small fixes in passing ½ day

The 3B Editor's `target: null` (make `target` a required choice of real scene ids). Make the retrieval eval harder (it scores a perfect 1.0). The chapter spine should come from written prose.

Done when: 0 illegal Editor answers in a 2×2 run, and retrieval MRR below 1.0.

Still not now: training your own judge

Fine-tuning a judge (DeepSeek-style RL, or a small LoRA) is tempting. The research agrees it's the path to beating off-the-shelf judges: trained pairwise models reach 78% on human story preferences, above every prompted judge. But it needs the labelled set from step 1. That set is also its training data, so step 1 pays twice.

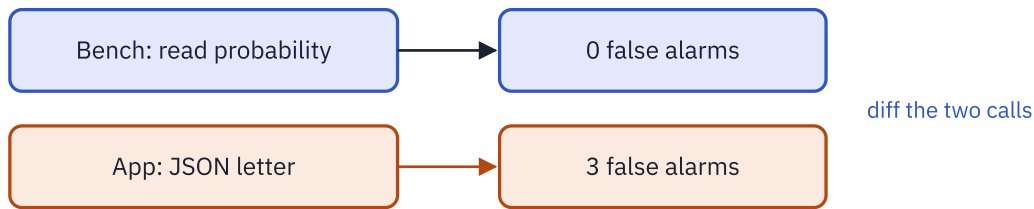


The labelled set pays twice: it measures the gate today and later becomes training data. Trained pairwise judges reach 78% on human story preferences, above every prompted judge.

8. What to take away as an engineer

Port exactly what you measured.

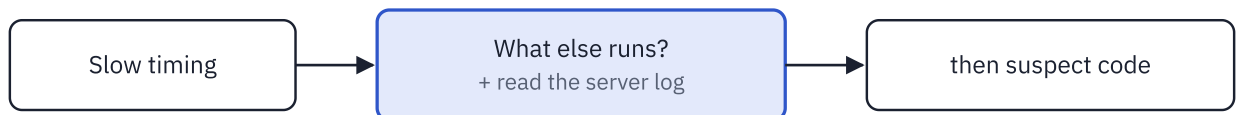
The bench read a probability; the app asked for a JSON letter. That one difference turned 0 false alarms into 3. When a result doesn't survive the move to production, compare the two calls byte for byte before blaming the idea.



One difference in how the answer was read turned 0 false alarms into 3. Compare the calls byte for byte before blaming the idea.

Check the machine before trusting a timing.

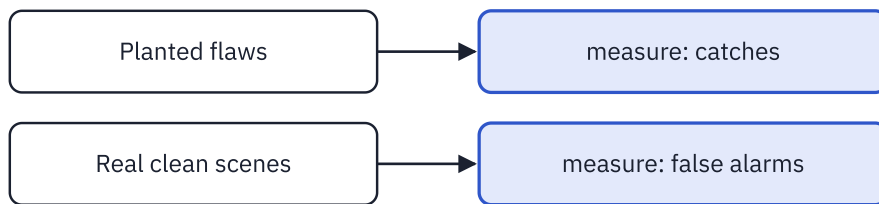
Another program sharing the GPU made every call reload the model, and the numbers looked like my code was slow. Look at what else is running, and read the server's own log.



Rule out the machine first: another program sharing the GPU can make your own code look slow.

Planted mistakes measure what you catch, not what you wrongly flag.

A continuity upgrade passed two planted sets with no false alarms, then flagged three harmless sentences in real scenes. Measure false alarms on real clean data, always.

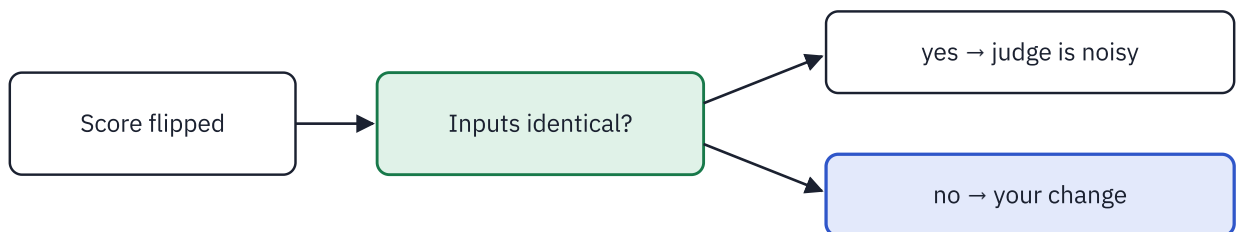


both needed

Each test set answers one question. False alarms only show up on real clean text.

Make sure the input really is the same before calling a judge unstable.

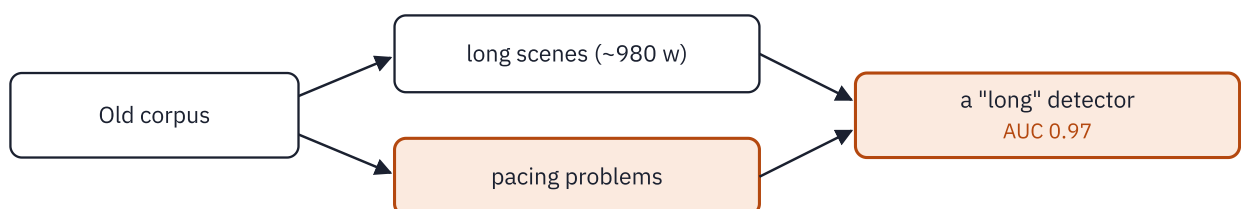
I blamed the voice judge for flipping. In fact my own fix had changed what it saw. Diff the inputs first.



Diff the inputs first. Here the "unstable" judge was scoring two different inputs, created by my own fix.

Ask where your labels came from before you trust a score on them.

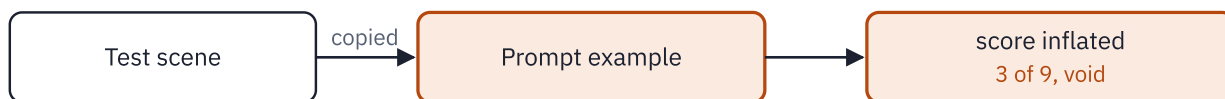
Every reviewer pacing problem came from one old batch of long scenes. A judge that only spotted "long" would have looked excellent. Split any test set by source before reading its numbers.



Every reviewer pacing problem came from one batch of long scenes, so spotting length alone scored an AUC of 0.97. Split test sets by source before trusting a score.

A prompt example can leak the answer.

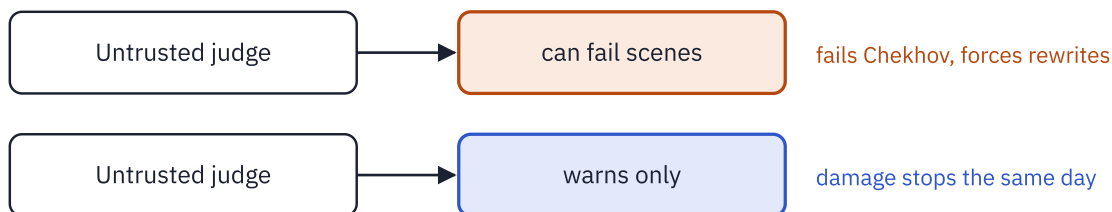
The first continuity result looked good because its example sentence was copied from a test scene. Write examples from scratch, and never from the data you're scoring on.



An example sentence copied from the test data hands the model the answer. Write examples from scratch.

If a judge can't be trusted, stop letting it decide.

Three judges were failing good scenes and Chekhov. Turning them into warnings cost nothing and stopped the damage the same day, while better replacements are still being built.



Demoting a judge to a warning costs nothing and stops the damage while a replacement is built.

Benchmark at your real size before reaching for a clever structure.

A quadtree answers "what's on screen" in 6 μ s at 50,000 nodes. But at 150 nodes the plain loop takes 2 μ s, and building the tree costs more than every query it saves. Know your N first. Reading the code to answer the question still found a real bug.

Screen query at 150 nodes, our real size

plain loop  2 μ s

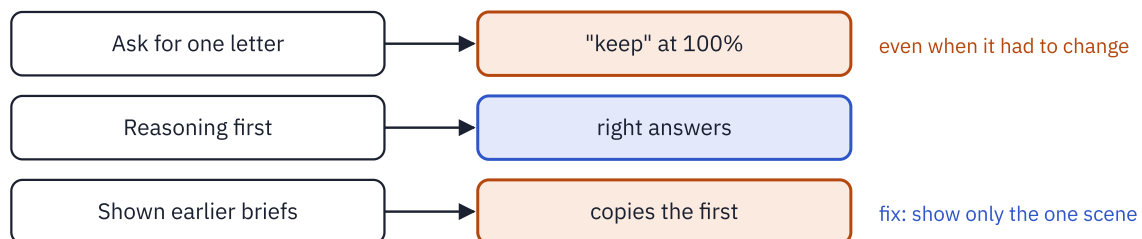
quadtree + build  89 μ s

the tree only wins in the thousands (6 μ s at 50,000 nodes)

Know your N first. At Versatile's size the plain loop beats a quadtree once building the tree is counted.

Let a small model think before it answers, and don't show it other answers.

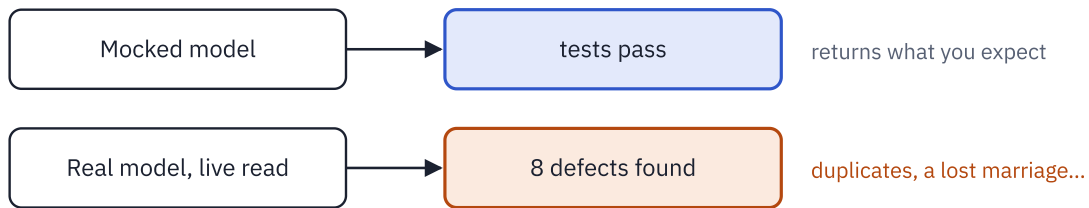
Asked for one letter, the 8B model said "keep" at 100% for scenes that obviously had to change. Asked for its reasoning first, it got them right. Shown the briefs it had already written, it copied the first into all the others. Shown only the one scene, it wrote a fitting brief for each.



Two ways a small model goes wrong: forced to answer instantly, and shown its own earlier answers. Reasoning first, and one scene at a time, fixed both.

Run the real thing before you believe the tests.

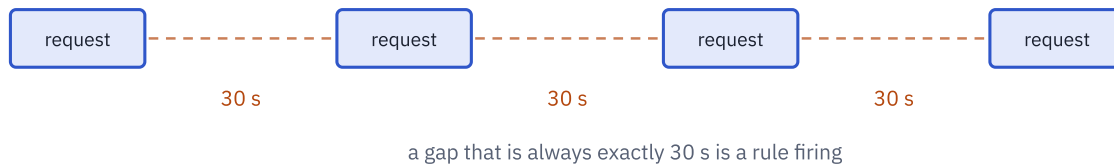
The analysis passed its tests the first time. Then one live read of *Ethan Frome* found eight defects: duplicated records, a silently dropped marriage, a husband merged into his wife. Mocked models return what you expect. Real ones don't.



One live read of *Ethan Frome* found eight defects that passing tests had hidden.

Look at the idle time, not just the busy time.

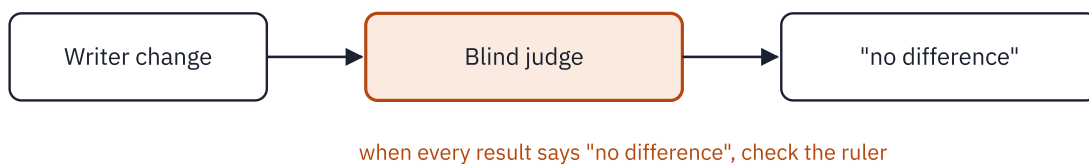
Every request looked reasonable. The waste was in the *gaps*: exactly 30 seconds, every time. A regular gap is a rule firing, and here the rule was pausing the program for itself.



The waste was in identical 30-second gaps between reasonable requests, a sign that some rule was pausing the program.

Measure the instrument before you measure with it.

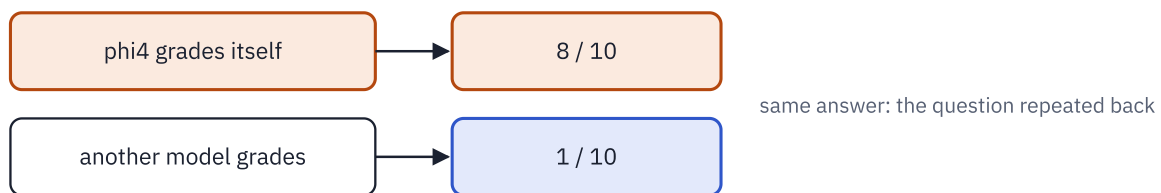
Three probes found "no effect". The effects may have been there all along, with a ruler that couldn't see them. When every result is "no difference", check the ruler first.



Three probes found "no effect" through a judge that couldn't see defects. The ruler has to be checked before any reading means anything.

Never let a model grade its own homework.

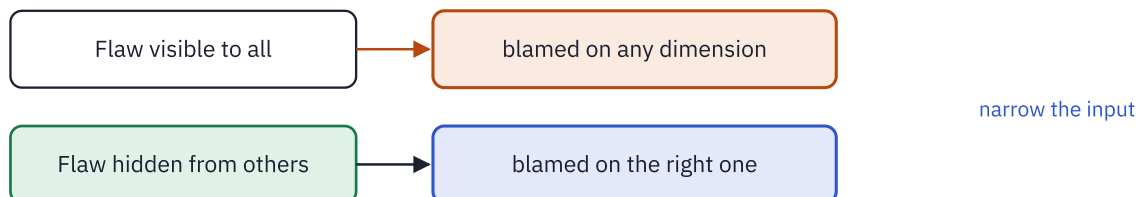
phi4-mini gave itself 8/10 for repeating the question back. A different model gave it 1/10. And "success" must mean something: an exit code of 0 after 28 errors is a lie that CI believes.



The same empty answer scored 8/10 from its own model and 1/10 from an independent one.

What a judge can see, it will use: so control what it sees.

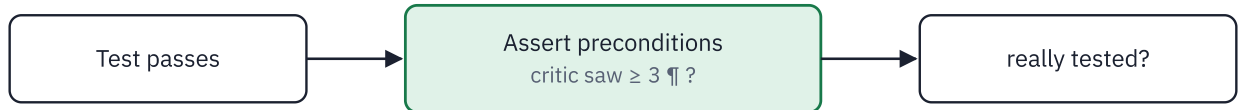
The judge found the planted flaw every time, then blamed it on whatever it was asked about. Better instructions didn't fix that. Hiding the flaw from judges it doesn't concern did. Narrow the input, not just the question.



Narrow the input, not just the question: a flaw a judge never sees can't be blamed on it.

A test that passes on the first try deserves suspicion.

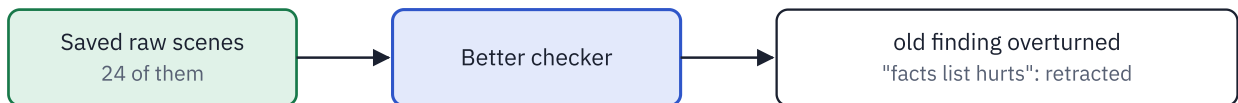
The repair test passed immediately, because it tested nothing: the fake input never reached the code path. Assert the preconditions ("the critic saw at least 3 paragraphs"), not just the outcome.



Assert that the path under test was reached, not just that the outcome looks right.

When the ruler improves, re-measure the old conclusions.

"The facts list makes contradictions worse" was a finding for a week. It was the old judge's false alarms. Saving the raw outputs (the 24 scenes) is what made it cheap to overturn. Keep the evidence, not just the counts.



Keeping the evidence, not just the counts, made it cheap to overturn a week-old conclusion.

Before calling it a false alarm, read the evidence.

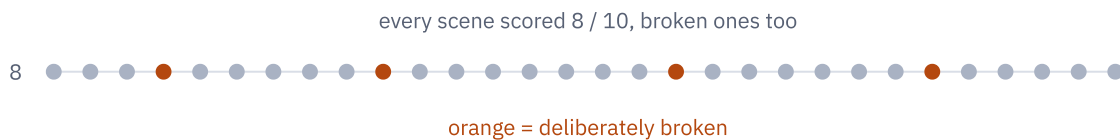
The last "false fails" weren't false. The judge kept failing the same flat dialogue on every draw, and reading it showed it really was flat. Your test data is only as good as what you know about it: "not broken by me" isn't "good".



A failure that repeats on every draw is a signal. Reading the scene showed the judge was right.

A number that never changes isn't a measurement.

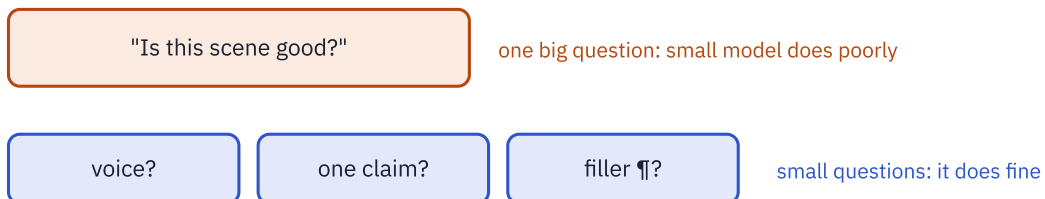
The old critic gave 8/10 to 30 different scenes, including broken ones. A constant tells you nothing. Watch for it anywhere a model produces a score.



Schematic of the old critic's scores across 30 scenes: a flat line. A constant tells you nothing about any scene.

Ask one small question at a time.

One question per call fixed voice and show-tell detection. One claim per call is the same idea for continuity. Small models are decent at small questions and bad at big ones.



One dimension per call fixed voice and show-tell detection. One claim per call does the same for continuity.

The model knows more than it prints.

A printed "7" can hide "30% sure it's a 5". Reading the probabilities is DeepSeek-GRM's voting idea in a single call, for free.

10 separate calls



7 of 10 say 7, 3 say 5

same as

1 call, read the probabilities
 $P(7) = 0.7 \cdot P(5) = 0.3$

Schematic. The probabilities of one call carry the spread that many separate votes would show, so a printed 7 can't hide the 30% doubt.

Code beats opinion wherever code can reach.

This is DeepSeek-R1's central lesson, and your repetition guard proved it here. Keep pushing checks down into code, and keep the LLM for what code can't see.

LLM judge: only what code can't see (taste, meaning)

↑ what passes

Code: repetition, quotes exist, dialogue counts, lengths

Push every check that code can do into code: it is exact, free and can't be flattered. The model sits on top, for the rest.

